Tina4 for Python Developers

This is not a framework

Andre van Zuydam

v3.1.1 - 2026

Table of Contents

1. Chapter 1: Getting Started with Tina4 Python
2. Chapter 2: Routing
3. Chapter 3: Request and Response
4. Chapter 4: Templates
5. Chapter 5: Database
6. Chapter 6: ORM
7. Chapter 7: Authentication
8. Chapter 8: Middleware
9. Chapter 9: Sessions and Cookies
10. Chapter 10: Swagger / OpenAPI
11. Chapter 11: Queues
12. Chapter 12: Real-time with WebSocket
13. Chapter 13: Email with Messenger
14. Chapter 14: Caching
15. Chapter 15: Frontend with tina4css
16. Chapter 16: GraphQL
17. Chapter 17: Testing
18. Chapter 18: Dev Tools
19. Chapter 19: CLI & Scaffolding
20. Chapter 20: Deployment
21. Chapter 21: Building a Complete App
22. Chapter 22: Vibe Coding with AI

Chapter 1: Getting Started with Tina4 Python

1. What Is Tina4 Python

Tina4 Python is a zero-dependency web framework for Python 3.12+. One package. Under 5,000 lines of code. Routing, ORM, template engine, authentication, queues, WebSocket, and 70 other features -- all built in.

It belongs to the Tina4 family -- four identical frameworks in Python, PHP, Ruby, and Node.js. Everything you learn here transfers to the other three. Same project structure. Same template syntax. Same CLI commands. Same `.env` variables.

Tina4 Python follows Python convention: `snake_case` for methods and functions (`fetch_one()`, `soft_delete()`, `has_many()`), `PascalCase` for classes, `UPPER_SNAKE_CASE` for constants. Route handlers are `async def` functions decorated with `@get`, `@post`, and friends.

By the end of this chapter, you will have a running Tina4 Python project with an API endpoint and a rendered HTML page.

2. Prerequisites and Installation

What You Need

Three tools. Nothing else.

1. **Python 3.12 or later** -- check with:

```
python3 --version
```

You should see output like:

```
Python 3.12.3
```

If you see a version lower than 3.12, upgrade Python first.

2. **uv** -- a fast Python package manager and project tool. Check with:

```
uv --version
```

You should see:

```
uv 0.6.9
```

If uv is not installed, get it from https://docs.astral.sh/uv/:

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

3. **The Tina4 CLI** -- a Rust-based binary that manages all four Tina4 frameworks:

```
# macOS (Homebrew)
brew install tina4stack/tap/tina4

# Linux / macOS (install script)
curl -fsSL https://raw.githubusercontent.com/tina4stack/tina4/main/install.sh | bash
```

```
# Windows (PowerShell)
irm https://raw.githubusercontent.com/tina4stack/tina4/main/install.ps1 | iex
```

Verify the CLI is installed:

```
tina4 --version
```

```
tina4 0.1.0
```

Creating a New Project

One command. One package. No dependency tree.

```
tina4 init my-store
```

You should see:

```
Creating Tina4 project in ./my-store ...
  Detected language: Python (pyproject.toml)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created src/migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-store
// ...
```

Install the Python dependencies:

```
cd my-store
uv sync
```

```
Resolved 1 package in 0.8s
Installed 1 package in 0.3s
 + tina4-python==3.0.0
```

One package. No dependency tree. No version conflicts. Just `tina4-python`.

Starting the Dev Server

```
tina4 serve

 _____ _             _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

  Tina4 Python v3.0.0
```

```
  Server running at http://0.0.0.0:7145
  Debug mode: ON
  Database: sqlite:///data/app.db
  Press Ctrl+C to stop
```

Open your browser to `http://localhost:7145`. The Tina4 welcome page greets you.

Open `http://localhost:7145/health` in your browser or curl it:

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 12,
  "version": "3.0.0",
  "framework": "tina4-python"
}
```

Your Tina4 Python project is running.

3. Project Structure Walkthrough

Here is what `tina4 init` created:

```
my-store/
■■■ .env                    # Your configuration (gitignored)
■■■ .env.example            # Template for other developers
■■■ .gitignore              # Pre-configured
■■■ pyproject.toml          # Python project definition (uv / pip)
■■■ .venv/                  # Virtual environment (gitignored)
■■■ src/
■   ■■■ routes/             # Your route handlers go here
■   ■■■ orm/                # Your ORM model classes go here
■   ■■■ migrations/         # SQL migration files
■   ■■■ seeds/              # Database seed files
■   ■■■ templates/          # Frond templates
■   ■   ■■■ errors/         # Custom 404.html, 500.html
■   ■■■ public/             # Static files (CSS, JS, images)
■   ■   ■■■ js/
■   ■   ■   ■■■ frond.js    # Auto-provided JS helper library
■   ■   ■■■ css/
■   ■   ■   ■■■ tina4.css   # Built-in CSS utility framework
■   ■   ■■■ scss/
■   ■   ■■■ images/
■   ■   ■■■ icons/
■   ■■■ locales/            # Translation files
■       ■■■ en.json
■■■ data/                   # SQLite databases (gitignored)
■■■ logs/                   # Log files (gitignored)
// ...
```

Key directories:

• **`src/routes/`** -- Every `.py` file here is auto-loaded at startup. Drop your route definitions here. Organize into subdirectories if you want.

• **`src/orm/`** -- Every `.py` file here is auto-loaded. Drop your ORM model classes here.

• **`src/templates/`** -- Frond looks here when you call `response.render("my-page.html", data)`.

• **`src/public/`** -- Files served directly. `src/public/images/logo.png` lives at `/images/logo.png`.

• **`data/`** -- The default SQLite database (`app.db`) lives here. Gitignored because databases do not belong in version control.

4. Your First Route

Create the file `src/routes/greeting.py`:

```
from tina4_python.core.router import get

@get("/api/greeting/{name}")
async def greeting(request, response):
    name = request.params["name"]
    from datetime import datetime
    return response.json({
        "message": f"Hello, {name}!",
        "timestamp": datetime.now().isoformat()
    })
```

Save the file. The dev server picks up the change through live reload. If not, restart with `tina4 serve`.

Test It

Open your browser to:

```
http://localhost:7145/api/greeting/Alice
```

You should see:

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00.000000"
}
```

Or use curl:

```
curl http://localhost:7145/api/greeting/Alice
```

```
{"message":"Hello, Alice!","timestamp":"2026-03-22T14:30:00.000000"}
```

The browser shows pretty-printed JSON (browser extensions or dev mode). Curl shows compact JSON. Force pretty output with `?pretty=true`:

```
curl "http://localhost:7145/api/greeting/Alice?pretty=true"
```

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00.000000"
}
```

Understanding What Happened

Five steps. No magic.

1. You created a file in `src/routes/`. Tina4 auto-discovered it at startup.

2. `@get("/api/greeting/{name}")` registered a GET route with a path parameter `{name}`.

3. When you requested `/api/greeting/Alice`, the router matched the pattern and called your handler.

4. `request.params["name"]` gave you `"Alice"` from the URL.

5. `response.json(...)` serialized the dictionary to JSON, set `Content-Type: application/json`, and returned a `200 OK`.

Adding More HTTP Methods

Update `src/routes/greeting.py`:

```
from tina4_python.core.router import get, post

@get("/api/greeting/{name}")
async def greeting(request, response):
    name = request.params["name"]
    from datetime import datetime
    return response.json({
        "message": f"Hello, {name}!",
        "timestamp": datetime.now().isoformat()
    })

@post("/api/greeting")
async def create_greeting(request, response):
    name = request.body.get("name", "World")
    language = request.body.get("language", "en")

    greetings = {
        "en": "Hello",
        "es": "Hola",
        "fr": "Bonjour",
        "de": "Hallo",
        "ja": "Konnichiwa"
    }

    greeting_word = greetings.get(language, greetings["en"])
// ...
```

Test the POST endpoint:

```
curl -X POST http://localhost:7145/api/greeting \
  -H "Content-Type: application/json" \
  -d '{"name": "Carlos", "language": "es"}'
```

```
{"message":"Hola, Carlos!","language":"es"}
```

The HTTP status code is `201 Created` (the second argument to `response.json()`).

5. Your First Template

Tina4 uses the **Frond** template engine -- a zero-dependency, Twig-compatible engine built from scratch. If you have used Twig, Jinja2, or Nunjucks, Frond will feel familiar.

Create a Base Layout

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
```

```
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        .product-card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; margin: 8px 0; }
        .product-card h3 { margin-top: 0; }
        .price { color: #2d8f2d; font-weight: bold; font-size: 1.2em; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.8em; }
        .badge-success { background: #d4edda; color: #155724; }
        .badge-danger { background: #f8d7da; color: #721c24; }
        nav { background: #333; color: white; padding: 12px 20px; }
        nav a { color: white; text-decoration: none; margin-right: 16px; }
    </style>
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
    </nav>
// ...
```

This base layout defines two blocks (`title` and `content`) that child templates override. It includes `tina4.css` (the built-in CSS framework) and `frond.js` (the built-in JS helper library).

Create a Product Listing Page

Create `src/templates/products.html`:

```
{% extends "base.html" %}

{% block title %}Products - My Store{% endblock %}

{% block content %}
    <h1>Our Products</h1>
    <p>Showing {{ products | length }} product{{ products | length != 1 ? "s" : "" }}</p>

    {% if products | length > 0 %}
        {% for product in products %}
            <div class="product-card">
                <h3>{{ product.name }}</h3>
                <p>{{ product.description }}</p>
                <p class="price">${{ product.price | number_format(2) }}</p>
                {% if product.in_stock %}
                    <span class="badge badge-success">In Stock</span>
                {% else %}
                    <span class="badge badge-danger">Out of Stock</span>
                {% endif %}
            </div>
        {% endfor %}
    {% else %}
        <p>No products available at the moment.</p>
    {% endif %}
{% endblock %}
```

Create the Route That Renders the Template

Create `src/routes/pages.py`:

```
from tina4_python.core.router import get

@get("/products")
async def products_page(request, response):
    products = [
        {
            "name": "Wireless Keyboard",
            "description": "Ergonomic wireless keyboard with backlit keys.",
```

```
            "price": 79.99,
            "in_stock": True
        },
        {
            "name": "USB-C Hub",
            "description": "7-port USB-C hub with HDMI, SD card reader, and Ethernet.",
            "price": 49.99,
            "in_stock": True
        },
        {
            "name": "Monitor Stand",
            "description": "Adjustable aluminum monitor stand with cable management.",
            "price": 129.99,
            "in_stock": False
        },
        {
            "name": "Mechanical Mouse",
// ...
```

See It in the Browser

Open `http://localhost:7145/products`. You should see:

• A dark navigation bar with "Home" and "Products" links

• The heading "Our Products"

• A subheading showing "Showing 4 products"

• Four product cards, each with name, description, price, and stock badge

• The "Monitor Stand" card shows a red "Out of Stock" badge

• The other three show green "In Stock" badges

How Template Rendering Works

Eight steps. All automatic.

1. `response.render("products.html", {"products": products})` tells Frond to render `src/templates/products.html` with the given data.

2. Frond sees `{% extends "base.html" %}` and loads the base template.

3. The `{% block content %}` in `products.html` replaces the same block in `base.html`.

4. `{{ product.name }}` outputs the value, auto-escaped for HTML safety.

5. `{{ product.price | number_format(2) }}` formats the number with 2 decimal places.

6. `{% for product in products %}` loops through the list.

7. `{% if product.in_stock %}` renders the stock badge conditionally.

8. `{{ products | length }}` returns the item count.

About tina4css

The `tina4.css` file is Tina4's built-in CSS utility framework. Layout utilities. Typography. Common UI patterns. No Bootstrap. No Tailwind. No download. It ships with every scaffolded project.

6. Understanding .env

Open the `.env` file at the root of your project:

```
TINA4_DEBUG=true
```

That is likely all you see. The scaffold creates a minimal `.env` with debug mode enabled. Everything else uses defaults.

The important defaults for development:

| Variable | Default Value | What It Means |

|----------|---------------|---------------|

| `TINA4_PORT` | `7145` | Server runs on port 7145 |

| `DATABASE_URL` | `sqlite:///data/app.db` | SQLite database in the `data/` directory |

| `TINA4_LOG_LEVEL` | `ALL` | All log messages are output |

| `CORS_ORIGINS` | `*` | All origins allowed (fine for development) |

| `TINA4_RATE_LIMIT` | `60` | 60 requests per minute per IP |

To change the port, add it to `.env`:

```
TINA4_DEBUG=true
TINA4_PORT=8080
```

Restart the server (`Ctrl+C`, then `tina4 serve`). It now runs on port 8080.

For the complete `.env` reference with all 68 variables, see Book 0, Chapter 4.

7. The Dev Dashboard

With `TINA4_DEBUG=true`, Tina4 provides a built-in development dashboard. Enable it by adding a console token to your `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=my-dev-token
```

Restart the server and navigate to:

```
http://localhost:7145/tina4/console
```

Enter your token (`my-dev-token`) when prompted. The dashboard shows:

• **System Overview** -- framework version, Python version, uptime, memory usage, database status

• **Request Inspector** -- recent HTTP requests with method, path, status, duration, and request ID. Click any request to see full headers, body, database queries, and template renders.

• **Error Log** -- unhandled exceptions with stack traces and occurrence counts

• **Queue Manager** -- queue status (pending, reserved, failed, dead-letter messages)

• **WebSocket Monitor** -- active WebSocket connections with metadata

• **Routes** -- all registered routes with their methods, paths, and middleware

The console shows you what your application is doing without adding print statements or log calls.

When you visit any HTML page (like `/products`), a **debug overlay** appears -- a toolbar at the bottom showing:

• Request details (method, URL, duration)

• Database queries executed (with timing)

• Template renders (with timing)

• Session data

• Recent log entries

This overlay is only visible when `TINA4_DEBUG=true`. Production never sees it.

8. Exercise: Greeting API + Product List Template

Build the following two features from scratch, without looking at the examples above.

Exercise Part A: Greeting API

Create an API endpoint at `GET /api/greet` that:

1. Accepts a query parameter `name` (e.g., `/api/greet?name=Sarah`)

2. If `name` is missing, defaults to `"Stranger"`

3. Returns JSON like:

```
{
  "greeting": "Welcome, Sarah!",
  "time_of_day": "afternoon"
}
```

4. The `time_of_day` should be calculated from the server's current hour:

• 5:00 - 11:59 = "morning"

• 12:00 - 16:59 = "afternoon"

• 17:00 - 20:59 = "evening"

• 21:00 - 4:59 = "night"

Test your endpoint with:

```
curl "http://localhost:7145/api/greet?name=Sarah"
curl "http://localhost:7145/api/greet"
```

Exercise Part B: Product List Page

Create a page at `GET /store` that:

1. Displays a list of at least 5 products (hardcoded for now)

2. Each product has: name, category, price, and a boolean `featured` flag

3. Featured products get a visual highlight (different background color, border, or badge)

4. The page shows the total number of products and the number of featured products

5. Uses template inheritance -- a layout template and a page template that extends it

6. Includes `tina4.css` and `frond.js`

Your products data should look like this in your route handler:

```
products = [
    {"name": "Espresso Machine", "category": "Kitchen", "price": 299.99, "featured": True},
    {"name": "Yoga Mat", "category": "Fitness", "price": 29.99, "featured": False},
    {"name": "Standing Desk", "category": "Office", "price": 549.99, "featured": True},
    {"name": "Noise-Canceling Headphones", "category": "Electronics", "price": 199.99, "featured": True},
    {"name": "Water Bottle", "category": "Fitness", "price": 24.99, "featured": False}
]
```

Expected browser output:

• A page titled "Our Store"

• Text showing "5 products, 3 featured"

• A list of product cards with name, category, price, and a "Featured" badge on highlighted items

• Featured products have a distinct visual style (your choice -- different border color, background, star icon, etc.)

9. Solutions

Solution A: Greeting API

Create `src/routes/greet.py`:

```
from tina4_python.core.router import get

@get("/api/greet")
async def greet(request, response):
    name = request.query.get("name", "Stranger")

    from datetime import datetime
    hour = datetime.now().hour

    if 5 <= hour < 12:
        time_of_day = "morning"
    elif 12 <= hour < 17:
        time_of_day = "afternoon"
    elif 17 <= hour < 21:
        time_of_day = "evening"
    else:
        time_of_day = "night"

    return response.json({
        "greeting": f"Welcome, {name}!",
        "time_of_day": time_of_day
    })
```

Test:

```
curl "http://localhost:7145/api/greet?name=Sarah"

{"greeting":"Welcome, Sarah!","time_of_day":"afternoon"}

curl "http://localhost:7145/api/greet"

{"greeting":"Welcome, Stranger!","time_of_day":"afternoon"}
```

Solution B: Product List Page

Create `src/templates/store-layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        header { background: #1a1a2e; color: white; padding: 16px 20px; }
        header h1 { margin: 0; }
        .stats { color: #888; margin: 8px 0 20px; }
        .product-grid { display: grid; gap: 16px; }
        .product-card { background: white; border: 2px solid #e0e0e0; border-radius: 8px; padding: 16px; }
        .product-card.featured { border-color: #ffc107; background: #fffdf0; }
        .product-name { font-size: 1.2em; font-weight: bold; margin: 0 0 4px; }
        .product-category { color: #666; font-size: 0.9em; }
        .product-price { color: #2d8f2d; font-weight: bold; font-size: 1.1em; margin-top: 8px; }
        .featured-badge { background: #ffc107; color: #333; padding: 2px 8px; border-radius: 4px; font-size: 
    </style>
</head>
<body>
    <header>
        <h1>{% block header %}Store{% endblock %}</h1>
// ...
```

Create `src/templates/store.html`:

```
{% extends "store-layout.html" %}

{% block title %}Our Store{% endblock %}
{% block header %}Our Store{% endblock %}

{% block content %}
    <p class="stats">{{ products | length }} products, {{ featured_count }} featured</p>

    <div class="product-grid">
        {% for product in products %}
            <div class="product-card{{ product.featured ? ' featured' : '' }}">
                <p class="product-name">
                    {{ product.name }}
                    {% if product.featured %}
                        <span class="featured-badge">Featured</span>
                    {% endif %}
                </p>
                <p class="product-category">{{ product.category }}</p>
                <p class="product-price">${{ product.price | number_format(2) }}</p>
            </div>
        {% endfor %}
    </div>
{% endblock %}
```

Create `src/routes/store.py`:

```
from tina4_python.core.router import get

@get("/store")
async def store_page(request, response):
    products = [
```

```
    {"name": "Espresso Machine", "category": "Kitchen", "price": 299.99, "featured": True},
    {"name": "Yoga Mat", "category": "Fitness", "price": 29.99, "featured": False},
    {"name": "Standing Desk", "category": "Office", "price": 549.99, "featured": True},
    {"name": "Noise-Canceling Headphones", "category": "Electronics", "price": 199.99, "featured": True},
    {"name": "Water Bottle", "category": "Fitness", "price": 24.99, "featured": False}
]

featured_count = sum(1 for p in products if p["featured"])

return response.render("store.html", {
    "products": products,
    "featured_count": featured_count
})
```

Open `http://localhost:7145/store` in your browser. You should see:

• A dark header reading "Our Store"

• Text showing "5 products, 3 featured"

• Five product cards in a grid

• Three cards (Espresso Machine, Standing Desk, Noise-Canceling Headphones) have a yellow border, light yellow background, and a "Featured" badge

• Two cards (Yoga Mat, Water Bottle) have a standard white background with gray border

• Each card shows the product name, category, and price formatted with two decimal places

10. Gotchas

1. File not auto-discovered

Problem: You created a route file but nothing happens when you visit the URL.

Cause: The file is not in `src/routes/`. It must be inside `src/routes/` (or a subdirectory), and the file must end with `.py`.

Fix: Move the file to `src/routes/your-file.py` and restart the server.

2. Missing import

Problem: `NameError: name 'get' is not defined` or similar.

Cause: You forgot to import the route decorator.

Fix: Start your route file with the correct import: `from tina4_python.core.router import get, post` (include whichever decorators you need).

3. Handler not async

Problem: Your route handler runs but returns an error about a coroutine object.

Cause: You defined the handler with `def` instead of `async def`. Tina4 Python expects all route handlers to be async.

Fix: Change `def greeting(request, response):` to `async def greeting(request, response):`. Every route handler must be `async def`.

4. Template not found

Problem: `Template "my-page.html" not found` error.

Cause: The template file is not in `src/templates/`, or there is a typo in the filename.

Fix: Check that the file exists at `src/templates/my-page.html`. The name in `response.render()` is relative to `src/templates/`.

5. Port already in use

Problem: `Error: Address already in use (port 7145)`

Cause: Another process (or another Tina4 instance) is using port 7145.

Fix: Stop the other process, or change the port:

```
TINA4_PORT=8080
```

Or use the CLI flag: `tina4 serve --port 8080`.

6. Changes not reflected

Problem: You edited a file but the browser shows the old version.

Cause: Live reload may not be active. Browser caching can serve stale versions.

Fix: Hard-refresh the browser (`Ctrl+Shift+R` or `Cmd+Shift+R`). If that fails, restart the dev server with `Ctrl+C` and `tina4 serve`.

7. .env not loaded

Problem: Environment variables have no effect.

Cause: The `.env` file must be at the project root (same directory as `pyproject.toml`). A subdirectory placement hides it from Tina4.

Fix: Move `.env` to the project root.

Chapter 2: Routing

1. How Routing Works in Tina4

Every web application maps URLs to code. A browser requests `/products`. The framework finds the handler for `/products`, runs it, sends back the result. That mapping is routing.

In Tina4 Python, routes live in Python files inside `src/routes/`. Every `.py` file in that directory (and its subdirectories) is auto-loaded at startup. No registration file. No central config. Drop a file in. It works.

The simplest route:

```
from tina4_python.core.router import get

@get("/hello")
async def hello(request, response):
    return response.json({"message": "Hello, World!"})
```

Save that as `src/routes/hello.py`, start the server with `tina4 serve`, and visit `http://localhost:7145/hello`:

```
{"message":"Hello, World!"}
```

One decorator. One function. Done.

2. HTTP Methods

Tina4 supports all five standard HTTP methods. Each has a decorator:

```
from tina4_python.core.router import get, post, put, patch, delete

@get("/products")
async def list_products(request, response):
    return response.json({"action": "list all products"})

@post("/products")
async def create_product(request, response):
    return response.json({"action": "create a product"}, 201)

@put("/products/{id}")
async def replace_product(request, response):
    id = request.params["id"]
    return response.json({"action": f"replace product {id}"})

@patch("/products/{id}")
async def update_product(request, response):
    id = request.params["id"]
    return response.json({"action": f"update product {id}"})

@delete("/products/{id}")
async def delete_product(request, response):
    id = request.params["id"]
    return response.json({"action": f"delete product {id}"})
```

Test each one:

```
curl http://localhost:7145/products
```

```
{"action":"list all products"}
```

```
curl -X POST http://localhost:7145/products \
```

```
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'
```

```
{"action":"create a product"}
```

```
curl -X PUT http://localhost:7145/products/42
```

```
{"action":"replace product 42"}
```

```
curl -X PATCH http://localhost:7145/products/42
```

```
{"action":"update product 42"}
```

```
curl -X DELETE http://localhost:7145/products/42
```

```
{"action":"delete product 42"}
```

`GET` reads. `POST` creates. `PUT` replaces. `PATCH` patches. `DELETE` removes. REST convention. Predictable API.

3. Path Parameters

Path parameters capture values from the URL. Wrap the parameter name in curly braces:

```
from tina4_python.core.router import get

@get("/users/{id}/posts/{post_id}")
async def user_post(request, response):
    user_id = request.params["id"]
    post_id = request.params["post_id"]

    return response.json({
        "user_id": user_id,
        "post_id": post_id
    })
```

```
curl http://localhost:7145/users/5/posts/99
```

```
{"user_id":"5","post_id":"99"}
```

Notice `user_id` came back as the string `"5"`, not the integer `5`. Path parameters are strings by default.

Typed Parameters

Enforce a type by adding a colon and the type after the parameter name:

```
from tina4_python.core.router import get

@get("/orders/{id:int}")
async def get_order(request, response):
    id = request.params["id"]  # This is now an integer
    return response.json({
        "order_id": id,
        "type": type(id).__name__
    })
```

```
curl http://localhost:7145/orders/42
```

```
{"order_id":42,"type":"int"}
```

Pass a non-integer value and the route does not match. A 404:

```
curl http://localhost:7145/orders/abc
```

```
{"error":"Not found","path":"/orders/abc","status":404}
```

Supported types:

| Type | Matches | Example |

|------|---------|---------|

| `int` | Digits only | `{id:int}` matches `42` but not `abc` |

| `float` | Decimal numbers | `{price:float}` matches `19.99` |

| `alpha` | Letters only | `{slug:alpha}` matches `hello` but not `hello123` |

| `alphanumeric` | Letters and digits | `{code:alphanumeric}` matches `abc123` |

4. Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them through `request.query`:

```
from tina4_python.core.router import get

@get("/search")
async def search(request, response):
    q = request.query.get("q", "")
    page = int(request.query.get("page", 1))
    limit = int(request.query.get("limit", 10))

    return response.json({
        "query": q,
        "page": page,
        "limit": limit,
        "offset": (page - 1) * limit
    })
```

```
curl "http://localhost:7145/search?q=keyboard&page=2&limit=20"
```

```
{"query":"keyboard","page":2,"limit":20,"offset":20}
```

If a query parameter is missing, `request.query.get("key")` returns `None`. Use `.get()` with a default value.

5. Route Groups

A set of routes sharing a common prefix belongs in a `group()`:

```
from tina4_python.core.router import get, post, group

@group("/api/v1")
def api_v1():

    @get("/users")
    async def list_users(request, response):
        return response.json({"users": []})

    @get("/users/{id:int}")
    async def get_user(request, response):
        id = request.params["id"]
        return response.json({"user": {"id": id, "name": "Alice"}})

    @post("/users")
    async def create_user(request, response):
        return response.json({"created": True}, 201)
```

```
    @get("/products")
    async def list_products(request, response):
        return response.json({"products": []})
```

These routes register as `/api/v1/users`, `/api/v1/users/{id}`, and `/api/v1/products`. Short paths inside the group. Tina4 prepends the prefix.

```
curl http://localhost:7145/api/v1/users
```

```
{"users":[]}
```

```
curl http://localhost:7145/api/v1/products
```

```
{"products":[]}
```

Groups nest:

```
from tina4_python.core.router import get, group

@group("/api")
def api():

    @group("/v1")
    def v1():
        @get("/status")
        async def v1_status(request, response):
            return response.json({"version": "1.0"})

    @group("/v2")
    def v2():
        @get("/status")
        async def v2_status(request, response):
            return response.json({"version": "2.0"})
```

```
curl http://localhost:7145/api/v1/status
```

```
{"version":"1.0"}
```

```
curl http://localhost:7145/api/v2/status
```

```
{"version":"2.0"}
```

6. Middleware on Routes

Middleware is code that runs before or after your route handler. Authentication. Logging. Rate limiting. Input validation. Anything that belongs on multiple routes. Chapter 8 covers middleware in depth. Here is how it connects to routes.

Middleware on a Single Route

Attach middleware with the `@middleware` decorator:

```
from tina4_python.core.router import get, middleware
import time

async def log_request(request, response, next_handler):
    start = time.time()
    print(f"[{time.strftime('%Y-%m-%d %H:%M:%S')}] {request.method} {request.path}")

    result = await next_handler(request, response)

    duration = round((time.time() - start) * 1000, 2)
    print(f"  Completed in {duration}ms")
```

```
    return result

@get("/api/data")
@middleware(log_request)
async def get_data(request, response):
    return response.json({"data": [1, 2, 3]})
```

The middleware function receives `request`, `response`, and `next_handler`. Call `await next_handler(request, response)` to continue to the route handler. Skip that call and the handler never runs -- the chain stops cold. Useful for blocking unauthorized requests.

Blocking Middleware

Middleware that checks for an API key:

```
from tina4_python.core.router import get, middleware

async def require_api_key(request, response, next_handler):
    api_key = request.headers.get("X-API-Key", "")

    if api_key != "my-secret-key":
        return response.json({"error": "Invalid API key"}, 401)

    return await next_handler(request, response)

@get("/api/secret")
@middleware(require_api_key)
async def secret_data(request, response):
    return response.json({"secret": "The answer is 42"})
```

```
curl http://localhost:7145/api/secret
```

```
{"error":"Invalid API key"}
```

Status: `401 Unauthorized`.

```
curl http://localhost:7145/api/secret -H "X-API-Key: my-secret-key"
```

```
{"secret":"The answer is 42"}
```

Multiple Middleware

Chain multiple middleware as arguments:

```
@get("/api/important")
@middleware(log_request, require_api_key)
async def important_data(request, response):
    return response.json({"data": "important stuff"})
```

Middleware runs left to right: `log_request` first, then `require_api_key`, then the route handler. If any middleware skips `next_handler`, the chain stops there.

7. Route Decorators: @noauth() and @secured()

Two decorators control authentication at the route level.

@noauth() -- Public Routes

When your application has global authentication middleware, `@noauth()` marks a route as public:

```
from tina4_python.core.router import get, noauth
```

```
@noauth()
@get("/api/public/info")
async def public_info(request, response):
    return response.json({
        "app": "My Store",
        "version": "1.0.0"
    })
```

`@noauth()` tells Tina4 to skip authentication checks for this route, even if global auth middleware is configured in `.env` or applied to the parent group.

@secured() -- Protected GET Routes

`@secured()` marks a GET route as requiring authentication:

```
from tina4_python.core.router import get, secured

@secured()
@get("/api/profile")
async def profile(request, response):
    # request.user is populated by the auth middleware
    return response.json({
        "user": request.user
    })
```

By default, `POST`, `PUT`, `PATCH`, and `DELETE` routes are secured. `GET` routes are public unless you add `@secured()`. This matches the common pattern: reading data is public, modifying data requires authentication.

8. Wildcard and Catch-All Routes

Wildcard Routes

Use `*` at the end of a path to match anything after it:

```
from tina4_python.core.router import get

@get("/docs/*")
async def docs_handler(request, response):
    path = request.params.get("*", "")
    return response.json({
        "section": "docs",
        "path": path
    })
```

```
curl http://localhost:7145/docs/getting-started
```

```
{"section":"docs","path":"getting-started"}
```

```
curl http://localhost:7145/docs/api/authentication/jwt
```

```
{"section":"docs","path":"api/authentication/jwt"}
```

Catch-All Route (Custom 404)

A catch-all handles any unmatched URL:

```
from tina4_python.core.router import get

@get("/*")
async def not_found(request, response):
    return response.json({
```

```
        "error": "Page not found",
        "path": request.path
    }, 404)
```

Define this route last (or in a file that sorts alphabetically after your other route files). Tina4 matches routes in registration order -- first match wins.

You can also create a custom 404 page by placing a template at `src/templates/errors/404.html`:

```
{% extends "base.html" %}

{% block title %}Not Found{% endblock %}

{% block content %}
    <h1>404 - Page Not Found</h1>
    <p>The page you are looking for does not exist.</p>
    <a href="/">Go back home</a>
{% endblock %}
```

Tina4 uses this template for any unmatched route when the file exists.

9. Route Listing via CLI

As your application grows, see all registered routes at a glance:

```
tina4 routes

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /hello                        -                   public
GET      /products                     -                   public
POST     /products                     -                   secured
PUT      /products/{id}                -                   secured
PATCH    /products/{id}                -                   secured
DELETE   /products/{id}                -                   secured
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/public/info              -                   @noauth
GET      /api/profile                  -                   @secured
GET      /search                       -                   public
GET      /docs/*                       -                   public
```

The `Auth` column shows whether a route is public, secured (default for non-GET methods), `@noauth`, or `@secured`.

Filter by method:

```
tina4 routes --method POST

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
POST     /products                     -                   secured
POST     /api/v1/users                 -                   secured
```

Search for a path pattern:

```
tina4 routes --filter users

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
```

```
POST     /api/v1/users                -                    secured
```

10. Organizing Route Files

Organize route files any way you want. Tina4 loads every `.py` file in `src/routes/` recursively. Two common patterns:

Pattern 1: One File Per Resource

```
src/routes/
■■■ products.py     # All product routes
■■■ users.py        # All user routes
■■■ orders.py       # All order routes
■■■ pages.py        # HTML page routes
```

Pattern 2: Subdirectories by Feature

```
src/routes/
■■■ api/
■   ■■■ products.py
■   ■■■ users.py
■   ■■■ orders.py
■■■ admin/
■   ■■■ dashboard.py
■   ■■■ settings.py
■■■ pages/
    ■■■ home.py
    ■■■ about.py
```

Both work identically. The directory structure has no effect on URL paths -- only the route definitions inside the files matter. Choose whichever keeps your project navigable.

11. Exercise: Build a Full CRUD API for Products

Build a complete REST API for managing products. All data stored in a Python list (no database yet -- Chapter 5 adds that).

Requirements

Create a file `src/routes/product_api.py` with the following routes:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/products` | List all products. Support `?category=` filter. |

| `GET` | `/api/products/{id:int}` | Get a single product by ID. Return 404 if not found. |

| `POST` | `/api/products` | Create a new product. Return 201. |

| `PUT` | `/api/products/{id:int}` | Replace a product. Return 404 if not found. |

| `DELETE` | `/api/products/{id:int}` | Delete a product. Return 204 with no body. |

Each product has: `id` (int), `name` (string), `category` (string), `price` (float), `in_stock` (bool).

Start with this seed data:

```
products = [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": True},
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": True},
    {"id": 3, "name": "Coffee Grinder", "category": "Kitchen", "price": 49.99, "in_stock": False},
    {"id": 4, "name": "Standing Desk", "category": "Office", "price": 549.99, "in_stock": True},
    {"id": 5, "name": "Running Shoes", "category": "Fitness", "price": 119.99, "in_stock": True}
]
```

Test with:

```
# List all
curl http://localhost:7145/api/products

# Filter by category
curl "http://localhost:7145/api/products?category=Fitness"

# Get one
curl http://localhost:7145/api/products/3

# Create
curl -X POST http://localhost:7145/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Desk Lamp", "category": "Office", "price": 39.99, "in_stock": true}'

# Update
curl -X PUT http://localhost:7145/api/products/3 \
  -H "Content-Type: application/json" \
  -d '{"name": "Burr Coffee Grinder", "category": "Kitchen", "price": 59.99, "in_stock": true}'

# Delete
curl -X DELETE http://localhost:7145/api/products/3

# Not found
curl http://localhost:7145/api/products/999
```

12. Solution

Create `src/routes/product_api.py`:

```
from tina4_python.core.router import get, post, put, delete

# In-memory product store (resets on server restart)
products = [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": True},
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": True},
    {"id": 3, "name": "Coffee Grinder", "category": "Kitchen", "price": 49.99, "in_stock": False},
    {"id": 4, "name": "Standing Desk", "category": "Office", "price": 549.99, "in_stock": True},
    {"id": 5, "name": "Running Shoes", "category": "Fitness", "price": 119.99, "in_stock": True}
]

next_id = 6


# List all products, optionally filter by category
@get("/api/products")
async def list_products(request, response):
    category = request.query.get("category")

    if category is not None:
        filtered = [p for p in products if p["category"].lower() == category.lower()]
        return response.json({"products": filtered, "count": len(filtered)})
```

```
    return response.json({"products": products, "count": len(products)})

// ...
```

Expected output for the test commands:

List all:

```
{"products":[{"id":1,"name":"Wireless Keyboard","category":"Electronics","price":79.99,"in_stock":true},{"id"
```

Filter by category:

```
{"products":[{"id":2,"name":"Yoga Mat","category":"Fitness","price":29.99,"in_stock":true},{"id":5,"name":"Run
```

Get one:

```
{"id":3,"name":"Coffee Grinder","category":"Kitchen","price":49.99,"in_stock":false}
```

Create:

```
{"id":6,"name":"Desk Lamp","category":"Office","price":39.99,"in_stock":true}
```

(Status: `201 Created`)

Update:

```
{"id":3,"name":"Burr Coffee Grinder","category":"Kitchen","price":59.99,"in_stock":true}
```

Delete: empty response with status `204 No Content`.

Not found:

```
{"error":"Product not found","id":999}
```

(Status: `404 Not Found`)

13. Gotchas

1. Trailing slashes matter

Problem: `/products` works but `/products/` returns a 404 (or vice versa).

Cause: Tina4 treats `/products` and `/products/` as different routes by default.

Fix: Pick one convention and stick with it. If you want both to work, register the route without a trailing slash -- Tina4 redirects `/products/` to `/products` when `TINA4_TRAILING_SLASH_REDIRECT=true` is set in `.env`.

2. Parameter names must be unique in a path

Problem: `/users/{id}/posts/{id}` behaves wrong -- both parameters share a name.

Cause: The second `{id}` overwrites the first in `request.params`.

Fix: Use distinct names: `/users/{user_id}/posts/{post_id}`.

3. Method conflicts

Problem: You defined `@get("/items/{id}")` and `@get("/items/{action}")` and the wrong handler runs.

Cause: Both patterns match `/items/42`. The first one registered wins.

Fix: Use typed parameters to disambiguate: `@get("/items/{id:int}")` matches integers only, leaving `/items/export` free for the other route. Or restructure your paths: `/items/{id:int}` and `/items/actions/{action}`.

4. Route handler must return a response

Problem: Your route handler runs but the browser shows an empty page or a 500 error.

Cause: You forgot the `return` statement. Without `return`, the handler returns `None` and Tina4 has nothing to send back.

Fix: Every handler must return something: `return response.json(...)` or `return response.html(...)` or `return response.render(...)`.

5. Decorator order matters

Problem: Your `@middleware` decorator has no effect, or your `@noauth()` is ignored.

Cause: Python decorators apply bottom-up. Wrong stacking order breaks registration.

Fix: Put the route decorator (`@get`, `@post`, etc.) first (closest to the function), then additional decorators above it:

```
@middleware(require_api_key)  # Applied second (wraps the route)
@get("/api/secret")           # Applied first (registers the route)
async def secret(request, response):
    ...
```

6. Forgetting async def

Problem: Your route handler raises a `TypeError` about a coroutine or the response is a coroutine object instead of JSON.

Cause: You used `def` instead of `async def`.

Fix: Every route handler in Tina4 Python must be `async def`. The framework runs on an async server. Change `def my_handler(request, response):` to `async def my_handler(request, response):`.

7. Group prefix must start with a slash

Problem: `@group("api/v1")` produces routes that do not match.

Cause: The group prefix should start with `/` for consistency.

Fix: Start group prefixes with `/`: `@group("/api/v1")`.

Chapter 3: Request and Response

1. The Two Objects You Use Everywhere

Every route handler receives two arguments: `request` and `response`. The request carries everything the client sent. The response builds what you send back. Master these two objects and you handle any HTTP scenario.

Picture a file-sharing service. A user uploads a document. You validate the file type. You check their session. You return a success JSON or an error page. Every step flows through `request` and `response`.

2. The Request Object

The `request` object gives you access to every piece of information about the incoming HTTP request.

request.method

The HTTP method as an uppercase string:

```
from tina4_python.core.router import get, post

@get("/api/check")
async def check_method(request, response):
    return response.json({"method": request.method})
```

```
curl http://localhost:7145/api/check
```

```
{"method":"GET"}
```

request.path

The URL path without query parameters:

```
@get("/api/info")
async def info(request, response):
    return response.json({"path": request.path})
```

```
curl "http://localhost:7145/api/info?foo=bar"
```

```
{"path":"/api/info"}
```

request.params

Path parameters captured from the URL pattern:

```
@get("/users/{id:int}/posts/{slug}")
async def user_post(request, response):
    return response.json({
        "user_id": request.params["id"],
        "slug": request.params["slug"]
    })
```

```
curl http://localhost:7145/users/5/posts/hello-world
```

```
{"user_id":5,"slug":"hello-world"}
```

request.query

Query string parameters as a dictionary:

```
@get("/search")
```

```
async def search(request, response):
    return response.json({
        "q": request.query.get("q", ""),
        "page": int(request.query.get("page", 1)),
        "sort": request.query.get("sort", "relevance")
    })
```

```
curl "http://localhost:7145/search?q=laptop&page=2&sort=price"
```

```
{"q":"laptop","page":2,"sort":"price"}
```

request.body

The parsed request body. JSON requests become a dictionary. Form submissions contain form fields:

```
@post("/api/feedback")
async def feedback(request, response):
    name = request.body.get("name", "Anonymous")
    message = request.body.get("message", "")
    rating = request.body.get("rating", 0)

    return response.json({
        "received": {
            "name": name,
            "message": message,
            "rating": rating
        }
    }, 201)
```

```
curl -X POST http://localhost:7145/api/feedback \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "message": "Great product!", "rating": 5}'
```

```
{"received":{"name":"Alice","message":"Great product!","rating":5}}
```

request.headers

A dictionary of HTTP headers. Header names are case-insensitive:

```
@get("/api/headers")
async def show_headers(request, response):
    return response.json({
        "content_type": request.headers.get("Content-Type", "not set"),
        "user_agent": request.headers.get("User-Agent", "not set"),
        "accept": request.headers.get("Accept", "not set"),
        "custom": request.headers.get("X-Custom-Header", "not set")
    })
```

```
curl http://localhost:7145/api/headers -H "X-Custom-Header: hello-tina4"
```

```
{"content_type":"not set","user_agent":"curl/8.1.2","accept":"*/*","custom":"hello-tina4"}
```

request.ip

The client's IP address:

```
@get("/api/whoami")
async def whoami(request, response):
    return response.json({"ip": request.ip})
```

```
curl http://localhost:7145/api/whoami
```

```
{"ip":"127.0.0.1"}
```

request.cookies

A dictionary of cookies sent by the client:

```
@get("/api/cookies")
async def show_cookies(request, response):
    return response.json({
        "cookies": request.cookies,
        "session_id": request.cookies.get("session_id", "none")
    })
```

```
curl http://localhost:7145/api/cookies -b "session_id=abc123; theme=dark"
```

```
{"cookies":{"session_id":"abc123","theme":"dark"},"session_id":"abc123"}
```

request.files

For multipart file uploads, `request.files` holds the uploaded files:

```
@post("/api/upload")
async def upload(request, response):
    if "document" not in request.files:
        return response.json({"error": "No file uploaded"}, 400)

    file = request.files["document"]

    return response.json({
        "filename": file.filename,
        "content_type": file.content_type,
        "size": len(file.content)
    })
```

```
curl -X POST http://localhost:7145/api/upload \
  -F "document=@report.pdf"
```

```
{"filename":"report.pdf","content_type":"application/pdf","size":45231}
```

To save the file to disk:

```
@post("/api/upload/save")
async def upload_and_save(request, response):
    if "photo" not in request.files:
        return response.json({"error": "No photo uploaded"}, 400)

    file = request.files["photo"]

    # Validate file type
    allowed = ["image/jpeg", "image/png", "image/webp"]
    if file.content_type not in allowed:
        return response.json({"error": f"File type {file.content_type} not allowed"}, 400)

    # Save to public directory
    import os
    save_path = os.path.join("src", "public", "images", file.filename)
    with open(save_path, "wb") as f:
        f.write(file.content)

    return response.json({
        "message": "Photo uploaded",
        "url": f"/images/{file.filename}"
    }, 201)
```

3. The Response Object

The `response` object builds HTTP responses. Every route handler must return one.

response.json()

Return JSON data with an optional status code:

```
@get("/api/users")
async def users(request, response):
    users = [
        {"id": 1, "name": "Alice"},
        {"id": 2, "name": "Bob"}
    ]
    return response.json({"users": users, "count": len(users)})

@post("/api/users")
async def create_user(request, response):
    # 201 Created
    return response.json({"id": 3, "name": request.body["name"]}, 201)

@get("/api/error")
async def error_example(request, response):
    # 400 Bad Request
    return response.json({"error": "Something went wrong"}, 400)
```

response.html()

Return raw HTML:

```
@get("/welcome")
async def welcome(request, response):
    return response.html("""
        <!DOCTYPE html>
        <html>
        <body>
            <h1>Welcome to My Store</h1>
            <p>Browse our <a href="/products">products</a>.</p>
        </body>
        </html>
    """)
```

response.text()

Return plain text:

```
@get("/robots.txt")
async def robots(request, response):
    return response.text("User-agent: *\nAllow: /")
```

response.render()

Render a Frond template with data:

```
@get("/dashboard")
async def dashboard(request, response):
    return response.render("dashboard.html", {
        "user_name": "Alice",
        "notifications": 3,
        "recent_orders": [
            {"id": 101, "total": 59.99},
            {"id": 102, "total": 124.50}
        ]
    })
```

response.redirect()

Redirect the client to another URL:

```
@get("/old-page")
async def old_page(request, response):
    return response.redirect("/new-page")

@post("/api/logout")
async def logout(request, response):
    # 303 See Other -- redirect after POST
    return response.redirect("/login", 303)

@get("/moved")
async def moved(request, response):
    # 301 Permanent redirect
    return response.redirect("/new-location", 301)
```

The default status code for `redirect()` is `302 Found` (temporary redirect).

response.file()

Send a file as the response:

```
@get("/download/report")
async def download_report(request, response):
    return response.file("data/reports/monthly-report.pdf")
```

Tina4 auto-detects the content type from the file extension. The browser displays or downloads the file based on content type.

To force a download (instead of inline display):

```
@get("/download/data")
async def download_data(request, response):
    return response.file("data/export.csv", download=True, filename="sales-data.csv")
```

Setting Status Codes

Every response method accepts an optional status code as the second argument:

```
# 200 OK (default)
return response.json({"ok": True})

# 201 Created
return response.json({"id": 1}, 201)

# 204 No Content
return response.json(None, 204)

# 400 Bad Request
return response.json({"error": "Invalid input"}, 400)

# 401 Unauthorized
return response.json({"error": "Login required"}, 401)

# 403 Forbidden
return response.json({"error": "Not allowed"}, 403)

# 404 Not Found
return response.json({"error": "Not found"}, 404)

# 500 Internal Server Error
return response.json({"error": "Server error"}, 500)
```

Setting Response Headers

Add custom headers:

```
@get("/api/data")
async def data_with_headers(request, response):
    return response.header("X-Custom-Header", "my-value") \
                   .header("X-Request-Id", "abc-123") \
                   .json({"data": [1, 2, 3]})
```

```
curl -i http://localhost:7145/api/data
```

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Custom-Header: my-value
X-Request-Id: abc-123

{"data":[1,2,3]}
```

Setting Cookies

Set cookies on the response:

```
@post("/api/login")
async def login(request, response):
    # Set a session cookie
    return response.cookie("session_id", "abc123", {
        "httponly": True,
        "secure": True,
        "max_age": 3600,
        "path": "/"
    }).json({"message": "Logged in"})

@post("/api/logout")
async def logout(request, response):
    # Delete a cookie by setting max_age to 0
    return response.cookie("session_id", "", {
        "max_age": 0,
        "path": "/"
    }).json({"message": "Logged out"})
```

Cookie options:

| Option | Type | Description |

|--------|------|----------|

| `httponly` | bool | Cookie not accessible via JavaScript |

| `secure` | bool | Cookie only sent over HTTPS |

| `max_age` | int | Lifetime in seconds (0 = delete) |

| `path` | string | URL path scope |

| `domain` | string | Domain scope |

| `samesite` | string | "Strict", "Lax", or "None" |

4. Real-World Example: File Upload with Validation

A complete file upload endpoint that validates type, size, and returns appropriate errors:

```
from tina4_python.core.router import post
import os
import uuid
```

```
UPLOAD_DIR = "src/public/uploads"
MAX_FILE_SIZE = 5 * 1024 * 1024  # 5 MB
ALLOWED_TYPES = {
    "image/jpeg": ".jpg",
    "image/png": ".png",
    "image/webp": ".webp",
    "application/pdf": ".pdf"
}

@post("/api/files/upload")
async def upload_file(request, response):
    if "file" not in request.files:
        return response.json({"error": "No file provided. Send a file with field name 'file'"}, 400)

    file = request.files["file"]

    # Check file type
    if file.content_type not in ALLOWED_TYPES:
        return response.json({
            "error": f"File type '{file.content_type}' not allowed",
            "allowed": list(ALLOWED_TYPES.keys())
// ...
```

```
curl -X POST http://localhost:7145/api/files/upload \
  -F "file=@photo.jpg"
```

```
{
  "message": "File uploaded successfully",
  "file": {
    "original_name": "photo.jpg",
    "saved_as": "a1b2c3d4e5f6.jpg",
    "url": "/uploads/a1b2c3d4e5f6.jpg",
    "size": 245760,
    "content_type": "image/jpeg"
  }
}
```

5. Exercise: Build a Contact Form API

Build an API that processes contact form submissions with full validation.

Requirements

Create `src/routes/contact.py` with these endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `POST` | `/api/contact` | Submit a contact form. Validate all fields. Return 201 on success. |

| `GET` | `/api/contact/submissions` | List all submissions. Support `?status=` filter. |

The contact form body should include: `name` (required, 2-100 chars), `email` (required, must contain @), `subject` (required), `message` (required, 10+ chars), `urgency` (optional, one of "low", "medium", "high", default "medium").

Validation rules:

• Return 400 with an `errors` list if any field is invalid

- Store submissions in a Python list with an auto-incremented ID and timestamp
- Each submission has a `status` of "new"

Test with:

```
# Valid submission
curl -X POST http://localhost:7145/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "subject": "Question", "message": "I have a question abo

# Invalid submission (missing fields, short message)
curl -X POST http://localhost:7145/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "A", "email": "bad-email", "message": "Short"}'

# List all
curl http://localhost:7145/api/contact/submissions

# Filter by status
curl "http://localhost:7145/api/contact/submissions?status=new"
```

6. Solution

```
from tina4_python.core.router import get, post
from datetime import datetime

submissions = []
next_id = 1


@post("/api/contact")
async def submit_contact(request, response):
    global next_id
    body = request.body
    errors = []

    # Validate name
    name = body.get("name", "")
    if not name:
        errors.append("Name is required")
    elif len(name) < 2 or len(name) > 100:
        errors.append("Name must be between 2 and 100 characters")

    # Validate email
    email = body.get("email", "")
    if not email:
        errors.append("Email is required")
    elif "@" not in email:
// ...
```

Valid submission output:

```
{
  "message": "Contact form submitted successfully",
  "submission": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "subject": "Question",
    "message": "I have a question about your product pricing.",
```

```
    "urgency": "high",
    "status": "new",
    "created_at": "2026-03-22T14:30:00.000000"
  }
}
```

(Status: `201 Created`)

Invalid submission output:

```
{
  "errors": [
    "Name must be between 2 and 100 characters",
    "Email must contain @",
    "Subject is required",
    "Message must be at least 10 characters"
  ]
}
```

(Status: `400 Bad Request`)

7. Gotchas

1. request.body is None for GET requests

Problem: Accessing `request.body` in a GET handler returns `None` or an empty dict.

Cause: GET requests carry no body by convention. Browsers and curl do not send one.

Fix: Use `request.query` for GET parameters. The body is only populated for POST, PUT, and PATCH requests.

2. Forgetting to parse Content-Type

Problem: `request.body` is an empty dictionary even though you sent JSON.

Cause: You did not set `Content-Type: application/json` in your request. Without it, Tina4 does not parse the body as JSON.

Fix: Include `-H "Content-Type: application/json"` when sending JSON via curl. In JavaScript fetch, set `headers: {"Content-Type": "application/json"}`.

3. File upload with wrong field name

Problem: `request.files["photo"]` raises a `KeyError`.

Cause: The form field name does not match the key you look up.

Fix: Check that the form uses `<input type="file" name="photo">` and curl uses `-F "photo=@file.jpg"`. The string after `-F` must match the key in `request.files`.

4. response.redirect() in AJAX calls

Problem: Your JavaScript `fetch()` call gets a redirect response but the browser does not navigate.

Cause: `fetch()` follows redirects silently and returns the final response. It does not trigger browser navigation.

Fix: For AJAX calls, return a JSON response with the redirect URL and handle navigation in JavaScript: `window.location.href = data.redirect_url`. Use `response.redirect()` only for traditional form submissions

and browser navigation.

5. Cookie not being set

Problem: You called `response.cookie()` but the cookie does not appear in the browser.

Cause: If you set `secure: True`, the cookie is only sent over HTTPS. On `http://localhost`, the browser ignores it.

Fix: During development on HTTP, set `"secure": False`. Enable `"secure": True` only in production where HTTPS is configured.

6. Large file uploads fail silently

Problem: Uploading a large file results in an empty `request.files` dictionary or a 413 error.

Cause: The default maximum upload size may be smaller than your file.

Fix: Set `TINA4_MAX_UPLOAD_SIZE` in your `.env` file (value in bytes). For 50 MB: `TINA4_MAX_UPLOAD_SIZE=52428800`.

7. Chaining response methods in wrong order

Problem: Setting headers or cookies after calling `response.json()` has no effect.

Cause: `response.json()` finalizes and returns the response. Anything after it is too late.

Fix: Chain headers and cookies before the final response method:

```
# Correct
return response.header("X-Custom", "value").cookie("token", "abc").json({"ok": True})

# Wrong -- header is set after json() already returned
result = response.json({"ok": True})
response.header("X-Custom", "value")  # Too late
return result
```

Chapter 4: Templates

1. Beyond JSON -- Rendering HTML

Every route so far returns JSON. That works for APIs. But web applications need HTML -- product listings, dashboards, login forms, email templates. Tina4 uses the **Frond** template engine for this.

Frond is a zero-dependency template engine built from scratch. Its syntax is compatible with Twig, Jinja2, and Nunjucks. If you know any of those, you know Frond. If you do not, the syntax is three things: `{{ }}` for output, `{% %}` for logic, `{# #}` for comments.

Picture an online store. A product catalog page. Items in a grid. Featured products highlighted. Prices formatted. Layout inherited from a shared template. That is what this chapter builds.

2. The @template Decorator

The shortest path to a rendered template:

```
from tina4_python.core.router import template

@template("/about", "about.html")
async def about_page(request, response):
    return {
        "title": "About Us",
        "company": "My Store",
        "founded": 2020
    }
```

Create `src/templates/about.html`:

```
<!DOCTYPE html>
<html>
<head><title>{{ title }}</title></head>
<body>
    <h1>{{ title }}</h1>
    <p>{{ company }} was founded in {{ founded }}.</p>
</body>
</html>
```

Visit `http://localhost:7145/about` and the rendered page appears.

The `@template` decorator maps a URL to a template file. The function returns a dictionary. Those values become template variables. You can also use `response.render()` in any route handler -- `@template` is shorthand.

3. Variables and Output

Basic Output

Use `{{ }}` to output a variable:

```
<h1>Hello, {{ name }}!</h1>
<p>Your balance is {{ balance }}.</p>
```

With data `{"name": "Alice", "balance": 150.50}`, this renders:

```
<h1>Hello, Alice!</h1>
<p>Your balance is 150.5.</p>
```

Accessing Nested Properties

Dot notation for dictionaries and object attributes:

```
<p>{{ user.name }}</p>
<p>{{ user.address.city }}</p>
<p>{{ order.items.0.name }}</p>
```

With data:

```
{
    "user": {
        "name": "Alice",
        "address": {"city": "Cape Town"}
    },
    "order": {
        "items": [{"name": "Keyboard"}, {"name": "Mouse"}]
    }
}
```

Renders:

```
<p>Alice</p>
<p>Cape Town</p>
<p>Keyboard</p>
```

Auto-Escaping

Frond escapes HTML characters in output by default. XSS attacks die here:

```
<p>{{ user_input }}</p>
```

With `{"user_input": "<script>alert('hacked')</script>"}`, this renders:

```
<p>&lt;script&gt;alert(&#39;hacked&#39;)&lt;/script&gt;</p>
```

The script tag is escaped. Displayed as text. Never executes. If you need raw HTML (and you trust the source), use the `|safe` filter:

```
<div>{{ trusted_html | safe }}</div>
```

4. Filters

Filters transform output. The pipe `|` applies them:

```
{{ name | upper }}          {# ALICE #}
{{ name | lower }}          {# alice #}
{{ name | capitalize }}     {# Alice #}
{{ name | title }}          {# Alice Smith #}
{{ bio | trim }}            {# Removes leading/trailing whitespace #}
```

Number Formatting

```
{{ price | number_format(2) }}          {# 79.99 #}
{{ large_number | number_format(0) }}   {# 1,234,567 #}
{{ percentage | round(1) }}             {# 85.3 #}
```

String Filters

```
{{ text | replace("old", "new") }}      {# Replace substring #}
{{ title | striptags }}                  {# Remove HTML tags #}
```

```
{{ content | nl2br }}                  {# Convert newlines to <br> #}
{{ slug | url_encode }}                {# URL-encode a string #}
{{ items | join(", ") }}               {# Join list with separator #}
```

Collection Filters

```
{{ items | length }}          {# Number of items #}
{{ items | first }}           {# First item #}
{{ items | last }}            {# Last item #}
{{ items | sort }}            {# Sort ascending #}
{{ items | reverse }}         {# Reverse order #}
{{ items | unique }}          {# Remove duplicates #}
{{ items | slice(0, 3) }}     {# First 3 items #}
{{ items | batch(3) }}        {# Group into batches of 3 #}
```

Date Formatting

```
{{ created_at | date("Y-m-d") }}        {# 2026-03-22 #}
{{ created_at | date("d M Y") }}        {# 22 Mar 2026 #}
{{ created_at | date("H:i:s") }}        {# 14:30:00 #}
```

Encoding Filters

```
{{ data | json_encode }}                {# {"key": "value"} #}
{{ text | base64encode }}               {# Base64 encoded #}
{{ encoded | base64decode }}            {# Base64 decoded #}
```

Chaining Filters

Filters chain left to right:

```
{{ name | trim | lower | capitalize }}
{# "  alice smith  " → "alice smith" → "Alice smith" #}

{{ items | sort | reverse | first }}
{# Sort, reverse, take first = largest item #}
```

The default Filter

A fallback when a variable is empty or undefined:

```
{{ username | default("Guest") }}
{{ bio | default("No bio provided.") }}
{{ theme | default("light") }}
```

5. Control Tags

if / elif / else

```
{% if user.role == "admin" %}
    <span class="badge">Admin</span>
{% elif user.role == "moderator" %}
    <span class="badge">Moderator</span>
{% else %}
    <span class="badge">Member</span>
{% endif %}
```

Comparisons and logical operators:

```
{% if price > 100 and in_stock %}
    <p>Premium item, available now!</p>
{% endif %}
```

```
{% if not user.verified %}
    <p>Please verify your email.</p>
{% endif %}

{% if items | length == 0 %}
    <p>Your cart is empty.</p>
{% endif %}
```

for Loops

```
{% for product in products %}
    <div class="product-card">
        <h3>{{ product.name }}</h3>
        <p>${{ product.price | number_format(2) }}</p>
    </div>
{% endfor %}
```

Inside a for loop, the `loop` variable gives you context:

| Variable | Description |

|----------|----------|

| `loop.index` | Current iteration (1-based) |

| `loop.index0` | Current iteration (0-based) |

| `loop.first` | True on the first iteration |

| `loop.last` | True on the last iteration |

| `loop.length` | Total number of items |

```
<ol>
{% for item in items %}
    <li class="{{ loop.first ? 'first' : '' }} {{ loop.last ? 'last' : '' }}">
        {{ loop.index }}. {{ item.name }}
    </li>
{% endfor %}
</ol>
```

for / else

The `{% else %}` block inside `{% for %}` runs when the list is empty:

```
{% for product in products %}
    <div class="product-card">{{ product.name }}</div>
{% else %}
    <p>No products found.</p>
{% endfor %}
```

set -- Local Variables

Create or update a variable inside a template:

```
{% set greeting = "Hello" %}
{% set full_name = user.first_name ~ " " ~ user.last_name %}

<p>{{ greeting }}, {{ full_name }}!</p>
```

The `~` operator concatenates strings.

6. Template Inheritance

Template inheritance kills duplication. A base template defines blocks. Child templates override them.

Base Template

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    {% block head %}{% endblock %}
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
        <a href="/about">About</a>
    </nav>

    <main>
        {% block content %}{% endblock %}
    </main>

    <footer>
        <p>&copy; 2026 My Store</p>
    </footer>

    <script src="/js/frond.js"></script>
// ...
```

Child Template

Create `src/templates/home.html`:

```
{% extends "base.html" %}

{% block title %}Home - My Store{% endblock %}

{% block content %}
    <h1>Welcome to My Store</h1>
    <p>Browse our collection of quality products.</p>
{% endblock %}
```

When Frond renders `home.html`:

1. It sees `{% extends "base.html" %}` and loads the base template.

2. The `{% block title %}` in `home.html` replaces the one in `base.html`.

3. The `{% block content %}` in `home.html` replaces the one in `base.html`.

4. Blocks not overridden (`head`, `scripts`) keep their default content (empty here).

Calling Parent Blocks

Use `{{ parent() }}` to include the parent block's content:

```
{% extends "base.html" %}

{% block head %}
    {{ parent() }}
```

```
    <link rel="stylesheet" href="/css/products.css">
{% endblock %}
```

This keeps whatever the parent had in `{% block head %}` and adds the extra stylesheet.

7. Include and Macro

include

Pull in another template file:

```
{% include "partials/header.html" %}

<main>
    <h1>Products</h1>
</main>

{% include "partials/footer.html" %}
```

Pass variables to included templates:

```
{% include "partials/product-card.html" with {"product": featured_product} %}
```

macro -- Reusable Template Functions

Macros are functions for templates. Define once, use everywhere:

Create `src/templates/macros/forms.html`:

```
{% macro input(name, label, type="text", value="", required=false) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label }}{% if required %} *{% endif %}</label>
        <input type="{{ type }}" name="{{ name }}" id="{{ name }}"
               value="{{ value }}" {{ required ? "required" : "" }}>
    </div>
{% endmacro %}

{% macro textarea(name, label, value="", rows=4) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label }}</label>
        <textarea name="{{ name }}" id="{{ name }}" rows="{{ rows }}">{{ value }}</textarea>
    </div>
{% endmacro %}

{% macro button(text, type="submit", class="btn-primary") %}
    <button type="{{ type }}" class="btn {{ class }}">{{ text }}</button>
{% endmacro %}
```

Use them:

```
{% import "macros/forms.html" as forms %}

<form method="POST" action="/api/contact">
    {{ forms.input("name", "Your Name", required=true) }}
    {{ forms.input("email", "Email Address", type="email", required=true) }}
    {{ forms.textarea("message", "Your Message", rows=6) }}
    {{ forms.button("Send Message") }}
</form>
```

Consistent markup. Change the macro once and every form in your application updates.

8. Comments

Use `{# #}` for template comments. Stripped from output:

```
{# This comment will not appear in the HTML source #}
<p>Visible content</p>

{#
    Multi-line comments work too.
    Use them to document template logic.
#}
```

Unlike HTML comments (`<!-- -->`), Frond comments never reach the browser.

9. tina4css

The `tina4.css` file is Tina4's built-in CSS utility framework. It ships with every project. Layout utilities. Typography. Spacing. Common UI patterns. No Bootstrap. No Tailwind. No separate download.

Some common classes:

```
{# Grid layout #}
<div class="grid grid-cols-3 gap-4">
    <div class="card">Item 1</div>
    <div class="card">Item 2</div>
    <div class="card">Item 3</div>
</div>

{# Flex layout #}
<div class="flex justify-between items-center">
    <h1>Title</h1>
    <button class="btn btn-primary">Action</button>
</div>

{# Spacing #}
<div class="p-4 m-2">Padded and margined</div>

{# Typography #}
<p class="text-lg text-gray-600">Large gray text</p>
```

The full tina4css reference is in Book 0. For this chapter, inline styles in the examples work fine.

10. Exercise: Build a Product Catalog Page

Build a product catalog page with categories, filtering, and a detail view.

Requirements

1. Create a route at `GET /catalog` that renders a product catalog page

2. Create a route at `GET /catalog/{id:int}` that renders a single product detail page

3. Use template inheritance with a base layout

4. Display products in a grid with:

• Product name, price (formatted to 2 decimal places), category

• "In Stock" / "Out of Stock" badge

• Featured products get a highlighted border

5. Show category filter links at the top (all categories from the data)

6. Support `?category=` query parameter to filter products

7. The detail page shows full product info including description

8. Use at least one macro (e.g., for the product card)

9. Use the `|default` filter somewhere meaningful

Use this data:

```
products = [
    {"id": 1, "name": "Espresso Machine", "category": "Kitchen", "price": 299.99, "in_stock": True, "featured"
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": True, "featured": False,
    {"id": 3, "name": "Standing Desk", "category": "Office", "price": 549.99, "in_stock": True, "featured": T
    {"id": 4, "name": "Noise-Canceling Headphones", "category": "Electronics", "price": 199.99, "in_stock": Fa
    {"id": 5, "name": "Water Bottle", "category": "Fitness", "price": 24.99, "in_stock": True, "featured": Fal
    {"id": 6, "name": "Desk Lamp", "category": "Office", "price": 79.99, "in_stock": True, "featured": False,
]
```

11. Solution

Create `src/templates/macros/catalog.html`:

```
{% macro product_card(product) %}
    <div class="product-card {{ product.featured ? 'featured' : '' }}">
        {% if product.featured %}
            <span class="featured-badge">Featured</span>
        {% endif %}
        <h3><a href="/catalog/{{ product.id }}">{{ product.name }}</a></h3>
        <p class="category">{{ product.category }}</p>
        <p class="price">${{ product.price | number_format(2) }}</p>
        {% if product.in_stock %}
            <span class="badge badge-success">In Stock</span>
        {% else %}
            <span class="badge badge-danger">Out of Stock</span>
        {% endif %}
    </div>
{% endmacro %}
```

Create `src/templates/catalog.html`:

```
{% extends "base.html" %}
{% import "macros/catalog.html" as catalog %}

{% block title %}{{ page_title | default("Product Catalog") }}{% endblock %}

{% block content %}
    <h1>{{ page_title | default("Product Catalog") }}</h1>

    <div class="category-filters">
        <a href="/catalog" class="{{ active_category is not defined ? 'active' : '' }}">All</a>
        {% for cat in categories %}
            <a href="/catalog?category={{ cat }}"
               class="{{ active_category == cat ? 'active' : '' }}">{{ cat }}</a>
        {% endfor %}
    </div>

    <p class="stats">Showing {{ products | length }} product{{ products | length != 1 ? "s" : "" }}</p>
```

```
    <div class="product-grid">
        {% for product in products %}
            {{ catalog.product_card(product) }}
        {% else %}
            <p>No products found in this category.</p>
        {% endfor %}
    </div>
// ...
```

Create `src/templates/product_detail.html`:

```
{% extends "base.html" %}

{% block title %}{{ product.name }} - My Store{% endblock %}

{% block content %}
    <a href="/catalog">&larr; Back to catalog</a>

    <div class="product-detail">
        <h1>{{ product.name }}</h1>
        <p class="category">{{ product.category }}</p>
        <p class="price">${{ product.price | number_format(2) }}</p>
        {% if product.in_stock %}
            <span class="badge badge-success">In Stock</span>
        {% else %}
            <span class="badge badge-danger">Out of Stock</span>
        {% endif %}
        {% if product.featured %}
            <span class="featured-badge">Featured</span>
        {% endif %}
        <div class="description">
            <h2>Description</h2>
            <p>{{ product.description | default("No description available.") }}</p>
        </div>
    </div>
{% endblock %}
```

Create `src/routes/catalog.py`:

```
from tina4_python.core.router import get

products = [
    {"id": 1, "name": "Espresso Machine", "category": "Kitchen", "price": 299.99, "in_stock": True, "featured"
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": True, "featured": False,
    {"id": 3, "name": "Standing Desk", "category": "Office", "price": 549.99, "in_stock": True, "featured": Tr
    {"id": 4, "name": "Noise-Canceling Headphones", "category": "Electronics", "price": 199.99, "in_stock": Fa
    {"id": 5, "name": "Water Bottle", "category": "Fitness", "price": 24.99, "in_stock": True, "featured": Fal
    {"id": 6, "name": "Desk Lamp", "category": "Office", "price": 79.99, "in_stock": True, "featured": False,
]


@get("/catalog")
async def catalog_page(request, response):
    category = request.query.get("category")
    categories = sorted(set(p["category"] for p in products))

    if category:
        filtered = [p for p in products if p["category"].lower() == category.lower()]
    else:
        filtered = products

    return response.render("catalog.html", {
        "products": filtered,
        "categories": categories,
```

```
// ...
```

Open `http://localhost:7145/catalog` in your browser. You should see:

• A heading "Product Catalog"

• Category filter links: All, Electronics, Fitness, Kitchen, Office

• A count of products shown

• Product cards in a grid with names, prices, category labels, and stock badges

• Featured products have a highlighted style and a "Featured" badge

• Clicking a product name navigates to the detail page

• Clicking a category link filters the list

12. Gotchas

1. Whitespace in output

Problem: Your rendered HTML has unexpected blank lines or spaces.

Cause: Template tags produce whitespace on the line they occupy.

Fix: Use whitespace control with `{%-` and `-%}` to strip whitespace around tags:

```
{%- for item in items -%}
    <li>{{ item.name }}</li>
{%- endfor -%}
```

2. Variable not defined error

Problem: Frond raises an error when a variable does not exist in the context.

Cause: You used `{{ user.name }}` but did not pass `user` in the template data.

Fix: Use the `|default` filter: `{{ user.name | default("Guest") }}`. Or check first: `{% if user is defined %}{{ user.name }}{% endif %}`.

3. Extends must be the first tag

Problem: `{% extends "base.html" %}` has no effect and the page renders without the layout.

Cause: `{% extends %}` must be the first tag in the template. Any text, HTML, or tags before it cause Frond to treat the template as standalone.

Fix: Move `{% extends "base.html" %}` to the first line. No content before it.

4. Macro not found

Problem: `{{ forms.input(...) }}` produces an error about `forms` being undefined.

Cause: You forgot the `{% import %}` statement, or the import path is wrong.

Fix: Add `{% import "macros/forms.html" as forms %}` at the top of the template (after `{% extends %}` if using inheritance). The path is relative to `src/templates/`.

5. Filter produces wrong type

Problem: `{{ price | number_format(2) }}` shows an error instead of a formatted number.

Cause: The variable `price` is a string, not a number. Filters expect specific types.

Fix: Pass correct types from your route handler. Use `float(price)` in Python before passing to the template, or convert in the template: `{{ price | float | number_format(2) }}`.

6. Escaped HTML when you want raw output

Problem: Your HTML content shows as text with visible `<tags>` instead of rendering.

Cause: Frond auto-escapes all `{{ }}` output to prevent XSS.

Fix: Use the `|safe` filter: `{{ trusted_html | safe }}`. Only use this with content you trust -- never with user input.

7. Include file path wrong

Problem: `{% include "header.html" %}` gives a "template not found" error even though the file exists.

Cause: The path in `{% include %}` is relative to `src/templates/`, not the current template file.

Fix: Use the full path from the templates root: `{% include "partials/header.html" %}` for a file at `src/templates/partials/header.html`.

Chapter 5: Database

1. From Lists to Real Data

Every example so far stored data in Python lists. Restart the server. Data gone. A real application needs a database.

Tina4 Python makes this painless. Set a `DATABASE_URL` in your `.env`. Create a `Database()` connection. Run SQL. No ORM required (Chapter 6 adds that). Write the SQL you already know.

Picture a notes application. Users create, edit, and delete notes. Those notes need to survive restarts, support searching, and handle concurrent users. That is what a database provides.

2. Configuration

DATABASE_URL

Set your database connection in `.env`:

```
DATABASE_URL=sqlite:///data/app.db
```

That is the default -- a SQLite database stored in `data/`. SQLite comes with Python. No install required.

Tina4 supports six database engines:

| Engine | DATABASE_URL Format | Package Required |

|--------|-------------------|----------------|

| SQLite | `sqlite:///data/app.db` | None (stdlib) |

| PostgreSQL | `postgresql://user:pass@host:5432/dbname` | `psycopg2` |

| MySQL | `mysql://user:pass@host:3306/dbname` | `mysql-connector-python` |

| MSSQL | `mssql://user:pass@host:1433/dbname` | `pymssql` |

| Firebird | `firebird://user:pass@host:3050/path/to/db.fdb` | `firebird-driver` |

| ODBC | `odbc://DSN_NAME` | `pyodbc` |

Switch databases by changing the URL. Your Python code stays the same. All drivers implement the same adapter interface.

Installing Database Drivers

SQLite requires nothing. For other databases, install the driver with uv:

```
# PostgreSQL
uv add psycopg2-binary

# MySQL
uv add mysql-connector-python

# MSSQL
uv add pymssql

# Firebird
uv add firebird-driver
```

```
# ODBC
uv add pyodbc
```

3. Creating a Connection

```
from tina4_python.database.connection import Database

db = Database()
```

That is it. `Database()` reads `DATABASE_URL` from your `.env` and connects. If the SQLite database file does not exist, it creates one.

You can also pass a URL directly:

```
db = Database("sqlite:///data/test.db")
```

4. Running Queries

fetch -- Get Multiple Rows

```
from tina4_python.core.router import get
from tina4_python.database.connection import Database

@get("/api/notes")
async def list_notes(request, response):
    db = Database()
    notes = db.fetch("SELECT id, title, content, created_at FROM notes ORDER BY created_at DESC")

    return response.json({"notes": notes, "count": len(notes)})
```

`fetch()` returns a list of dictionaries. Each dictionary represents a row:

```
{
  "notes": [
    {"id": 1, "title": "Shopping List", "content": "Milk, eggs, bread", "created_at": "2026-03-22 14:30:00"},
    {"id": 2, "title": "Meeting Notes", "content": "Discuss Q2 roadmap", "created_at": "2026-03-22 10:00:00"}
  ],
  "count": 2
}
```

fetch_one -- Get a Single Row

```
@get("/api/notes/{id:int}")
async def get_note(request, response):
    db = Database()
    note = db.fetch_one(
        "SELECT id, title, content, created_at FROM notes WHERE id = :id",
        {"id": request.params["id"]}
    )

    if note is None:
        return response.json({"error": "Note not found"}, 404)

    return response.json(note)
```

`fetch_one()` returns a single dictionary or `None` if no row matches.

execute -- Insert, Update, Delete

```
from tina4_python.core.router import post, put, delete
```

```
from tina4_python.database.connection import Database

@post("/api/notes")
async def create_note(request, response):
    db = Database()
    body = request.body

    if not body.get("title"):
        return response.json({"error": "Title is required"}, 400)

    db.execute(
        "INSERT INTO notes (title, content) VALUES (:title, :content)",
        {"title": body["title"], "content": body.get("content", "")}
    )

    note = db.fetch_one("SELECT * FROM notes WHERE id = last_insert_rowid()")

    return response.json({"message": "Note created", "note": note}, 201)
```

`execute()` runs an INSERT, UPDATE, or DELETE statement and returns the number of affected rows.

5. Parameterised Queries

Never concatenate user input into SQL strings. Parameterised queries are the wall between you and SQL injection:

```
# WRONG -- SQL injection vulnerability
db.fetch(f"SELECT * FROM notes WHERE title = '{user_input}'")

# CORRECT -- parameterised query
db.fetch("SELECT * FROM notes WHERE title = :title", {"title": user_input})
```

Named parameters use the `:name` syntax. Pass a dictionary of values as the second argument:

```
db.fetch(
    "SELECT * FROM notes WHERE category = :category AND created_at > :since",
    {"category": "work", "since": "2026-03-01"}
)
```

Tina4 handles parameter escaping and type conversion for all database engines.

6. Transactions

Multiple operations must succeed or fail together. Transactions enforce that contract:

```
@post("/api/transfer")
async def transfer_funds(request, response):
    db = Database()
    body = request.body

    from_account = body["from_account"]
    to_account = body["to_account"]
    amount = float(body["amount"])

    db.start_transaction()

    try:
        # Deduct from sender
        db.execute(
```

```
            "UPDATE accounts SET balance = balance - :amount WHERE id = :id AND balance >= :amount",
            {"amount": amount, "id": from_account}
        )

        # Check if deduction succeeded
        sender = db.fetch_one(
            "SELECT balance FROM accounts WHERE id = :id",
            {"id": from_account}
        )

        if sender is None:
// ...
```

The three transaction methods:

• `db.start_transaction()` -- begin a transaction

• `db.commit()` -- save all changes since the transaction started

• `db.rollback()` -- undo all changes since the transaction started

Without transactions, each `execute()` call auto-commits on its own.

7. Batch Operations with execute_many

Inserting or updating many rows at once calls for `execute_many()`. It batches operations for speed:

```
@post("/api/notes/import")
async def import_notes(request, response):
    db = Database()
    notes = request.body.get("notes", [])

    if not notes:
        return response.json({"error": "No notes provided"}, 400)

    params_list = [
        {"title": note["title"], "content": note.get("content", "")}
        for note in notes
    ]

    db.execute_many(
        "INSERT INTO notes (title, content) VALUES (:title, :content)",
        params_list
    )

    return response.json({"message": f"Imported {len(notes)} notes"}, 201)
```

```
curl -X POST http://localhost:7145/api/notes/import \
  -H "Content-Type: application/json" \
  -d '{"notes": [{"title": "Note 1", "content": "First"}, {"title": "Note 2", "content": "Second"}, {"title":
```

```
{"message":"Imported 3 notes"}
```

`execute_many()` runs far faster than `execute()` in a loop. One round trip instead of many.

8. Insert, Update, and Delete Helpers

Tina4 provides shorthand methods that cut the boilerplate:

insert

```
last_id = db.insert("notes", {
    "title": "Quick Note",
    "content": "Created with insert helper"
})
```

Returns the last inserted ID.

update

```
affected = db.update("notes", {"title": "Updated Title", "content": "New content"}, "id = :id", {"id": 1})
```

The third argument is the WHERE clause, the fourth is its parameters. Returns the number of affected rows.

delete

```
affected = db.delete("notes", "id = :id", {"id": 1})
```

Returns the number of deleted rows.

These helpers eliminate boilerplate INSERT/UPDATE/DELETE SQL. For complex queries, `execute()` is always there.

9. Migrations

Migrations are SQL files that version your database schema. Write them once. Apply them in order. Roll them back when needed.

Creating a Migration

```
tina4 migrate:create create_notes_table
```

```
Created migration: src/migrations/20260322160000_create_notes_table.sql
```

Open the generated file and add your SQL:

```
-- UP
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT DEFAULT '',
    category TEXT DEFAULT 'general',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_notes_category ON notes (category);
CREATE INDEX idx_notes_created_at ON notes (created_at);

-- DOWN
DROP INDEX IF EXISTS idx_notes_created_at;
DROP INDEX IF EXISTS idx_notes_category;
DROP TABLE IF EXISTS notes;
```

The `-- UP` section runs when applying the migration. The `-- DOWN` section runs when rolling it back.

Running Migrations

```
tina4 migrate
```

```
Running migrations...
  Applied: 20260322160000_create_notes_table.sql
Done. 1 migration applied.
```

Rolling Back

```
tina4 migrate:rollback
```

```
Rolling back last migration...
  Rolled back: 20260322160000_create_notes_table.sql
Done.
```

Migration Best Practices

1. **One migration per change** -- do not pile multiple table changes into one file
2. **Always write a DOWN section** -- so you can roll back cleanly
3. **Never edit a migration that has been applied** -- create a new migration instead
4. **Use descriptive names** -- `create_users_table`, `add_email_to_orders`, `create_category_index`

10. Query Caching

Expensive queries that return the same result on every call deserve caching. Tina4 builds it in:

```
from tina4_python.database.connection import Database

db = Database()

# Cache this query result for 300 seconds (5 minutes)
categories = db.fetch(
    "SELECT DISTINCT category, COUNT(*) as count FROM notes GROUP BY category",
    cache_ttl=300
)
```

The first call runs the SQL and caches the result. Subsequent calls within the TTL skip the database entirely.

To invalidate the cache when data changes:

```
@post("/api/notes")
async def create_note(request, response):
    db = Database()
    db.execute(
        "INSERT INTO notes (title, content, category) VALUES (:title, :content, :category)",
        {"title": request.body["title"], "content": request.body.get("content", ""), "category": request.body
    )

    # Clear cached queries that might be stale
    db.clear_cache()

    note = db.fetch_one("SELECT * FROM notes WHERE id = last_insert_rowid()")
    return response.json({"note": note}, 201)
```

11. Exercise: Build a Notes App API

Build a complete notes application API with database persistence.

Requirements

1. Create a migration for a `notes` table with: `id`, `title` (required), `content`, `category` (default "general"), `pinned` (boolean, default false), `created_at`, `updated_at`

2. Build these endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/notes` | List all notes. Support `?category=` and `?pinned=true` filters. |

| `GET` | `/api/notes/{id:int}` | Get a single note. 404 if not found. |

| `POST` | `/api/notes` | Create a note. Title required. Return 201. |

| `PUT` | `/api/notes/{id:int}` | Update a note. Return 404 if not found. |

| `DELETE` | `/api/notes/{id:int}` | Delete a note. Return 204. |

| `GET` | `/api/notes/categories` | List all distinct categories with counts. |

Test with:

```
# Create notes
curl -X POST http://localhost:7145/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "Shopping List", "content": "Milk, eggs, bread", "category": "personal"}'

curl -X POST http://localhost:7145/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "Sprint Planning", "content": "Review backlog, assign tasks", "category": "work", "pinned": tr

# List all
curl http://localhost:7145/api/notes

# Filter by category
curl "http://localhost:7145/api/notes?category=work"

# Filter by pinned
curl "http://localhost:7145/api/notes?pinned=true"

# Get categories
curl http://localhost:7145/api/notes/categories

# Update
curl -X PUT http://localhost:7145/api/notes/1 \
  -H "Content-Type: application/json" \
  -d '{"title": "Updated Shopping List", "content": "Milk, eggs, bread, butter"}'
// ...
```

12. Solution

Migration

Create `src/migrations/20260322160000_create_notes_table.sql`:

```
-- UP
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
```

```
    content TEXT DEFAULT '',
    category TEXT NOT NULL DEFAULT 'general',
    pinned INTEGER NOT NULL DEFAULT 0,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_notes_category ON notes (category);
CREATE INDEX idx_notes_pinned ON notes (pinned);

-- DOWN
DROP INDEX IF EXISTS idx_notes_pinned;
DROP INDEX IF EXISTS idx_notes_category;
DROP TABLE IF EXISTS notes;
```

Run the migration:

```
tina4 migrate
```

Routes

Create `src/routes/notes.py`:

```
from tina4_python.core.router import get, post, put, delete
from tina4_python.database.connection import Database


@get("/api/notes")
async def list_notes(request, response):
    db = Database()
    category = request.query.get("category")
    pinned = request.query.get("pinned")

    sql = "SELECT * FROM notes"
    params = {}
    conditions = []

    if category:
        conditions.append("category = :category")
        params["category"] = category

    if pinned == "true":
        conditions.append("pinned = 1")

    if conditions:
        sql += " WHERE " + " AND ".join(conditions)

    sql += " ORDER BY pinned DESC, created_at DESC"
// ...
```

Expected output for creating a note:

```
{
  "message": "Note created",
  "note": {
    "id": 1,
    "title": "Shopping List",
    "content": "Milk, eggs, bread",
    "category": "personal",
    "pinned": 0,
    "created_at": "2026-03-22 16:00:00",
    "updated_at": "2026-03-22 16:00:00"
  }
}
```

(Status: `201 Created`)

Expected output for categories:

```
{
  "categories": [
    {"category": "personal", "count": 1},
    {"category": "work", "count": 1}
  ]
}
```

13. Gotchas

1. SQLite boolean quirk

Problem: Boolean values come back as `0` and `1` instead of `false` and `true` in JSON.

Cause: SQLite does not have a native boolean type. It stores booleans as integers.

Fix: This is expected behavior. In your route handler, you can convert them: `note["pinned"] = bool(note["pinned"])`. Or handle it in the frontend. The ORM (Chapter 6) does this conversion automatically with `BooleanField`.

2. last_insert_rowid() is SQLite-specific

Problem: `SELECT * FROM notes WHERE id = last_insert_rowid()` does not work on PostgreSQL or MySQL.

Cause: `last_insert_rowid()` is a SQLite function. Other databases use different mechanisms.

Fix: Use `db.insert()` which returns the last inserted ID regardless of database engine. Or use database-specific syntax: PostgreSQL uses `RETURNING id` in the INSERT statement, MySQL uses `LAST_INSERT_ID()`.

3. String vs integer comparison

Problem: `WHERE id = :id` does not find the row even though the ID exists.

Cause: Path parameters come as strings by default. If `id` is `"5"` (string) and the column is an integer, some databases handle this differently.

Fix: Use typed path parameters (`{id:int}`) so the value is already an integer, or explicitly cast: `{"id": int(request.params["id"])}`.

4. Connection not closed

Problem: After many requests, the application runs out of database connections.

Cause: You are creating `Database()` instances without them being properly cleaned up.

Fix: Tina4's `Database()` manages connection pooling internally. In most cases, creating `Database()` in each handler is fine because it reuses connections from the pool. If you are seeing connection issues, check that you are not holding transactions open for too long.

5. Migration order matters

Problem: A migration fails because it references a table that does not exist yet.

Cause: Migrations run in alphabetical/timestamp order. If migration B depends on the table created by migration A, migration A must have an earlier timestamp.

Fix: Use the `tina4 migrate:create` command, which auto-generates timestamps. Migrations are named `YYYYMMDDHHMMSS_description.sql` and run in chronological order.

6. Forgetting the DOWN section

Problem: `tina4 migrate:rollback` fails or does nothing.

Cause: The migration file does not have a `-- DOWN` section.

Fix: Always include a `-- DOWN` section in every migration. It should undo exactly what `-- UP` did. For `CREATE TABLE`, the DOWN is `DROP TABLE IF EXISTS`. For `ALTER TABLE ADD COLUMN`, the DOWN is `ALTER TABLE DROP COLUMN` (though SQLite does not support dropping columns -- in that case, you may need to recreate the table).

7. SQL injection through string formatting

Problem: Your application is vulnerable to SQL injection attacks.

Cause: You used f-strings or string concatenation to build SQL queries with user input: `f"WHERE name = '{name}'"`.

Fix: Always use parameterised queries: `"WHERE name = :name", {"name": name}`. This is the single most important security practice for database code. Tina4 will handle escaping and quoting for you.

Chapter 6: ORM

1. From SQL to Objects

The last chapter was raw SQL. It works. It also gets repetitive. Every insert: an INSERT statement. Every update: an UPDATE. Every fetch: column names mapped to dictionary keys. Over and over.

Tina4's ORM turns database rows into Python objects. Define a model class with fields. The ORM writes the SQL. It remains SQL-first -- you can drop to raw SQL any time -- but for the 90% case of CRUD operations, the ORM handles the grunt work.

Picture a blog. Authors, posts, comments. Authors own many posts. Posts own many comments. Comments belong to posts. Modeling these relationships with raw SQL means JOINs and manual foreign key management. The ORM makes this declarative.

2. Defining a Model

Create a model file in `src/orm/`. Every `.py` file in that directory is auto-loaded.

Create `src/orm/note.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, BooleanField, DateTimeField

class Note(ORM):
    table_name = "notes"
    primary_key = "id"

    id = IntegerField(auto_increment=True)
    title = StringField(required=True, max_length=200)
    content = StringField(default="")
    category = StringField(default="general")
    pinned = BooleanField(default=False)
    created_at = DateTimeField(auto_now_add=True)
    updated_at = DateTimeField(auto_now=True)
```

A complete model. Here is what each piece does:

• `table_name` -- the database table this model maps to

• `primary_key` -- the primary key column (defaults to `"id"`)

• Each field is a class-level attribute with a field type

Field Types

| Field Type | Python Type | SQL Type | Description |

|----------|----------|----------|----------|

| `IntegerField` | `int` | `INTEGER` | Whole numbers |

| `StringField` | `str` | `TEXT` or `VARCHAR` | Text strings |

| `NumericField` | `float` | `REAL` or `NUMERIC` | Decimal numbers |

| `BooleanField` | `bool` | `INTEGER` (0/1) | True/False |

| `DateTimeField` | `str` | `TEXT` or `TIMESTAMP` | Date and time |

| `TextField` | `str` | `TEXT` | Long text |

| `BlobField` | `bytes` | `BLOB` | Binary data |

| `ForeignKeyField` | `int` | `INTEGER` | Foreign key reference |

Verbose names (`IntegerField`, `StringField`, `BooleanField`) are the standard. Short aliases (`IntField`, `StrField`, `BoolField`) also work.

Field Options

| Option | Type | Description |

|--------|------|-------------|

| `required` | `bool` | Field must have a value (not None) |

| `default` | any | Default value when not provided |

| `max_length` | `int` | Maximum string length |

| `min_length` | `int` | Minimum string length |

| `min_value` | number | Minimum numeric value |

| `max_value` | number | Maximum numeric value |

| `choices` | list | Allowed values |

| `auto_increment` | `bool` | Auto-incrementing integer |

| `auto_now_add` | `bool` | Set to current time on create |

| `auto_now` | `bool` | Set to current time on every save |

| `regex` | `str` | Pattern the value must match |

| `validator` | callable | Custom validation function |

3. create_table -- Schema from Models

You can create the database table directly from your model definition:

```
Note.create_table()
```

This generates and runs the CREATE TABLE SQL based on your field definitions. Good for development and testing. For production, use migrations (Chapter 5) for version-controlled schema changes.

```
tina4 shell
>>> from src.orm.note import Note
>>> Note.create_table()
```

4. CRUD Operations

save -- Create or Update

```
from tina4_python.core.router import post, put
from src.orm.note import Note

@post("/api/notes")
```

```
async def create_note(request, response):
    note = Note()
    note.title = request.body["title"]
    note.content = request.body.get("content", "")
    note.category = request.body.get("category", "general")
    note.pinned = request.body.get("pinned", False)
    note.save()

    return response.json({"message": "Note created", "note": note.to_dict()}, 201)
```

`save()` detects whether the record is new (INSERT) or existing (UPDATE) based on whether the primary key has a value.

To update an existing record:

```
@put("/api/notes/{id:int}")
async def update_note(request, response):
    note = Note()
    note.load("id = :id", {"id": request.params["id"]})

    if note.id is None:
        return response.json({"error": "Note not found"}, 404)

    body = request.body
    if "title" in body:
        note.title = body["title"]
    if "content" in body:
        note.content = body["content"]
    if "category" in body:
        note.category = body["category"]
    if "pinned" in body:
        note.pinned = body["pinned"]

    note.save()

    return response.json({"message": "Note updated", "note": note.to_dict()})
```

load -- Fetch One Record

```
from tina4_python.core.router import get
from src.orm.note import Note

@get("/api/notes/{id:int}")
async def get_note(request, response):
    note = Note()
    note.load("id = :id", {"id": request.params["id"]})

    if note.id is None:
        return response.json({"error": "Note not found"}, 404)

    return response.json(note.to_dict())
```

`load()` takes a WHERE clause and parameters. It populates the model instance with the first matching row. If no row matches, the primary key remains `None`.

delete -- Remove a Record

```
from tina4_python.core.router import delete as delete_route
from src.orm.note import Note

@delete_route("/api/notes/{id:int}")
async def delete_note(request, response):
    note = Note()
    note.load("id = :id", {"id": request.params["id"]})
```

```
    if note.id is None:
        return response.json({"error": "Note not found"}, 404)

    note.delete()

    return response.json(None, 204)
```

select -- Fetch Multiple Records

```
@get("/api/notes")
async def list_notes(request, response):
    category = request.query.get("category")

    if category:
        result = Note.select(
            "category = :category",
            {"category": category},
            order_by="created_at DESC"
        )
    else:
        result = Note.select(order_by="pinned DESC, created_at DESC")

    notes = [note.to_dict() for note in result]

    return response.json({"notes": notes, "count": len(notes)})
```

`select()` returns a list of model instances. You can pass a filter, parameters, ordering, and pagination:

```
# With pagination
result = Note.select(
    order_by="created_at DESC",
    limit=20,
    skip=40
)

# Paginated result with metadata
paginated = Note.to_paginate(
    limit=20,
    skip=0,
    order_by="created_at DESC"
)
# paginated = {"data": [...], "total": 150, "page": 1, "per_page": 20, ...}
```

5. to_dict and to_json

to_dict

Convert a model instance to a dictionary:

```
note = Note()
note.load("id = :id", {"id": 1})

data = note.to_dict()
# {"id": 1, "title": "Shopping List", "content": "Milk, eggs", "category": "personal", "pinned": False, "creat
```

You can include or exclude specific fields:

```
# Only include these fields
data = note.to_dict(include=["id", "title", "category"])
# {"id": 1, "title": "Shopping List", "category": "personal"}

# Exclude sensitive fields
```

```
data = note.to_dict(exclude=["updated_at"])
```

to_json

Convert directly to a JSON string:

```
json_string = note.to_json()
# '{"id": 1, "title": "Shopping List", ...}'
```

6. Relationships

has_many

An author has many posts:

Create `src/orm/author.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField

class Author(ORM):
    table_name = "authors"

    id = IntegerField(auto_increment=True)
    name = StringField(required=True)
    email = StringField(required=True)
    bio = StringField(default="")
    created_at = DateTimeField(auto_now_add=True)
```

Create `src/orm/blog_post.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField, ForeignKeyField

class BlogPost(ORM):
    table_name = "posts"

    id = IntegerField(auto_increment=True)
    author_id = ForeignKeyField("authors.id", required=True)
    title = StringField(required=True, max_length=300)
    slug = StringField(required=True)
    content = StringField(default="")
    status = StringField(default="draft", choices=["draft", "published", "archived"])
    created_at = DateTimeField(auto_now_add=True)
    updated_at = DateTimeField(auto_now=True)
```

Now use `has_many` to get an author's posts:

```
@get("/api/authors/{id:int}")
async def get_author(request, response):
    author = Author()
    author.load("id = :id", {"id": request.params["id"]})

    if author.id is None:
        return response.json({"error": "Author not found"}, 404)

    posts = author.has_many(BlogPost, "author_id")

    data = author.to_dict()
    data["posts"] = [post.to_dict(include=["id", "title", "slug", "status"]) for post in posts]

    return response.json(data)
```

```
{
  "id": 1,
```

```
  "name": "Alice",
  "email": "alice@example.com",
  "bio": "Tech writer",
  "posts": [
    {"id": 1, "title": "Getting Started with Tina4", "slug": "getting-started", "status": "published"},
    {"id": 2, "title": "Advanced Routing", "slug": "advanced-routing", "status": "draft"}
  ]
}
```

has_one

A user has one profile:

```
profile = user.has_one(Profile, "user_id")
```

Returns a single model instance or `None`.

belongs_to

A post belongs to an author:

```
@get("/api/posts/{id:int}")
async def get_post(request, response):
    post = BlogPost()
    post.load("id = :id", {"id": request.params["id"]})

    if post.id is None:
        return response.json({"error": "Post not found"}, 404)

    author = post.belongs_to(Author, "author_id")

    data = post.to_dict()
    data["author"] = author.to_dict(include=["id", "name", "email"]) if author else None

    return response.json(data)
```

```
{
  "id": 1,
  "author_id": 1,
  "title": "Getting Started with Tina4",
  "slug": "getting-started",
  "content": "...",
  "status": "published",
  "author": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

Eager Loading with include

To avoid N+1 queries, use `include` to eager-load relationships:

```
@get("/api/authors")
async def list_authors(request, response):
    authors = Author.select(
        order_by="name ASC",
        include=[{"model": BlogPost, "foreign_key": "author_id", "as": "posts"}]
    )

    data = []
    for author in authors:
        author_dict = author.to_dict()
        author_dict["posts"] = [p.to_dict(include=["id", "title", "status"]) for p in author.posts]
```

```
        data.append(author_dict)

    return response.json({"authors": data})
```

Without eager loading, 10 authors and their posts cost 11 queries (1 for authors + 10 for posts). With eager loading: 2 queries. That is the difference between a fast page and a slow one.

7. Soft Delete

Sometimes a record needs to disappear from queries without leaving the database. Soft delete handles this:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField

class Task(ORM):
    table_name = "tasks"
    soft_delete = True  # Enable soft delete

    id = IntegerField(auto_increment=True)
    title = StringField(required=True)
    completed = BooleanField(default=False)
    deleted_at = DateTimeField()  # Required for soft delete
    created_at = DateTimeField(auto_now_add=True)
```

When `soft_delete = True`:

• `task.delete()` sets `deleted_at` to the current timestamp instead of running a DELETE query

• `Task.select()` automatically filters out soft-deleted records

• `task.restore()` clears `deleted_at` and makes the record visible again

• `task.force_delete()` permanently removes the record

• `Task.select(with_trashed=True)` includes soft-deleted records

```
# Soft delete
task.delete()  # Sets deleted_at, does not remove the row

# Restore
task.restore()  # Clears deleted_at

# Permanently delete
task.force_delete()  # Actually removes the row

# Query including soft-deleted
all_tasks = Task.select(with_trashed=True)
```

8. Scopes

Scopes are reusable query filters baked into the model:

```
class BlogPost(ORM):
    table_name = "posts"

    id = IntegerField(auto_increment=True)
    title = StringField(required=True)
    status = StringField(default="draft")
    created_at = DateTimeField(auto_now_add=True)
```

```
    @classmethod
    def published(cls):
        return cls.select("status = :status", {"status": "published"}, order_by="created_at DESC")

    @classmethod
    def drafts(cls):
        return cls.select("status = :status", {"status": "draft"}, order_by="created_at DESC")

    @classmethod
    def recent(cls, days=7):
        return cls.select(
            "created_at > datetime('now', :days)",
            {"days": f"-{days} days"},
            order_by="created_at DESC"
        )
```

Use them in your routes:

```
@get("/api/posts/published")
async def published_posts(request, response):
    posts = BlogPost.published()
    return response.json({"posts": [p.to_dict() for p in posts]})

@get("/api/posts/recent")
async def recent_posts(request, response):
    days = int(request.query.get("days", 7))
    posts = BlogPost.recent(days)
    return response.json({"posts": [p.to_dict() for p in posts]})
```

9. Input Validation

Field definitions carry validation rules. Call `validate()` before `save()` and the ORM checks every constraint:

```
from tina4_python.orm import ORM, IntegerField, StringField, NumericField

class Product(ORM):
    table_name = "products"

    id = IntegerField(auto_increment=True)
    name = StringField(required=True, min_length=2, max_length=200)
    sku = StringField(required=True, regex=r"^[A-Z]{2}-\d{4}$")  # e.g., EL-1234
    price = NumericField(required=True, min_value=0.01, max_value=999999.99)
    category = StringField(choices=["Electronics", "Kitchen", "Office", "Fitness"])

@post("/api/products")
async def create_product(request, response):
    product = Product()
    product.name = request.body.get("name")
    product.sku = request.body.get("sku")
    product.price = request.body.get("price")
    product.category = request.body.get("category")

    errors = product.validate()
    if errors:
        return response.json({"errors": errors}, 400)

    product.save()
    return response.json({"product": product.to_dict()}, 201)
```

If validation fails, `validate()` returns a list of error messages:

```
{
  "errors": [
    "name: Must be at least 2 characters",
    "sku: Must match pattern ^[A-Z]{2}-\\d{4}$",
    "price: Must be at least 0.01",
    "category: Must be one of: Electronics, Kitchen, Office, Fitness"
  ]
}
```

10. Exercise: Build a Blog with Relationships

Build a blog API with authors, posts, and comments.

Requirements

1. Create these models:

Author: `id`, `name` (required), `email` (required), `bio`, `created_at`

Post: `id`, `author_id` (foreign key), `title` (required, max 300), `slug` (required), `content`, `status` (choices: draft/published/archived, default draft), `created_at`, `updated_at`

Comment: `id`, `post_id` (foreign key), `author_name` (required), `author_email` (required), `body` (required, min 5 chars), `created_at`

2. Build these endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `POST` | `/api/authors` | Create an author |

| `GET` | `/api/authors/{id:int}` | Get author with their posts |

| `POST` | `/api/posts` | Create a post (requires author_id) |

| `GET` | `/api/posts` | List published posts with author info |

| `GET` | `/api/posts/{id:int}` | Get post with author and comments |

| `POST` | `/api/posts/{id:int}/comments` | Add comment to a post |

11. Solution

Create `src/orm/author.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField

class Author(ORM):
    table_name = "authors"

    id = IntegerField(auto_increment=True)
    name = StringField(required=True, min_length=2)
    email = StringField(required=True)
    bio = StringField(default="")
    created_at = DateTimeField(auto_now_add=True)
```

Create `src/orm/blog_post.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField, ForeignKeyField

class BlogPost(ORM):
    table_name = "posts"

    id = IntegerField(auto_increment=True)
    author_id = ForeignKeyField("authors.id", required=True)
    title = StringField(required=True, max_length=300)
    slug = StringField(required=True)
    content = StringField(default="")
    status = StringField(default="draft", choices=["draft", "published", "archived"])
    created_at = DateTimeField(auto_now_add=True)
    updated_at = DateTimeField(auto_now=True)

    @classmethod
    def published(cls):
        return cls.select("status = :status", {"status": "published"}, order_by="created_at DESC")
```

Create `src/orm/comment.py`:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField, ForeignKeyField

class Comment(ORM):
    table_name = "comments"

    id = IntegerField(auto_increment=True)
    post_id = ForeignKeyField("posts.id", required=True)
    author_name = StringField(required=True)
    author_email = StringField(required=True)
    body = StringField(required=True, min_length=5)
    created_at = DateTimeField(auto_now_add=True)
```

Create `src/routes/blog.py`:

```
from tina4_python.core.router import get, post
from src.orm.author import Author
from src.orm.blog_post import BlogPost
from src.orm.comment import Comment


@post("/api/authors")
async def create_author(request, response):
    author = Author()
    author.name = request.body.get("name")
    author.email = request.body.get("email")
    author.bio = request.body.get("bio", "")

    errors = author.validate()
    if errors:
        return response.json({"errors": errors}, 400)

    author.save()
    return response.json({"author": author.to_dict()}, 201)


@get("/api/authors/{id:int}")
async def get_author(request, response):
    author = Author()
    author.load("id = :id", {"id": request.params["id"]})
// ...
```

12. Gotchas

1. Forgetting to call save()

Problem: You set properties on a model but the database does not change.

Cause: Setting `note.title = "New Title"` only changes the Python object. The database is not updated until you call `note.save()`.

Fix: Always call `save()` after modifying properties.

2. load() returns None silently

Problem: You call `note.load(...)` but the note object has all None fields, and you do not get an error.

Cause: `load()` does not raise an exception when no row matches. It just leaves the object empty.

Fix: Always check the primary key after `load()`: `if note.id is None: return 404`.

3. Circular imports with relationships

Problem: `from src.orm.post import BlogPost` in `author.py` and `from src.orm.author import Author` in `post.py` causes an `ImportError`.

Cause: Python cannot handle circular imports at module level.

Fix: Import inside the method that uses the relationship, not at the top of the file. Or pass the model class as a parameter in the route handler where you use both models.

4. to_dict() includes everything

Problem: `user.to_dict()` includes `password_hash` in the API response.

Cause: `to_dict()` includes all fields by default.

Fix: Use the `include` or `exclude` parameter: `user.to_dict(exclude=["password_hash"])`. Make this a habit for any model with sensitive fields.

5. Validation only runs on validate()

Problem: You call `save()` without calling `validate()` first, and invalid data gets into the database.

Cause: `save()` does not automatically validate. This is by design -- sometimes you need to save partial data or bypass validation for bulk operations.

Fix: Always call `errors = model.validate()` before `save()` in your route handlers. Or create a helper method that validates and saves in one step.

6. Foreign key not enforced

Problem: You save a post with `author_id = 999` and it succeeds, even though no author with ID 999 exists.

Cause: SQLite does not enforce foreign key constraints by default. The `ForeignKeyField` in the ORM defines the relationship for Tina4's relationship methods, but the database itself may not enforce it.

Fix: Enable SQLite foreign keys with `PRAGMA foreign_keys = ON;` in a migration, or validate the foreign key in your route handler before saving.

7. N+1 query problem

Problem: Listing 100 authors with their posts runs 101 queries (1 for authors + 100 for posts), making the page slow.

Cause: You are calling `author.has_many(BlogPost, "author_id")` inside a loop for each author.

Fix: Use eager loading with the `include` parameter on `select()`, or fetch all posts in a single query and group them manually:

```
authors = Author.select()
all_posts = BlogPost.select("author_id IN (" + ",".join(str(a.id) for a in authors) + ")")
posts_by_author = {}
for post in all_posts:
    posts_by_author.setdefault(post.author_id, []).append(post)
```

Chapter 7: Authentication

1. Locking the Door

Every endpoint built so far is public. Anyone with the URL can read, create, update, and delete data. Fine for a tutorial. Unacceptable for production.

A real application needs to know two things: who is making the request, and whether they are allowed to make it. This chapter covers Tina4's authentication system. JWT tokens. Password hashing. Middleware-based route protection. CSRF tokens for forms. Session management.

2. JWT Tokens

Tina4 uses JSON Web Tokens (JWT) for authentication. A JWT is a signed string carrying a payload -- user ID, role, expiry. The server mints the token at login. The client sends it with every request. The server verifies the signature without touching the database.

Generating a Token

```
from tina4_python.auth import Auth

payload = {
    "user_id": 42,
    "email": "alice@example.com",
    "role": "admin"
}

token = Auth.get_token(payload)
```

`get_token()` signs the payload with a secret key and returns a JWT string like:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo0MiwiZW1haWwiOiJhbGljZUBleGFtcGxlLmNvbSIsInJvbGUiOiJhZG1pb
```

The token has three parts separated by dots: header, payload, and signature. The signature ensures the token has not been tampered with.

Token Expiry

By default, tokens expire after 1 hour. Configure this in `.env`:

```
TINA4_JWT_EXPIRY=3600
```

The value is in seconds. Common settings:

| Value | Duration |

|-------|----------|

| `900` | 15 minutes |

| `3600` | 1 hour (default) |

| `86400` | 24 hours |

| `604800` | 7 days |

Validating a Token

```
is_valid = Auth.valid_token(token)
# Returns True if the token is valid and not expired, False otherwise
```

Reading the Payload

```
payload = Auth.get_payload(token)
```

Returns the original payload dictionary:

```
{
    "user_id": 42,
    "email": "alice@example.com",
    "role": "admin",
    "iat": 1711112600,  # issued at (Unix timestamp)
    "exp": 1711116200   # expires at (Unix timestamp)
}
```

If the token is invalid or expired, `get_payload()` returns `None`.

The Secret Key

Tina4 uses a secret key to sign and verify tokens. On first run, it automatically generates a random key and stores it in `secrets/jwt.key`. You can also set it explicitly:

```
TINA4_JWT_SECRET=my-very-long-and-random-secret-key-at-least-32-chars
```

Keep this key secret. If someone gets it, they can forge tokens.

3. Password Hashing

Plain-text passwords are a liability. Tina4 provides two functions for secure password handling:

Hashing a Password

```
from tina4_python.auth import Auth

hashed = Auth.hash_password("my-secure-password")
# Returns: "$2b$12$abc123...long-hash-string..."
```

Each hash includes a random salt, so hashing the same password twice produces different results.

Checking a Password

```
is_correct = Auth.check_password("my-secure-password", stored_hash)
# Returns True if the password matches the hash
```

Registration Example

```
from tina4_python.core.router import post, noauth
from tina4_python.auth import Auth
from tina4_python.database.connection import Database

@post("/api/register")
@noauth
async def register(request, response):
    body = request.body

    # Validate input
    if not body.get("name") or not body.get("email") or not body.get("password"):
        return response.json({"error": "Name, email, and password are required"}, 400)

    if len(body["password"]) < 8:
```

```
        return response.json({"error": "Password must be at least 8 characters"}, 400)

    db = Database()

    # Check if email already exists
    existing = db.fetch_one("SELECT id FROM users WHERE email = :email", {"email": body["email"]})
    if existing is not None:
        return response.json({"error": "Email already registered"}, 409)

    # Hash the password
    password_hash = Auth.hash_password(body["password"])
// ...
```

```
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"}
}
```

4. The Login Flow

The complete login flow. Client sends credentials. Server validates them. Server returns a JWT token.

```
from tina4_python.core.router import post, noauth
from tina4_python.auth import Auth
from tina4_python.database.connection import Database

@post("/api/login")
@noauth
async def login(request, response):
    body = request.body

    if not body.get("email") or not body.get("password"):
        return response.json({"error": "Email and password are required"}, 400)

    db = Database()

    # Find the user
    user = db.fetch_one(
        "SELECT id, name, email, password_hash FROM users WHERE email = :email",
        {"email": body["email"]}
    )

    if user is None:
        return response.json({"error": "Invalid email or password"}, 401)

    # Check the password
    if not Auth.check_password(body["password"], user["password_hash"]):
// ...
```

Notice `@noauth`. The login endpoint must be public. You cannot require a token to get a token.

```
curl -X POST http://localhost:7145/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
```

```
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

The client stores this token (in localStorage, a cookie, or memory) and sends it with subsequent requests.

5. Using Tokens in Requests

The client sends the token in the `Authorization` header with the `Bearer` prefix:

```
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

6. Protecting Routes

Auth Middleware

Create a reusable auth middleware:

```
from tina4_python.auth import Auth

async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")

    if not auth_header or not auth_header.startswith("Bearer "):
        return response.json({"error": "Authorization header required"}, 401)

    token = auth_header[7:]  # Remove "Bearer " prefix

    if not Auth.valid_token(token):
        return response.json({"error": "Invalid or expired token"}, 401)

    payload = Auth.get_payload(token)
    request.user = payload  # Attach user data to the request

    return await next_handler(request, response)
```

Applying Middleware to Routes

```
from tina4_python.core.router import get, middleware

@get("/api/profile")
@middleware(auth_middleware)
async def profile(request, response):
    return response.json({
        "user_id": request.user["user_id"],
        "email": request.user["email"],
        "name": request.user["name"]
    })
```

```
# Without token -- 401
curl http://localhost:7145/api/profile
```

```
{"error":"Authorization header required"}
```

```
# With valid token -- 200
curl http://localhost:7145/api/profile \
```

```
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

{
  "user_id": 1,
  "email": "alice@example.com",
  "name": "Alice"
}
```

Applying Middleware to a Group

```
from tina4_python.core.router import get, group, middleware

@group("/api/admin")
@middleware(auth_middleware)
def admin_routes():

    @get("/stats")
    async def admin_stats(request, response):
        return response.json({"active_users": 42})

    @get("/logs")
    async def admin_logs(request, response):
        return response.json({"logs": []})
```

Every route in the group requires a valid token.

7. @noauth and @secured Decorators

As introduced in Chapter 2, these decorators control authentication at the route level.

@noauth -- Skip Authentication

Use `@noauth` for public endpoints that should bypass any global or group-level auth:

```
from tina4_python.core.router import get, post, noauth

@get("/api/public/health")
@noauth
async def health(request, response):
    return response.json({"status": "ok"})

@post("/api/login")
@noauth
async def login(request, response):
    # Login logic
    pass

@post("/api/register")
@noauth
async def register(request, response):
    # Registration logic
    pass
```

@secured -- Require Authentication for GET Routes

By default, POST, PUT, PATCH, and DELETE routes are considered secured. GET routes are public. Use `@secured` to explicitly protect a GET route:

```
from tina4_python.core.router import get, secured

@get("/api/me")
@secured
```

```
async def me(request, response):
    # This GET route requires authentication
    return response.json(request.user)
```

Role-Based Authorization

Combine auth middleware with role checks:

```
from tina4_python.auth import Auth

def require_role(role):
    async def role_middleware(request, response, next_handler):
        auth_header = request.headers.get("Authorization", "")
        if not auth_header or not auth_header.startswith("Bearer "):
            return response.json({"error": "Authorization required"}, 401)

        token = auth_header[7:]
        if not Auth.valid_token(token):
            return response.json({"error": "Invalid or expired token"}, 401)

        payload = Auth.get_payload(token)

        if payload.get("role") != role:
            return response.json({"error": f"Forbidden. Required role: {role}"}, 403)

        request.user = payload
        return await next_handler(request, response)

    return role_middleware
```

Use it as middleware:

```
from tina4_python.core.router import delete as delete_route, middleware

@delete_route("/api/users/{id:int}")
@middleware(require_role("admin"))
async def delete_user(request, response):
    # Only admins can delete users
    return response.json({"deleted": True})
```

8. CSRF Protection

Traditional form-based applications (not SPAs) need CSRF protection. Tina4 provides it through form tokens.

Generating a Token

In your template, include the CSRF token in every form:

```
<form method="POST" action="/profile/update">
    {{ form_token() }}

    <div class="form-group">
        <label for="name">Name</label>
        <input type="text" name="name" id="name" value="{{ user.name }}">
    </div>

    <button type="submit">Update Profile</button>
</form>
```

`{{ form_token() }}` renders a hidden input field:

```
<input type="hidden" name="_token" value="abc123randomtoken456">
```

Validating the Token

In your route handler, check the token:

```
from tina4_python.core.router import post
from tina4_python.auth import Auth

@post("/profile/update")
async def update_profile(request, response):
    # Validate CSRF token
    if not Auth.validate_form_token(request.body.get("_token", "")):
        return response.json({"error": "Invalid form token. Please refresh and try again."}, 403)

    # Process the form...
    return response.redirect("/profile")
```

The CSRF token is tied to the session and expires after a single use. A malicious site cannot forge a form submission because it cannot guess the token.

When to Use CSRF Tokens

Use CSRF tokens for:

• HTML forms submitted by browsers

• Any POST/PUT/DELETE from server-rendered pages

You do not need CSRF tokens for:

• API endpoints that use JWT (the Bearer token already proves the request is intentional)

• Single-page applications that use `fetch()` with custom headers

9. Sessions

Tina4 supports server-side sessions for storing per-user state between requests. JWTs handle API authentication. Sessions handle stateful web pages. Both work side by side.

Session Configuration

Set the session backend in `.env`:

```
# File-based sessions (default)
TINA4_SESSION_HANDLER=file

# Redis
TINA4_SESSION_HANDLER=redis
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379

# MongoDB
TINA4_SESSION_HANDLER=mongodb
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=27017

# Valkey
TINA4_SESSION_HANDLER=valkey
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
```

File-based sessions work out of the box. No extra dependencies. For production deployments with multiple servers, use Redis or Valkey so sessions are shared across instances.

Using Sessions

Access session data via `request.session`:

```
from tina4_python.core.router import get, post

@post("/login-form")
async def login_form(request, response):
    # After validating credentials...
    request.session["user_id"] = 42
    request.session["user_name"] = "Alice"
    request.session["logged_in"] = True

    return response.redirect("/dashboard")

@get("/dashboard")
async def dashboard(request, response):
    if not request.session.get("logged_in"):
        return response.redirect("/login")

    return response.render("dashboard.html", {
        "user_name": request.session["user_name"]
    })

@post("/logout")
async def logout(request, response):
    # Clear all session data
    request.session.clear()

// ...
```

Session Options

```
TINA4_SESSION_LIFETIME=3600       # Session lifetime in seconds (default: 3600)
TINA4_SESSION_NAME=tina4_session  # Cookie name for the session ID
```

10. Exercise: Build Login, Register, and Profile

Build a complete authentication system with registration, login, profile viewing, and password changing.

Requirements

1. Create a `users` table migration with: `id`, `name`, `email` (unique), `password_hash`, `role` (default "user"), `created_at`

2. Build these endpoints:

| Method | Path | Auth | Description |

|--------|------|------|-------------|

| `POST` | `/api/register` | @noauth | Create an account. Validate name, email, password (min 8 chars). |

| `POST` | `/api/login` | @noauth | Login. Return JWT token. |

| `GET` | `/api/profile` | secured | Get current user's profile from token. |

| `PUT` | `/api/profile` | secured | Update name and email. |

| `PUT` | `/api/profile/password` | secured | Change password. Require current password. |

3. Create auth middleware that extracts the user from the JWT and attaches it to `request.user`.

Test with:

```
# Register
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'

# Login
curl -X POST http://localhost:7145/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'

# Save the token from login response, then:

# Get profile
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE"

# Update profile
curl -X PUT http://localhost:7145/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Smith"}'

# Change password
curl -X PUT http://localhost:7145/api/profile/password \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
// ...
```

11. Solution

Migration

Create `src/migrations/20260322160000_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

```
tina4 migrate
```

Routes

Create `src/routes/auth.py`:

```
from tina4_python.core.router import get, post, put, noauth, middleware
from tina4_python.auth import Auth
from tina4_python.database.connection import Database
```

```
async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")

    if not auth_header or not auth_header.startswith("Bearer "):
        return response.json({"error": "Authorization required. Send: Authorization: Bearer <token>"}, 401)

    token = auth_header[7:]

    if not Auth.valid_token(token):
        return response.json({"error": "Invalid or expired token. Please login again."}, 401)

    payload = Auth.get_payload(token)
    request.user = payload

    return await next_handler(request, response)


@post("/api/register")
@noauth
async def register(request, response):
// ...
```

Expected output for register:

```
{
  "message": "Registration successful",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "role": "user",
    "created_at": "2026-03-22 16:00:00"
  }
}
```

(Status: `201 Created`)

Expected output for profile without token:

```
{"error":"Authorization required. Send: Authorization: Bearer <token>"}
```

(Status: `401 Unauthorized`)

12. Gotchas

1. Token expiry confusion

Problem: Tokens that worked yesterday now return 401.

Cause: The default token lifetime is 1 hour. After that, the token is invalid even if the signature is correct.

Fix: Issue a new token at login. If your application needs long-lived sessions, use refresh tokens: a short-lived access token (15 minutes) paired with a long-lived refresh token (7 days) that can be used to get a new access token without re-entering credentials.

2. Secret key management

Problem: Tokens generated on one server are invalid on another, or tokens stop working after a deployment.

Cause: Each server generated its own random `secrets/jwt.key` file. Or the key file was deleted/regenerated during deployment.

Fix: Set `TINA4_JWT_SECRET` in `.env` explicitly and use the same value across all servers. Store it in your deployment secrets manager (not in version control). If the key changes, all existing tokens become invalid and users must log in again.

3. CORS with authentication

Problem: Frontend requests with the `Authorization` header fail with a CORS error, even though `CORS_ORIGINS=*` is set.

Cause: When the browser sends an `Authorization` header, it first sends a preflight `OPTIONS` request. The server must respond to the OPTIONS request with the correct CORS headers, including `Access-Control-Allow-Headers: Authorization`.

Fix: Tina4 handles preflight requests automatically. Make sure `CORS_ORIGINS` is set correctly. If it is still failing, check that you are not overriding CORS headers in middleware.

4. Storing tokens in localStorage

Problem: Your token is stolen via an XSS attack because it was stored in `localStorage`.

Cause: Any JavaScript on the page can read `localStorage`, including injected scripts from an XSS vulnerability.

Fix: Store tokens in `httpOnly` cookies when possible -- they cannot be accessed by JavaScript. For SPAs that must use `localStorage`, implement strict Content Security Policy headers and sanitize all user input.

5. Forgetting @noauth on login

Problem: Your login endpoint returns 401 -- you cannot log in because the endpoint requires authentication.

Cause: If you have global auth middleware, the login endpoint needs the `@noauth` decorator to bypass it.

Fix: Add `@noauth` to your login and register routes. Without it, users cannot authenticate because authentication is required to authenticate -- a catch-22.

6. Password hash column too short

Problem: Registration fails with a database error about the password hash being too long.

Cause: Bcrypt hashes are 60 characters long. If your `password_hash` column is defined as `VARCHAR(50)`, it gets truncated.

Fix: Use `TEXT` for the password hash column, or at minimum `VARCHAR(255)`. Never constrain the hash length.

7. Token in URL query parameters

Problem: Tokens in URLs like `/api/profile?token=eyJ...` leak through browser history, server logs, and the Referer header.

Cause: Query parameters are visible in many places where headers are not.

Fix: Always send tokens in the `Authorization` header, never in the URL. The only exception is WebSocket connections, where the initial HTTP upgrade request cannot carry custom headers -- use a short-lived token for that case.

Chapter 8: Middleware

1. The Pipeline Pattern

Every HTTP request passes through a series of gates before reaching your route handler. Rate limiter. Body parser. Auth check. Logger. These gates are middleware -- code that wraps your route handler and runs before, after, or both.

Picture a public API. Every request hits a rate limit check. Some endpoints require an API key. All responses need CORS headers. Errors need logging. Without middleware, that logic lives in every handler. Duplicated. Scattered. Fragile. With middleware, you write it once and attach it where it belongs.

Tina4 Python ships with built-in middleware (CORS, rate limiting) and lets you write your own. This chapter covers both.

2. Built-In Middleware

CorsMiddleware

CORS (Cross-Origin Resource Sharing) controls which websites can call your API from a browser. Tina4 includes CORS middleware that is configured via `.env`:

```
CORS_ORIGINS=https://app.example.com,https://admin.example.com
CORS_METHODS=GET,POST,PUT,DELETE
CORS_HEADERS=Content-Type,Authorization
CORS_CREDENTIALS=true
CORS_MAX_AGE=86400
```

With these settings, only `app.example.com` and `admin.example.com` can make cross-origin requests to your API. The browser automatically handles preflight `OPTIONS` requests.

For development, you can allow all origins:

```
CORS_ORIGINS=*
```

The CORS middleware is active by default. You do not need to register it manually.

RateLimiter

The rate limiter prevents abuse by limiting how many requests a single IP can make in a time window:

```
TINA4_RATE_LIMIT=60
```

This allows 60 requests per minute per IP address. When a client exceeds the limit, they receive a `429 Too Many Requests` response with a `Retry-After` header.

Rate limit headers are included in every response:

```
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 57
X-RateLimit-Reset: 1711113000
```

Like CORS, the rate limiter is active by default based on your `.env` configuration.

3. Writing Custom Middleware

A middleware function takes three arguments: `request`, `response`, and `next_handler`. It must return a response.

```
async def my_middleware(request, response, next_handler):
    # Code that runs BEFORE the route handler
    print("Before handler")

    # Call the next middleware or the route handler
    result = await next_handler(request, response)

    # Code that runs AFTER the route handler
    print("After handler")

    return result
```

Example: Request Timer

```
import time

async def timer_middleware(request, response, next_handler):
    start = time.time()

    result = await next_handler(request, response)

    duration_ms = round((time.time() - start) * 1000, 2)
    print(f"{request.method} {request.path} completed in {duration_ms}ms")

    return result
```

Example: Request Logger

```
from datetime import datetime

async def log_middleware(request, response, next_handler):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"[{timestamp}] {request.method} {request.path}")
    print(f"  Headers: {dict(request.headers)}")

    if request.body:
        print(f"  Body: {request.body}")

    result = await next_handler(request, response)

    print(f"  Status: {result.status_code if hasattr(result, 'status_code') else 'unknown'}")

    return result
```

Example: JSON Content-Type Enforcer

```
async def require_json(request, response, next_handler):
    if request.method in ("POST", "PUT", "PATCH"):
        content_type = request.headers.get("Content-Type", "")
        if "application/json" not in content_type:
            return response.json({
                "error": "Content-Type must be application/json"
            }, 415)

    return await next_handler(request, response)
```

4. The @middleware Decorator

Apply middleware to a single route with the `@middleware` decorator:

```
from tina4_python.core.router import get, post, middleware

@get("/api/data")
@middleware(timer_middleware)
async def get_data(request, response):
    return response.json({"data": [1, 2, 3]})
```

Apply multiple middleware by passing them as separate arguments:

```
@post("/api/items")
@middleware(log_middleware, require_json, timer_middleware)
async def create_item(request, response):
    return response.json({"item": request.body}, 201)
```

Middleware runs in the order you list them. In the example above: `log_middleware` runs first, then `require_json`, then `timer_middleware`, then the route handler.

5. Middleware on Route Groups

Apply middleware to all routes in a group:

```
from tina4_python.core.router import get, post, group, middleware

@group("/api/v1")
@middleware(log_middleware, timer_middleware)
def api_v1():

    @get("/users")
    async def list_users(request, response):
        return response.json({"users": []})

    @post("/users")
    async def create_user(request, response):
        return response.json({"created": True}, 201)

    @get("/products")
    async def list_products(request, response):
        return response.json({"products": []})
```

Every route inside the group now has `log_middleware` and `timer_middleware` applied. You can still add route-specific middleware on top:

```
@group("/api/v1")
@middleware(log_middleware)
def api_v1():

    @get("/public")
    async def public_endpoint(request, response):
        # Only log_middleware runs
        return response.json({"public": True})

    @post("/admin")
    @middleware(auth_middleware)
    async def admin_endpoint(request, response):
        # log_middleware + auth_middleware both run
        return response.json({"admin": True})
```

6. Execution Order

Stacked middleware forms a nested pipeline. Requests travel inward. Responses travel outward:

```
Request arrives
  → log_middleware (before)
    → auth_middleware (before)
      → timer_middleware (before)
        → route handler
      → timer_middleware (after)
    → auth_middleware (after)
  → log_middleware (after)
Response sent
```

Each middleware wraps around the next one. The outermost middleware runs first on the way in and last on the way out.

Here is a concrete example showing the order:

```
async def middleware_a(request, response, next_handler):
    print("A: before")
    result = await next_handler(request, response)
    print("A: after")
    return result

async def middleware_b(request, response, next_handler):
    print("B: before")
    result = await next_handler(request, response)
    print("B: after")
    return result

@get("/test")
@middleware(middleware_a, middleware_b)
async def test(request, response):
    print("Handler")
    return response.json({"ok": True})
```

When you request `/test`, the console shows:

```
A: before
B: before
Handler
B: after
A: after
```

7. Short-Circuiting

Skip `next_handler` and the chain stops cold. The route handler never runs. This is how blocking middleware works:

```
async def maintenance_mode(request, response, next_handler):
    import os
    if os.getenv("MAINTENANCE_MODE") == "true":
        return response.json({
            "error": "Service is under maintenance. Please try again later."
        }, 503)

    return await next_handler(request, response)
```

When `MAINTENANCE_MODE=true`, every request gets a 503 response without reaching any route handler.

Conditional Short-Circuit

```
async def require_api_key(request, response, next_handler):
    api_key = request.headers.get("X-API-Key", "")

    if not api_key:
        return response.json({"error": "API key required"}, 401)

    # Validate the key against a database or config
    valid_keys = ["key-abc-123", "key-def-456", "key-ghi-789"]
    if api_key not in valid_keys:
        return response.json({"error": "Invalid API key"}, 403)

    # Attach the key info to the request for the handler to use
    request.api_key = api_key

    return await next_handler(request, response)
```

```
# No key -- 401
curl http://localhost:7145/api/data
```

```
{"error":"API key required"}
```

```
# Invalid key -- 403
curl http://localhost:7145/api/data -H "X-API-Key: wrong-key"
```

```
{"error":"Invalid API key"}
```

```
# Valid key -- 200
curl http://localhost:7145/api/data -H "X-API-Key: key-abc-123"
```

```
{"data":[1,2,3]}
```

8. Modifying Request and Response

Middleware can modify the request before it reaches the handler, and the response before it reaches the client.

Adding Data to the Request

```
async def inject_user_agent(request, response, next_handler):
    ua = request.headers.get("User-Agent", "")

    request.is_mobile = "Mobile" in ua or "Android" in ua or "iPhone" in ua
    request.is_bot = "bot" in ua.lower() or "spider" in ua.lower()

    return await next_handler(request, response)
```

Now the route handler can access `request.is_mobile` and `request.is_bot`.

Modifying the Response

```
async def add_security_headers(request, response, next_handler):
    result = await next_handler(request, response)

    # Add security headers to every response
    return response.header("X-Content-Type-Options", "nosniff") \
                   .header("X-Frame-Options", "DENY") \
                   .header("X-XSS-Protection", "1; mode=block") \
                   .header("Strict-Transport-Security", "max-age=31536000; includeSubDomains")
```

9. Real-World Example: API Key Middleware with Database Lookup

```
from tina4_python.database.connection import Database
from datetime import datetime

async def api_key_middleware(request, response, next_handler):
    api_key = request.headers.get("X-API-Key", "")

    if not api_key:
        return response.json({
            "error": "API key required. Send it in the X-API-Key header."
        }, 401)

    db = Database()
    key_record = db.fetch_one(
        "SELECT id, name, rate_limit, is_active FROM api_keys WHERE key_value = :key",
        {"key": api_key}
    )

    if key_record is None:
        return response.json({"error": "Invalid API key"}, 403)

    if not key_record["is_active"]:
        return response.json({"error": "API key has been deactivated"}, 403)

    # Update last used timestamp
    db.execute(
// ...
```

10. Exercise: Build an API Key Middleware System

Build a complete API key system with key management and usage tracking.

Requirements

1. Create a migration for an `api_keys` table: `id`, `name`, `key_value` (unique), `is_active` (default true), `rate_limit` (default 100), `request_count` (default 0), `last_used_at`, `created_at`

2. Build these endpoints:

| Method | Path | Middleware | Description |

|--------|------|-----------|-------------|

| `POST` | `/admin/api-keys` | Auth | Create a new API key (generate random key) |

| `GET` | `/admin/api-keys` | Auth | List all API keys with usage stats |

| `DELETE` | `/admin/api-keys/{id:int}` | Auth | Deactivate an API key |

| `GET` | `/api/data` | API Key | Protected endpoint -- requires valid API key |

| `GET` | `/api/status` | API Key | Another protected endpoint |

3. The API key middleware should:

• Check `X-API-Key` header

• Validate against the database

- Reject deactivated keys
- Track usage (increment count, update last_used_at)
- Attach key info to the request

Test with:

```
# Create an API key (requires auth token from Chapter 7)
curl -X POST http://localhost:7145/admin/api-keys \
  -H "Authorization: Bearer YOUR_AUTH_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name": "Mobile App"}'

# Use the API key
curl http://localhost:7145/api/data \
  -H "X-API-Key: THE_GENERATED_KEY"

# List keys with stats
curl http://localhost:7145/admin/api-keys \
  -H "Authorization: Bearer YOUR_AUTH_TOKEN"
```

11. Solution

Migration

Create `src/migrations/20260322170000_create_api_keys_table.sql`:

```
-- UP
CREATE TABLE api_keys (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    key_value TEXT NOT NULL UNIQUE,
    is_active INTEGER NOT NULL DEFAULT 1,
    rate_limit INTEGER NOT NULL DEFAULT 100,
    request_count INTEGER NOT NULL DEFAULT 0,
    last_used_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX idx_api_keys_key ON api_keys (key_value);

-- DOWN
DROP INDEX IF EXISTS idx_api_keys_key;
DROP TABLE IF EXISTS api_keys;
```

Routes

Create `src/routes/api_keys.py`:

```
from tina4_python.core.router import get, post, delete as delete_route, middleware
from tina4_python.auth import Auth
from tina4_python.database.connection import Database
from datetime import datetime
import secrets


async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")
    if not auth_header or not auth_header.startswith("Bearer "):
        return response.json({"error": "Authorization required"}, 401)
```

```
    token = auth_header[7:]
    if not Auth.valid_token(token):
        return response.json({"error": "Invalid or expired token"}, 401)

    request.user = Auth.get_payload(token)
    return await next_handler(request, response)


async def api_key_middleware(request, response, next_handler):
    api_key = request.headers.get("X-API-Key", "")

    if not api_key:
        return response.json({"error": "API key required. Send in X-API-Key header."}, 401)
// ...
```

12. Gotchas

1. Forgetting to await next_handler

Problem: The route handler never runs, or you get an error about a coroutine object.

Cause: You called `next_handler(request, response)` without `await`.

Fix: Always use `await next_handler(request, response)`. Since Tina4 Python is async, every middleware and handler must be awaited.

2. Middleware modifies response after it is sent

Problem: Headers or cookies you add in the "after" phase of middleware do not appear in the response.

Cause: The response was already finalized by the route handler.

Fix: In the "after" phase, modify the result returned by `next_handler`, not the original `response` object. Some modifications may need to happen in the "before" phase instead.

3. Middleware applied to wrong routes

Problem: Your API key middleware runs on public routes that should not require a key.

Cause: The middleware is applied to the group, and the public route is inside that group.

Fix: Move public routes outside the group, or use `@noauth` on specific routes to bypass authentication middleware. For custom middleware like API key checks, you need to handle exemptions manually by checking the path in the middleware.

4. Middleware execution order surprises

Problem: Your auth check runs after your logging middleware, but you wanted it to run first.

Cause: Middleware in `@middleware(a, b, c)` runs in left-to-right order: `a` wraps `b` wraps `c` wraps handler.

Fix: Put the middleware you want to run first at the leftmost position: `@middleware(auth_middleware, log_middleware)`.

5. Error in middleware breaks the chain

Problem: An unhandled exception in middleware causes a 500 error without reaching the route handler.

Cause: If middleware throws an exception before calling `next_handler`, no subsequent middleware or the handler runs.

Fix: Wrap risky middleware code in try/except and return an appropriate error response instead of letting the exception propagate.

6. Database connections in middleware

Problem: Opening a database connection in middleware that runs on every request causes connection pool exhaustion.

Cause: Each middleware call creates a new `Database()` instance.

Fix: Tina4's `Database()` uses connection pooling internally, so this is usually safe. But if you are seeing issues, cache your database lookups (like API keys) in memory with a TTL instead of querying on every request.

7. Modifying request in middleware does not persist

Problem: You set `request.custom_field = "value"` in middleware, but the route handler does not see it.

Cause: In some edge cases, the request object may be copied between middleware stages.

Fix: Use `request.custom_field` consistently. If it is not persisting, check that you are modifying the same request object that is passed to `next_handler`. Do not create a new request object.

Chapter 9: Sessions and Cookies

1. Remembering Your Users

JWT tokens handle APIs. Traditional web applications need more. A shopping cart that persists across pages. A flash message that appears once after a redirect. A "remember me" checkbox on the login form. These features run on sessions and cookies -- server-side state tied to a browser.

Picture an e-commerce site. A customer adds three items to their cart. Navigates to a product page. Comes back to the cart. Without sessions, the cart is empty every time. Sessions give the server memory. Each browser gets its own state, preserved across requests.

2. Session Configuration

Set the session backend in `.env`:

```
TINA4_SESSION_HANDLER=file
```

That is the default. File-based sessions work out of the box with no additional dependencies.

Available Backends

| Backend | `.env` Value | Package Required | Best For |

|---------|-------------|-----------------|---------|

| File | `file` | None | Development, single server |

| Redis | `redis` | `redis` | Production, multi-server |

| MongoDB | `mongodb` | `pymongo` | Production, document stores |

| Valkey | `valkey` | `valkey` | Production, Redis alternative |

| Database | `database` | None | Production, using existing DB |

Redis Configuration

```
TINA4_SESSION_HANDLER=redis
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
TINA4_SESSION_PASSWORD=
```

Install the Redis driver:

```
uv add redis
```

MongoDB Configuration

```
TINA4_SESSION_HANDLER=mongodb
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=27017
TINA4_SESSION_DATABASE=tina4_sessions
```

Valkey Configuration

```
TINA4_SESSION_HANDLER=valkey
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
```

Session Lifetime

```
TINA4_SESSION_LIFETIME=3600       # 1 hour (default)
TINA4_SESSION_NAME=tina4_session  # Cookie name for session ID
```

3. Reading and Writing Session Data

Access session data through `request.session`. It behaves like a Python dictionary:

Writing to the Session

```
from tina4_python.core.router import post

@post("/login-form")
async def login_form(request, response):
    # After validating credentials...
    request.session["user_id"] = 42
    request.session["user_name"] = "Alice"
    request.session["role"] = "admin"
    request.session["logged_in"] = True

    return response.redirect("/dashboard")
```

Reading from the Session

```
from tina4_python.core.router import get

@get("/dashboard")
async def dashboard(request, response):
    if not request.session.get("logged_in"):
        return response.redirect("/login")

    return response.render("dashboard.html", {
        "user_name": request.session["user_name"],
        "role": request.session["role"]
    })
```

Deleting Session Data

```
# Delete a single key
del request.session["temp_data"]

# Or use pop with a default
request.session.pop("temp_data", None)

# Clear all session data (logout)
@post("/logout")
async def logout(request, response):
    request.session.clear()
    return response.redirect("/login")
```

Checking if a Key Exists

```
if "user_id" in request.session:
    user_id = request.session["user_id"]
else:
    user_id = None

# Or use .get() with a default
user_id = request.session.get("user_id", None)
```

4. Flash Messages

Flash messages are session values that are automatically deleted after they are read once. They are perfect for showing success or error messages after a redirect.

Setting a Flash Message

```
@post("/api/contact")
async def submit_contact(request, response):
    # Process the form...

    request.session["flash_message"] = "Your message has been sent!"
    request.session["flash_type"] = "success"

    return response.redirect("/contact")
```

Reading and Clearing a Flash Message

```
@get("/contact")
async def contact_page(request, response):
    flash_message = request.session.pop("flash_message", None)
    flash_type = request.session.pop("flash_type", "info")

    return response.render("contact.html", {
        "flash_message": flash_message,
        "flash_type": flash_type
    })
```

The `pop()` method reads the value and deletes it from the session in one step. The next time the user visits `/contact`, the flash message will be gone.

Flash Messages in Templates

```
{% if flash_message %}
    <div class="alert alert-{{ flash_type }}">
        {{ flash_message }}
    </div>
{% endif %}
```

5. Cookies

Cookies are small data fragments stored in the browser. Unlike sessions, cookies travel with every request and are visible to JavaScript (unless `httpOnly` is set).

Setting a Cookie

```
from tina4_python.core.router import post

@post("/preferences")
async def save_preferences(request, response):
    theme = request.body.get("theme", "light")
    language = request.body.get("language", "en")

    return response.cookie("theme", theme, {
        "max_age": 365 * 24 * 60 * 60,  # 1 year
        "path": "/",
        "samesite": "Lax"
    }).cookie("language", language, {
        "max_age": 365 * 24 * 60 * 60,
        "path": "/",
```

```
        "samesite": "Lax"
    }).json({"message": "Preferences saved"})
```

Reading a Cookie

```
from tina4_python.core.router import get

@get("/")
async def home(request, response):
    theme = request.cookies.get("theme", "light")
    language = request.cookies.get("language", "en")

    return response.render("home.html", {
        "theme": theme,
        "language": language
    })
```

Deleting a Cookie

Set `max_age` to 0:

```
@post("/clear-preferences")
async def clear_preferences(request, response):
    return response.cookie("theme", "", {"max_age": 0, "path": "/"}) \
                   .cookie("language", "", {"max_age": 0, "path": "/"}) \
                   .json({"message": "Preferences cleared"})
```

Cookie Options

| Option | Type | Description |

|--------|------|-------------|

| `max_age` | int | Lifetime in seconds. 0 = delete. Omit = session cookie (deleted when browser closes). |

| `path` | str | URL path scope. `/` means the whole site. `/admin` means only under `/admin`. |

| `domain` | str | Domain scope. `.example.com` includes subdomains. |

| `secure` | bool | Only send over HTTPS. Always `True` in production. |

| `httponly` | bool | Not accessible via JavaScript. Use for session cookies. |

| `samesite` | str | `"Strict"`, `"Lax"`, or `"None"`. Controls cross-site sending. |

When to Use Cookies vs Sessions

| Use Case | Use Cookies | Use Sessions |

|----------|------------|-------------|

| User preferences (theme, language) | Yes | No |

| Shopping cart | No | Yes |

| Authentication state | No (use JWT in header) | Yes (for form-based auth) |

| Flash messages | No | Yes |

| Tracking consent | Yes | No |

| Sensitive data (user ID, role) | No | Yes |

The rule: sensitive data or data the client should not see goes in sessions. Non-sensitive data that should persist beyond session expiry goes in cookies.

6. Remember Me

The "remember me" pattern extends the session lifetime when the user checks a box on the login form:

```
from tina4_python.core.router import post
from tina4_python.auth import Auth
from tina4_python.database.connection import Database

@post("/login-form")
async def login_form(request, response):
    body = request.body
    db = Database()

    user = db.fetch_one(
        "SELECT id, name, email, password_hash FROM users WHERE email = :email",
        {"email": body.get("email")}
    )

    if user is None or not Auth.check_password(body.get("password", ""), user["password_hash"]):
        request.session["flash_message"] = "Invalid email or password"
        request.session["flash_type"] = "error"
        return response.redirect("/login")

    # Set session data
    request.session["user_id"] = user["id"]
    request.session["user_name"] = user["name"]
    request.session["logged_in"] = True

    # Handle "remember me"
// ...
```

Then, on every page load, check for the remember token if the session has expired:

```
async def remember_me_middleware(request, response, next_handler):
    if not request.session.get("logged_in"):
        remember_token = request.cookies.get("remember_token")

        if remember_token and Auth.valid_token(remember_token):
            payload = Auth.get_payload(remember_token)
            db = Database()
            user = db.fetch_one(
                "SELECT id, name FROM users WHERE id = :id",
                {"id": payload["user_id"]}
            )

            if user:
                request.session["user_id"] = user["id"]
                request.session["user_name"] = user["name"]
                request.session["logged_in"] = True

    return await next_handler(request, response)
```

7. Session Security

Regenerate Session ID After Login

To prevent session fixation attacks, regenerate the session ID after login:

```
@post("/login-form")
```

```
async def login_form(request, response):
    # After validating credentials...

    # Regenerate session ID (prevents session fixation)
    request.session.regenerate()

    request.session["user_id"] = user["id"]
    request.session["logged_in"] = True

    return response.redirect("/dashboard")
```

Secure Cookie Settings

Always use these settings for the session cookie in production:

```
TINA4_SESSION_SECURE=true     # Only send over HTTPS
TINA4_SESSION_HTTPONLY=true   # Not accessible via JavaScript
TINA4_SESSION_SAMESITE=Lax   # Prevent CSRF via cross-site requests
```

8. Exercise: Build a Shopping Cart

Build a shopping cart using sessions.

Requirements

1. Create these endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/cart` | View cart contents (rendered HTML page) |

| `POST` | `/cart/add` | Add an item to the cart |

| `POST` | `/cart/update` | Update item quantity |

| `POST` | `/cart/remove` | Remove an item from the cart |

| `POST` | `/cart/clear` | Clear the entire cart |

| `GET` | `/cart/api` | Get cart as JSON (for AJAX) |

2. The cart is stored in `request.session["cart"]` as a list of dictionaries

3. Each cart item has: `product_id`, `name`, `price`, `quantity`

4. Adding an existing product increments its quantity

5. The cart page shows items, quantities, line totals, and a grand total

6. Use flash messages for feedback ("Item added", "Cart cleared", etc.)

Use this product data for validation:

```
PRODUCTS = {
    1: {"name": "Wireless Keyboard", "price": 79.99},
    2: {"name": "USB-C Hub", "price": 49.99},
    3: {"name": "Monitor Stand", "price": 129.99},
    4: {"name": "Mechanical Mouse", "price": 59.99},
    5: {"name": "Desk Lamp", "price": 39.99}
}
```

Test with:

```
# Add items
curl -X POST http://localhost:7145/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "quantity": 2}' \
  -c cookies.txt -b cookies.txt

curl -X POST http://localhost:7145/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 3, "quantity": 1}' \
  -c cookies.txt -b cookies.txt

# View cart
curl http://localhost:7145/cart/api -b cookies.txt

# Update quantity
curl -X POST http://localhost:7145/cart/update \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "quantity": 3}' \
  -c cookies.txt -b cookies.txt

# Remove item
curl -X POST http://localhost:7145/cart/remove \
  -H "Content-Type: application/json" \
  -d '{"product_id": 3}' \
  -c cookies.txt -b cookies.txt
// ...
```

9. Solution

Create `src/routes/cart.py`:

```
from tina4_python.core.router import get, post

PRODUCTS = {
    1: {"name": "Wireless Keyboard", "price": 79.99},
    2: {"name": "USB-C Hub", "price": 49.99},
    3: {"name": "Monitor Stand", "price": 129.99},
    4: {"name": "Mechanical Mouse", "price": 59.99},
    5: {"name": "Desk Lamp", "price": 39.99}
}


def get_cart(session):
    return session.get("cart", [])


def save_cart(session, cart):
    session["cart"] = cart


def calculate_totals(cart):
    for item in cart:
        item["line_total"] = round(item["price"] * item["quantity"], 2)
    grand_total = round(sum(item["line_total"] for item in cart), 2)
    return cart, grand_total

// ...
```

Expected output for cart API after adding items:

```
{
  "cart": [
    {"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 2, "line_total": 159.98},
    {"product_id": 3, "name": "Monitor Stand", "price": 129.99, "quantity": 1, "line_total": 129.99}
  ],
  "grand_total": 289.97,
  "item_count": 3
}
```

10. Gotchas

1. Session lost between requests

Problem: Data you stored in the session is gone on the next request.

Cause: The session cookie is not being sent back by the client. This happens when using curl without `-c cookies.txt -b cookies.txt`, or when the browser blocks cookies.

Fix: For curl testing, use `-c cookies.txt -b cookies.txt` to save and send cookies. In the browser, make sure cookies are not blocked for your site.

2. Session data is not JSON-serializable

Problem: Storing a Python object (like a datetime or a custom class) in the session causes an error.

Cause: Sessions are serialized to JSON (or pickled). Complex Python objects cannot be serialized directly.

Fix: Convert objects to strings or dictionaries before storing: `request.session["login_time"] = datetime.now().isoformat()`.

3. Cookie not sent on cross-origin requests

Problem: Your frontend at `http://localhost:3000` calls your API at `http://localhost:7145`, but cookies are not included.

Cause: Browsers do not send cookies cross-origin by default. You need both CORS configuration and explicit `credentials: "include"` in fetch.

Fix: Set `CORS_CREDENTIALS=true` in `.env` and use `fetch(url, { credentials: "include" })` in JavaScript. Also set `CORS_ORIGINS` to the specific frontend origin (not `*` -- wildcard does not work with credentials).

4. File-based sessions on multi-server deployments

Problem: A user is logged in on server A but logged out on server B.

Cause: File-based sessions are stored on the local filesystem. Each server has its own session files.

Fix: Switch to Redis, Valkey, MongoDB, or database-backed sessions so all servers share the same session store.

5. Session cookie overwritten by another app

Problem: Your session keeps getting reset even though you set it correctly.

Cause: Another application on the same domain uses the same session cookie name.

Fix: Set a unique session name: `TINA4_SESSION_NAME=myapp_session` in `.env`.

6. Secure cookies on localhost

Problem: Setting `"secure": True` on a cookie means it never gets sent during development.

Cause: Secure cookies are only sent over HTTPS. `http://localhost` is not HTTPS.

Fix: Do not set `"secure": True` during development. Use environment-based configuration: `"secure": os.getenv("TINA4_DEBUG") != "true"`.

7. Flash messages shown twice

Problem: A flash message appears on the page, but when you refresh, it shows again.

Cause: You read the flash message from the session but did not remove it. Using `request.session.get()` reads without deleting.

Fix: Use `request.session.pop("flash_message", None)` instead of `request.session.get()`. The `pop()` method reads and deletes in one step.

Chapter 10: Swagger / OpenAPI

1. Self-Documenting APIs

Routes built. Requests handled. Database connected. Authentication locked down. Now other developers need to know how to use your API. What endpoints exist. What parameters they accept. What the response looks like.

Tina4 Python auto-generates Swagger/OpenAPI documentation from your routes. Add a few decorators. Tina4 produces an interactive API reference that developers explore and test in the browser.

Picture handing your API to a frontend team. No separate document that rots the moment you change a route. You point them to `/swagger`. They see every endpoint. Examples. Parameter descriptions. A "Try it out" button on each one.

2. Accessing the Swagger UI

When `TINA4_DEBUG=true`, the Swagger UI is automatically available at:

```
http://localhost:7145/swagger
```

Open it in your browser and you will see all your registered routes, organized by tags, with request/response details.

The underlying OpenAPI JSON spec is available at:

```
http://localhost:7145/swagger/json
```

This is the raw JSON that tools like Postman, Insomnia, and code generators can import.

3. Documenting Routes with Decorators

@description -- Describe What an Endpoint Does

```
from tina4_python.core.router import get, post
from tina4_python.swagger import description

@get("/api/users")
@description("List all users", "Returns a paginated list of all registered users. Supports filtering by role a
async def list_users(request, response):
    return response.json({"users": [], "count": 0})
```

The first argument is a short summary (shown in the route list). The second is a detailed description (shown when you expand the route).

@tags -- Organize Endpoints into Groups

```
from tina4_python.swagger import tags

@get("/api/users")
@tags("Users")
@description("List all users")
async def list_users(request, response):
    return response.json({"users": []})

@post("/api/users")
```

```
@tags("Users")
@description("Create a new user")
async def create_user(request, response):
    return response.json({"user": request.body}, 201)

@get("/api/products")
@tags("Products")
@description("List all products")
async def list_products(request, response):
    return response.json({"products": []})
```

Tags group related endpoints together in the Swagger UI. All "Users" endpoints appear under one collapsible section, all "Products" under another.

@example -- Document Request Body

```
from tina4_python.swagger import example

@post("/api/users")
@tags("Users")
@description("Create a new user", "Registers a new user account. Email must be unique.")
@example({
    "name": "Alice Smith",
    "email": "alice@example.com",
    "password": "securePass123",
    "role": "user"
})
async def create_user(request, response):
    return response.json({"user": request.body}, 201)
```

The `@example` decorator shows a sample request body in the Swagger UI. Developers can click "Try it out" and the example is pre-filled.

@example_response -- Document Response Body

```
from tina4_python.swagger import example_response

@get("/api/users/{id:int}")
@tags("Users")
@description("Get a user by ID")
@example_response(200, {
    "id": 1,
    "name": "Alice Smith",
    "email": "alice@example.com",
    "role": "user",
    "created_at": "2026-03-22T14:30:00"
})
@example_response(404, {
    "error": "User not found"
})
async def get_user(request, response):
    user_id = request.params["id"]
    return response.json({"id": user_id, "name": "Alice"})
```

You can add multiple `@example_response` decorators for different status codes. This shows developers exactly what to expect for success and error cases.

4. Documenting Path Parameters

Path parameters are automatically detected from the route pattern. You can add descriptions with the `@description` decorator's extended syntax:

```
@get("/api/users/{id:int}/posts/{status}")
@tags("Posts")
@description(
    "Get user posts by status",
    "Returns all posts for a specific user filtered by status.",
    params={
        "id": "The user's unique identifier (integer)",
        "status": "Post status filter: 'draft', 'published', or 'archived'"
    }
)
async def user_posts(request, response):
    return response.json({"posts": []})
```

Documenting Query Parameters

```
@get("/api/products")
@tags("Products")
@description(
    "List products",
    "Returns a filtered and paginated list of products.",
    query={
        "category": {"type": "string", "description": "Filter by category name", "required": False},
        "min_price": {"type": "number", "description": "Minimum price filter", "required": False},
        "max_price": {"type": "number", "description": "Maximum price filter", "required": False},
        "page": {"type": "integer", "description": "Page number (default: 1)", "required": False},
        "limit": {"type": "integer", "description": "Items per page (default: 20)", "required": False}
    }
)
async def list_products(request, response):
    return response.json({"products": [], "count": 0})
```

5. Authentication in Swagger

When your API uses JWT authentication, document it so the Swagger UI includes an "Authorize" button:

```
from tina4_python.core.router import get, secured
from tina4_python.swagger import description, tags

@get("/api/profile")
@secured
@tags("Auth")
@description("Get current user profile", "Requires a valid JWT token in the Authorization header.")
async def get_profile(request, response):
    return response.json({"user": request.user})
```

Routes marked with `@secured` automatically show a lock icon in the Swagger UI. Developers can click "Authorize", paste their JWT token, and all subsequent requests will include it.

Public routes marked with `@noauth` show as unlocked:

```
from tina4_python.core.router import post, noauth

@post("/api/login")
@noauth
@tags("Auth")
@description("Login", "Authenticate with email and password. Returns a JWT token.")
@example({"email": "alice@example.com", "password": "securePass123"})
@example_response(200, {
```

```
    "message": "Login successful",
    "token": "eyJhbGciOiJIUzI1NiIs...",
    "user": {"id": 1, "name": "Alice", "email": "alice@example.com"}
})
@example_response(401, {"error": "Invalid email or password"})
async def login(request, response):
    return response.json({"token": "..."})
```

6. Complete API Documentation Example

Here is a fully documented User API:

```
from tina4_python.core.router import get, post, put, delete, noauth, secured, middleware
from tina4_python.swagger import description, tags, example, example_response
from tina4_python.auth import Auth
from tina4_python.database.connection import Database


async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")
    if not auth_header or not auth_header.startswith("Bearer "):
        return response.json({"error": "Authorization required"}, 401)
    token = auth_header[7:]
    if not Auth.valid_token(token):
        return response.json({"error": "Invalid or expired token"}, 401)
    request.user = Auth.get_payload(token)
    return await next_handler(request, response)


@post("/api/users")
@noauth
@tags("Users")
@description(
    "Register a new user",
    "Creates a new user account. Email must be unique. Password must be at least 8 characters."
)
@example({
// ...
```

7. Swagger Configuration

Control Swagger behavior in `.env`:

```
# Swagger is only available when debug mode is on (default behavior)
TINA4_DEBUG=true

# Custom API title and version shown in Swagger UI
TINA4_SWAGGER_TITLE=My Store API
TINA4_SWAGGER_VERSION=1.0.0
TINA4_SWAGGER_DESCRIPTION=REST API for the My Store e-commerce platform
```

8. Exercise: Document a User API

Take the authentication routes from Chapter 7 (register, login, profile, update profile, change password) and add full Swagger documentation.

Requirements

1. Add `@tags("Auth")` to all auth-related endpoints
2. Add `@description()` with both summary and detailed description
3. Add `@example()` for all POST and PUT endpoints
4. Add `@example_response()` for success and error cases on every endpoint
5. Document query parameters on any endpoint that accepts them
6. Visit `/swagger` and verify all routes appear correctly with examples

Expected Result

When you open `http://localhost:7145/swagger`, you should see:

• An "Auth" section with 5 endpoints

• Each endpoint has a summary and description

• POST/PUT endpoints show example request bodies

• Every endpoint shows example responses for success and error cases

• The "Authorize" button works for testing protected endpoints

9. Solution

Update `src/routes/auth.py` with Swagger decorators (showing just the decorator additions -- the function bodies remain the same as Chapter 7):

```
from tina4_python.core.router import get, post, put, noauth, middleware
from tina4_python.swagger import description, tags, example, example_response
from tina4_python.auth import Auth
from tina4_python.database.connection import Database


async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")
    if not auth_header or not auth_header.startswith("Bearer "):
        return response.json({"error": "Authorization required"}, 401)
    token = auth_header[7:]
    if not Auth.valid_token(token):
        return response.json({"error": "Invalid or expired token"}, 401)
    request.user = Auth.get_payload(token)
    return await next_handler(request, response)


@post("/api/register")
@noauth
@tags("Auth")
@description(
    "Register a new account",
    "Creates a new user account. All fields are required. Password must be at least 8 characters. Email must b
)
@example({
// ...
```

10. Gotchas

1. Swagger not showing up

Problem: Visiting `/swagger` returns a 404.

Cause: `TINA4_DEBUG` is not set to `true` in your `.env`. The Swagger UI is only available in debug mode.

Fix: Set `TINA4_DEBUG=true` in `.env` and restart the server.

2. Route appears but has no documentation

Problem: A route shows up in Swagger but has no description, examples, or parameter documentation.

Cause: You did not add Swagger decorators (`@description`, `@example`, etc.) to the route.

Fix: Add at least `@description()` and `@tags()` to every route you want documented. Without these, Swagger shows the route but with minimal information.

3. Decorator order causes issues

Problem: Adding `@description` or `@tags` breaks the route registration.

Cause: Swagger decorators must be placed above the route decorator (applied after it in Python's bottom-up decorator application).

Fix: Follow this order (bottom to top): route decorator first, then middleware, then Swagger decorators:

```
@tags("Users")                # Applied last
@description("List users")    # Applied third
@middleware(auth_middleware)   # Applied second
@get("/api/users")            # Applied first
async def list_users(...):
    ...
```

4. Example does not match actual response

Problem: The example response in Swagger does not match what the endpoint actually returns.

Cause: You updated the route handler but forgot to update the `@example_response` decorator.

Fix: Keep examples in sync with your actual response format. This is the main downside of Swagger decorators -- they are not enforced. Consider writing tests that validate your responses match the documented examples.

5. Sensitive data in examples

Problem: Your `@example` decorator includes a real password or API key.

Cause: You copy-pasted from a test and forgot to replace real values.

Fix: Always use placeholder values in examples: `"password": "securePass123"`, `"token": "eyJhbGciOiJIUzI1NiIs..."`. Never include real credentials.

6. Swagger in production

Problem: Your production API exposes the Swagger UI, revealing all endpoints and their parameters to the public.

Cause: `TINA4_DEBUG=true` in production.

Fix: Always set `TINA4_DEBUG=false` in production. The Swagger UI will not be available. If you need API docs in production, export the OpenAPI JSON and host it separately with access controls.

7. Too many tags

Problem: Your Swagger UI has 20 tags, making it hard to navigate.

Cause: You created a separate tag for every resource, sub-resource, and action.

Fix: Use broad tags that group related functionality: "Users", "Products", "Orders", "Auth". Most APIs need 5-10 tags. Avoid tags like "User Profile", "User Settings", "User Notifications" -- just use "Users" for all of them.

Chapter 11: Queues

1. Not Everything Should Happen Right Now

Some tasks are too slow for an HTTP request. Sending an email: 2 seconds. Generating a PDF report: 10 seconds. Processing a large CSV upload: a minute. Run these inside a route handler and the user stares at a spinner. Or the request times out.

Queues solve this. Push a message describing the work. A separate worker picks it up and does the job in the background. The user gets an instant response: "Your report is being generated."

Picture a store that sends order confirmations, generates invoices, and syncs inventory with a warehouse. None of these should block checkout. Each one becomes a queue message. A worker processes it on its own schedule.

2. Queue Configuration

Tina4 Python includes a built-in database-backed queue that requires zero additional setup. It uses your existing database connection to store queue messages.

Default (Database Queue)

The database queue works out of the box with no extra configuration:

```
# No TINA4_QUEUE_BACKEND needed -- database is the default
DATABASE_URL=sqlite:///data/app.db
```

The first time you push a message, Tina4 automatically creates the `tina4_queue` table.

Switching to RabbitMQ

When your application outgrows the database queue (millions of messages, multiple services, pub/sub patterns), switch to RabbitMQ by changing the config:

```
TINA4_QUEUE_BACKEND=rabbitmq
RABBITMQ_URL=amqp://user:pass@localhost:5672
```

Install the client library:

```
uv add pika
```

Switching to Kafka

For stream processing, event sourcing, or very high throughput:

```
TINA4_QUEUE_BACKEND=kafka
KAFKA_BROKERS=localhost:9092
KAFKA_GROUP_ID=tina4-workers
```

Install the client library:

```
uv add confluent-kafka
```

The key point: your code stays the same. `Queue`, `push`, `pop`, and `worker` work identically whether the backend is SQLite, PostgreSQL, RabbitMQ, or Kafka.

3. Pushing Messages

```
from tina4_python.queue import Queue

queue = Queue("emails")

# Push a message
message_id = queue.push("emails", {
    "to": "alice@example.com",
    "subject": "Order Confirmation",
    "body": "Your order #1234 has been confirmed."
})
```

The first argument is the queue name. The second is the payload -- any dictionary that can be serialized to JSON.

Priority

Higher priority messages are processed first:

```
# Normal priority (default: 0)
queue.push("emails", {"to": "user@example.com", "subject": "Newsletter"})

# High priority
queue.push("emails", {"to": "user@example.com", "subject": "Password Reset"}, priority=10)
```

Delayed Messages

Schedule a message to be processed later:

```
# Process in 5 minutes
queue.push("reminders", {
    "user_id": 42,
    "message": "Your trial expires tomorrow"
}, delay=300)

# Process at a specific time
queue.push("reports", {
    "report_type": "monthly",
    "month": "2026-03"
}, available_at="2026-04-01T00:00:00Z")
```

Max Attempts

Control how many times a failed message is retried:

```
queue.push("emails", payload, max_attempts=5)  # Default is 3
```

4. Consuming Messages

Simple Worker

```
from tina4_python.queue import Queue

queue = Queue("emails")

@queue.worker(concurrency=1, poll_interval=1)
def process_email(message):
    payload = message.payload
    send_email(payload["to"], payload["subject"], payload["body"])
    # Returning without error = auto-complete
```

```
    # Raising an exception = auto-fail with retry
```

The worker polls the queue every `poll_interval` seconds, pops a message, and calls your function. If the function returns normally, the message is marked as completed. If it raises an exception, the message is marked as failed and retried (up to `max_attempts`).

Manual Pop

For more control, pop messages manually:

```
message = queue.pop("emails")

if message is not None:
    try:
        send_email(message.payload["to"], message.payload["subject"])
        queue.complete(message)
    except Exception as e:
        queue.fail(message, str(e))
```

Batch Processing

Pop multiple messages at once:

```
messages = queue.pop_batch("emails", count=10)

for msg in messages:
    try:
        process(msg.payload)
        queue.complete(msg)
    except Exception as e:
        queue.fail(msg, str(e))
```

5. Message Lifecycle

A message moves through these statuses:

```
push() → PENDING → pop() → RESERVED → complete() → COMPLETED
                                    → fail() → FAILED
                                                ↓
                                          retry (auto)
                                                ↓
                                          PENDING (retry)
                                                ↓
                                     max attempts exceeded
                                                ↓
                                            DEAD_LETTER
```

Status Definitions

| Status | Description |

|--------|----------|

| `pending` | Waiting to be processed |

| `reserved` | Claimed by a worker, being processed |

| `completed` | Successfully processed |

| `failed` | Processing failed, will be retried |

| `dead_letter` | Failed too many times, requires manual intervention |

Inspecting Messages

```
# Get a specific message by ID
message = queue.get(message_id)
print(message.status)      # "pending", "reserved", "completed", etc.
print(message.attempts)    # How many times it has been tried
print(message.error)       # Last error message if failed

# Search messages
pending = queue.find("emails", status="pending")
failed = queue.find("emails", status="failed", limit=50)
```

6. Retry and Dead Letters

Automatic Retry with Exponential Backoff

When a message fails, Tina4 retries it with exponential backoff:

```
Attempt 1 fails → retry in 30 seconds
Attempt 2 fails → retry in 120 seconds
Attempt 3 fails → moved to dead letter queue
```

The backoff formula is: `delay = base_delay * (2 ^ (attempt - 1))`

Default base delay is 30 seconds.

Dead Letter Queue

Messages that exceed `max_attempts` are moved to dead letter status. They are not retried automatically -- you must handle them manually.

```
# List dead letter messages
dead_messages = queue.dead_letter("emails")

for msg in dead_messages:
    print(f"Failed message: {msg.id}")
    print(f"  Payload: {msg.payload}")
    print(f"  Error: {msg.error}")
    print(f"  Attempts: {msg.attempts}")
```

Manual Retry

You can manually retry a dead letter message:

```
# Retry a single message (resets attempts, moves back to pending)
queue.retry(message)
```

Purging Dead Letters

```
queue.purge("emails")  # Remove all messages from the queue
```

7. Queue in Route Handlers

The most common pattern is pushing messages from route handlers:

```
from tina4_python.core.router import post
from tina4_python.queue import Queue

@post("/api/orders")
async def create_order(request, response):
```

```
    body = request.body
    db = Database()

    # Create the order in the database
    db.execute(
        "INSERT INTO orders (user_id, total, status) VALUES (:user_id, :total, :status)",
        {"user_id": body["user_id"], "total": body["total"], "status": "confirmed"}
    )

    order = db.fetch_one("SELECT * FROM orders WHERE id = last_insert_rowid()")

    # Queue background tasks
    queue = Queue()

    # Send confirmation email
    queue.push("emails", {
        "type": "order_confirmation",
        "to": body["email"],
        "order_id": order["id"],
        "total": order["total"]
// ...
```

The user gets an instant response. The email, invoice, and warehouse sync happen in the background.

8. Switching Backends via .env

Switching backends is a config change, not a code change.

Development: SQLite

```
DATABASE_URL=sqlite:///data/app.db
# No TINA4_QUEUE_BACKEND needed -- database is default
```

Staging: PostgreSQL Database Queue

```
DATABASE_URL=postgresql://user:pass@localhost:5432/myapp
# Still using database queue, but now with PostgreSQL's FOR UPDATE SKIP LOCKED
# for safe concurrent workers
```

Production: RabbitMQ

```
TINA4_QUEUE_BACKEND=rabbitmq
RABBITMQ_URL=amqp://user:pass@rabbitmq.internal:5672
```

High-Scale Production: Kafka

```
TINA4_QUEUE_BACKEND=kafka
KAFKA_BROKERS=kafka-1:9092,kafka-2:9092,kafka-3:9092
KAFKA_GROUP_ID=tina4-workers
```

Your queue code does not change at all. The same `queue.push()` and `@queue.worker()` calls work with every backend.

Fallback Configuration

If your RabbitMQ goes down, Tina4 can fall back to the database queue automatically:

```
TINA4_QUEUE_BACKEND=rabbitmq
RABBITMQ_URL=amqp://user:pass@rabbitmq.internal:5672
QUEUE_FALLBACK_DRIVER=database
```

When RabbitMQ is unreachable, messages are stored in the database queue. When RabbitMQ comes back, processing resumes normally. Your application never stops accepting work.

9. Stale Job Recovery

If a worker crashes while processing a message, the message stays in `reserved` status forever. Tina4 automatically recovers these stale messages with a background sweep that runs every 60 seconds:

Messages stuck in `reserved` for longer than 5 minutes (configurable) are moved back to `pending` so another worker can pick them up.

```
TINA4_QUEUE_STALE_TIMEOUT=300  # 5 minutes (default)
```

10. Exercise: Build an Email Queue

Build an email queue system that sends emails in the background.

Requirements

1. Create these endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `POST` | `/api/emails/send` | Queue an email for sending |

| `GET` | `/api/emails/status/{id}` | Check the status of a queued email |

| `GET` | `/api/emails/queue` | List pending and failed emails |

| `POST` | `/api/emails/retry/{id}` | Retry a failed email |

2. The email payload should include: `to` (required), `subject` (required), `body` (required), `priority` ("low", "normal", "high")

3. Map priority strings to numeric values: low=0, normal=5, high=10

4. The status endpoint should return the message status, attempts, and error (if any)

5. Create a simulated email worker that "sends" emails by printing to the console and waiting 1 second

Test with:

```
# Queue an email
curl -X POST http://localhost:7145/api/emails/send \
  -H "Content-Type: application/json" \
  -d '{"to": "alice@example.com", "subject": "Welcome!", "body": "Thanks for signing up.", "priority": "high"

# Check status
curl http://localhost:7145/api/emails/status/MESSAGE_ID_HERE

# List queue
curl http://localhost:7145/api/emails/queue

# Retry failed
curl -X POST http://localhost:7145/api/emails/retry/MESSAGE_ID_HERE
```

11. Solution

Create `src/routes/email_queue.py`:

```
from tina4_python.core.router import get, post
from tina4_python.queue import Queue

queue = Queue()

PRIORITY_MAP = {
    "low": 0,
    "normal": 5,
    "high": 10
}


@post("/api/emails/send")
async def queue_email(request, response):
    body = request.body

    errors = []
    if not body.get("to"):
        errors.append("'to' is required")
    if not body.get("subject"):
        errors.append("'subject' is required")
    if not body.get("body"):
        errors.append("'body' is required")

    priority_name = body.get("priority", "normal")
// ...
```

Create a separate worker file `src/workers/email_worker.py`:

```
from tina4_python.queue import Queue
import time

queue = Queue()

@queue.worker(concurrency=1, poll_interval=2)
def send_email(message):
    payload = message.payload

    print(f"Sending email to {payload['to']}...")
    print(f"  Subject: {payload['subject']}")
    print(f"  Body: {payload['body'][:50]}...")

    # Simulate sending (replace with real email sending)
    time.sleep(1)

    print(f"  Email sent to {payload['to']} successfully!")
```

Expected output for queuing an email:

```
{
  "message": "Email queued for sending",
  "message_id": "550e8400-e29b-41d4-a716-446655440000",
  "priority": "high"
}
```

(Status: `201 Created`)

Expected output for checking status (before processing):

```
{
```

```
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "pending",
  "queue": "emails",
  "payload": {
    "to": "alice@example.com",
    "subject": "Welcome!",
    "body": "Thanks for signing up.",
    "priority_name": "high"
  },
  "attempts": 0,
  "max_attempts": 3,
  "error": null,
  "created_at": "2026-03-22T16:00:00Z",
  "completed_at": null,
  "failed_at": null
}
```

Expected output for checking status (after processing):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "completed",
  "attempts": 1,
  "completed_at": "2026-03-22T16:00:02Z"
}
```

12. Gotchas

1. Queue table not created

Problem: Pushing a message fails with a "table not found" error.

Cause: The `tina4_queue` table has not been created yet. This usually happens when using a new database.

Fix: Tina4 auto-creates the queue table on first use. If that fails, run a migration or create the table manually. Check that your `DATABASE_URL` is correct and the database is accessible.

2. Worker not picking up messages

Problem: Messages are pushed but nothing happens -- the worker is not processing them.

Cause: The worker process is not running, or it is polling a different queue name.

Fix: Make sure the worker is running (`python src/workers/email_worker.py` or via the Tina4 CLI). Check that the queue name in `queue.push("emails", ...)` matches the queue name in `@queue.worker()`.

3. Message processed twice

Problem: A message is processed by two workers simultaneously.

Cause: If using SQLite without WAL mode, concurrent access can cause race conditions.

Fix: For production with multiple workers, use PostgreSQL (which has `FOR UPDATE SKIP LOCKED`) or RabbitMQ. SQLite is fine for single-worker setups. Enable WAL mode for SQLite: `PRAGMA journal_mode=WAL;`

4. Payload too large

Problem: Pushing a message with a large payload fails or is slow.

Cause: The payload is serialized to JSON and stored in the database. Very large payloads (multiple MB) slow down the queue.

Fix: Keep payloads small. Instead of putting file contents in the payload, store the file on disk or in object storage and put the file path in the payload. Payloads should be metadata, not data.

5. Dead letters pile up

Problem: The dead letter queue grows indefinitely.

Cause: Failed messages that exceed max_attempts move to dead letter status but are never cleaned up.

Fix: Monitor your dead letter queue. Set up an alert when dead letters exceed a threshold. Either fix the underlying issue and retry them, or purge them after investigation. Completed messages older than 7 days are automatically purged by default.

6. Environment-specific queue name collision

Problem: Your development and staging environments process each other's queue messages.

Cause: Both environments use the same database or RabbitMQ instance and the same queue names.

Fix: Prefix queue names with the environment: `queue.push("dev_emails", ...)` or use separate database/RabbitMQ instances per environment. The simplest approach is separate databases.

7. Blocking the event loop

Problem: Your async Tina4 server becomes unresponsive while the queue worker is processing.

Cause: The queue worker's `time.sleep()` or CPU-intensive work blocks the async event loop.

Fix: Run queue workers as separate processes, not inside the web server process. Use `python src/workers/email_worker.py` as a separate command, or use Tina4's CLI: `tina4 worker emails`. For CPU-intensive work inside async handlers, use `asyncio.to_thread()` to run blocking code in a thread pool.

Chapter 12: Real-time with WebSocket

1. The Refresh Button Problem

Your project management app needs live updates. Someone moves a card from "In Progress" to "Done." Everyone else on the team should see it. No page refresh. No polling. No waiting.

Traditional HTTP is request-response. The client asks. The server answers. The server cannot push data on its own. WebSocket breaks that wall. A persistent, bi-directional connection between browser and server. Either side sends messages at any time. The connection stays open until one side closes it.

Tina4 treats WebSocket the same way it treats routing. A decorator. A handler. The path determines which handler processes the connection.

2. What WebSocket Is

HTTP works like this:

```
Client: "Give me /api/products"
Server: "Here are the products" (connection closes)
Client: "Give me /api/products" (new connection)
Server: "Here are the products" (connection closes)
```

WebSocket works like this:

```
Client: "I want to upgrade to WebSocket on /ws/chat"
Server: "Upgrade accepted. Connection open."
Client: "Hello everyone!"
Server: "Alice says: Hello everyone!"  (pushed to all clients)
Server: "Bob joined the chat"  (pushed to all clients, at any time)
Client: "Goodbye!"
Server: "Alice says: Goodbye!"
Client: (closes connection)
```

Key differences:

• **Persistent**: The connection stays open. No repeated handshakes.

• **Bi-directional**: The server can push data without the client asking.

• **Low overhead**: After the initial handshake, messages are tiny (no HTTP headers per message).

• **Real-time**: Messages arrive within milliseconds.

3. @websocket -- WebSocket as a Route

In Tina4, you define WebSocket handlers using the `@websocket` decorator:

```
from tina4_python.core.router import websocket

@websocket("/ws/echo")
async def echo_handler(connection, event, data):
    if event == "message":
        await connection.send(f"Echo: {data}")
```

This is the simplest WebSocket handler: it receives a message and sends it back with "Echo: " prepended. The handler receives three arguments:

• **connection**: The WebSocket connection object. Use it to send messages.

• **event**: The event type: `"open"`, `"message"`, or `"close"`.

• **data**: The message data (only present for `"message"` events).

Starting the Server

WebSocket runs alongside your HTTP server:

```
uv run python app.py
```

```
  Tina4 Python v3.0.0
  HTTP server running at http://0.0.0.0:7145
  WebSocket server running at ws://0.0.0.0:7145
  Press Ctrl+C to stop
```

Both HTTP and WebSocket share the same port. The server detects the protocol upgrade request and routes it to the correct handler.

4. Connection Events

Every WebSocket connection goes through three lifecycle events:

Open

Fires when a client connects:

```
import json
from tina4_python.core.router import websocket

@websocket("/ws/notifications")
async def notifications_handler(connection, event, data):
    if event == "open":
        print(f"Client connected: {connection.id}")
        await connection.send(json.dumps({
            "type": "welcome",
            "message": "Connected to notifications",
            "connection_id": connection.id
        }))
```

Message

Fires when a client sends data:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket

@websocket("/ws/notifications")
async def notifications_handler(connection, event, data):
    if event == "open":
        await connection.send(json.dumps({
            "type": "welcome",
            "connection_id": connection.id
        }))

    if event == "message":
```

```
        message = json.loads(data)
        print(f"Received from {connection.id}: {data}")

        if message.get("type") == "ping":
            await connection.send(json.dumps({
                "type": "pong",
                "timestamp": datetime.now(timezone.utc).isoformat()
            }))
```

Close

Fires when a client disconnects:

```
import json
from tina4_python.core.router import websocket

@websocket("/ws/notifications")
async def notifications_handler(connection, event, data):
    if event == "open":
        print(f"Client connected: {connection.id}")

    if event == "message":
        print(f"Message from {connection.id}: {data}")

    if event == "close":
        print(f"Client disconnected: {connection.id}")
        # Clean up: remove from tracking, notify others, etc.
```

A Complete Handler

Here is a handler that responds to all three events:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket

@websocket("/ws/chat")
async def chat_handler(connection, event, data):
    if event == "open":
        print(f"[Chat] New connection: {connection.id}")
        await connection.send(json.dumps({
            "type": "system",
            "message": "Welcome to the chat!",
            "your_id": connection.id
        }))

    elif event == "message":
        message = json.loads(data)
        print(f"[Chat] {connection.id}: {message.get('text', data)}")

        await connection.send(json.dumps({
            "type": "message",
            "from": connection.id,
            "text": message.get("text", data),
            "timestamp": datetime.now(timezone.utc).isoformat()
        }))

// ...
```

5. Sending to a Single Client

`connection.send()` sends a message to the specific client that triggered the event:

```
import json
import psutil
from datetime import datetime, timezone
from tina4_python.core.router import websocket

@websocket("/ws/private")
async def private_handler(connection, event, data):
    if event == "message":
        message = json.loads(data)
        action = message.get("action", "")

        if action == "get-time":
            await connection.send(json.dumps({
                "type": "time",
                "server_time": datetime.now(timezone.utc).isoformat()
            }))

        if action == "get-status":
            await connection.send(json.dumps({
                "type": "status",
                "uptime": 3600,
                "connections": 42,
                "memory_mb": round(psutil.Process().memory_info().rss / 1024 / 1024, 2)
            }))
```

Only the client that sent the message receives the response. Other connected clients do not see it.

6. Broadcasting to All Clients

`connection.broadcast()` sends a message to every client connected to the same WebSocket path:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket

@websocket("/ws/announcements")
async def announcements_handler(connection, event, data):
    if event == "open":
        await connection.broadcast(json.dumps({
            "type": "system",
            "message": "A new user joined",
            "online_count": connection.connection_count()
        }))

    if event == "message":
        message = json.loads(data)

        await connection.broadcast(json.dumps({
            "type": "announcement",
            "from": connection.id,
            "text": message.get("text", ""),
            "timestamp": datetime.now(timezone.utc).isoformat()
        }))

    if event == "close":
        await connection.broadcast(json.dumps({
// ...
```

When one client sends a message, every client connected to `/ws/announcements` receives it. This is the foundation for chat rooms, live dashboards, and collaborative editing.

Broadcast Excluding Sender

Sometimes you want to send to everyone except the client that triggered the event:

```
if event == "message":
    message = json.loads(data)

    # Send to sender (confirmation)
    await connection.send(json.dumps({
        "type": "sent",
        "text": message.get("text")
    }))

    # Send to everyone else
    await connection.broadcast(json.dumps({
        "type": "message",
        "from": message.get("username", "Anonymous"),
        "text": message.get("text"),
        "timestamp": datetime.now(timezone.utc).isoformat()
    }), exclude_sender=True)
```

The `exclude_sender=True` argument to `broadcast()` excludes the sender. The sender gets a "sent" confirmation, and everyone else gets the "message".

7. Path-Scoped Isolation

Different WebSocket paths are completely isolated. Clients connected to `/ws/chat/room-1` do not see messages from `/ws/chat/room-2`:

```
import json
from tina4_python.core.router import websocket

@websocket("/ws/chat/{room}")
async def room_handler(connection, event, data):
    room = connection.params["room"]

    if event == "open":
        print(f"[Room {room}] New connection: {connection.id}")
        await connection.broadcast(json.dumps({
            "type": "system",
            "message": f"Someone joined room {room}",
            "room": room,
            "online": connection.connection_count()
        }))

    if event == "message":
        message = json.loads(data)

        await connection.broadcast(json.dumps({
            "type": "message",
            "room": room,
            "from": message.get("username", "Anonymous"),
            "text": message.get("text", ""),
            "timestamp": datetime.now(timezone.utc).isoformat()
// ...
```

Connect to different rooms:

```
ws://localhost:7145/ws/chat/general    -- general chat
ws://localhost:7145/ws/chat/random     -- random chat
ws://localhost:7145/ws/chat/dev-team   -- dev team chat
```

Broadcasting in `/ws/chat/general` only reaches clients connected to `/ws/chat/general`. The `dev-team` and `random` rooms are separate.

Chat rooms. Project-specific notifications. Per-user channels. No extra configuration. The URL path is the isolation boundary.

8. Building a Live Chat

Here is a complete chat application with usernames, typing indicators, and message history:

WebSocket Handler

Create `src/routes/chat_ws.py`:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket

chat_users = {}

@websocket("/ws/livechat/{room}")
async def livechat_handler(connection, event, data):
    room = connection.params["room"]

    if event == "open":
        chat_users[connection.id] = {
            "id": connection.id,
            "username": "Anonymous",
            "room": room,
            "joined_at": datetime.now(timezone.utc).isoformat()
        }

        await connection.send(json.dumps({
            "type": "welcome",
            "message": f"Connected to room: {room}",
            "your_id": connection.id,
            "online": connection.connection_count()
        }))

// ...
```

9. Live Notifications

WebSocket is great for pushing notifications to users in real time:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket, post, push_to_websocket

@websocket("/ws/notifications/{user_id}")
async def notification_handler(connection, event, data):
    user_id = connection.params["user_id"]

    if event == "open":
```

```
        print(f"[Notifications] User {user_id} connected")
        await connection.send(json.dumps({
            "type": "connected",
            "message": "Listening for notifications"
        }))

    if event == "message":
        message = json.loads(data)
        if message.get("type") == "mark-read":
            print(f"[Notifications] User {user_id} read notification {message['id']}")


@post("/api/orders/{order_id}/ship")
async def ship_order(request, response):
    order_id = request.params["order_id"]
    user_id = request.body.get("user_id", 0)
// ...
```

The `push_to_websocket()` function lets your HTTP handlers send messages to WebSocket clients. This bridges the gap between traditional request-response endpoints and real-time notifications.

10. Connecting from JavaScript

Tina4 provides `frond.js`, a built-in JavaScript helper library that includes WebSocket support:

Basic Connection

```
<script src="/js/frond.js"></script>
<script>
    const ws = frond.ws("/ws/chat/general");

    ws.on("open", function () {
        console.log("Connected to chat");
    });

    ws.on("message", function (data) {
        const message = JSON.parse(data);
        console.log("Received:", message);

        if (message.type === "message") {
            addMessageToUI(message.from, message.text, message.timestamp);
        }

        if (message.type === "system") {
            addSystemMessage(message.message);
        }
    });

    ws.on("close", function () {
        console.log("Disconnected from chat");
    });

// ...
```

Auto-Reconnect

`frond.js` automatically reconnects when the connection drops:

```
const ws = frond.ws("/ws/notifications/42", {
    reconnect: true,           // Enable auto-reconnect (default: true)
    reconnectInterval: 3000,   // Retry every 3 seconds
```

```
    maxReconnectAttempts: 10   // Give up after 10 attempts
});

ws.on("reconnect", function (attempt) {
    console.log("Reconnecting... attempt " + attempt);
});

ws.on("reconnect_failed", function () {
    console.log("Failed to reconnect after maximum attempts");
});
```

Using Native WebSocket

If you prefer not to use `frond.js`, the native WebSocket API works too:

```
const ws = new WebSocket("ws://localhost:7145/ws/chat/general");

ws.onopen = function () {
    console.log("Connected");
    ws.send(JSON.stringify({ type: "set-username", username: "Alice" }));
};

ws.onmessage = function (event) {
    const message = JSON.parse(event.data);
    console.log("Received:", message);
};

ws.onclose = function () {
    console.log("Disconnected");
};

ws.onerror = function (error) {
    console.error("WebSocket error:", error);
};
```

`frond.js` gives you auto-reconnect, message buffering during reconnection, and a cleaner event API. Native WebSocket does not.

11. A Complete Chat Page

Here is a full chat page using templates and WebSocket:

Create `src/templates/chat.html`:

```
{% extends "base.html" %}

{% block title %}Chat - {{ room }}{% endblock %}

{% block content %}
    <h1>Chat Room: {{ room }}</h1>
    <p id="status">Connecting...</p>
    <p id="online">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: scroll; padding: 12px; margin
    </div>

    <div id="typing" style="height: 20px; color: #888; font-style: italic; margin-bottom: 4px;"></div>

    <form id="chat-form" style="display: flex; gap: 8px;">
        <input type="text" id="message-input" placeholder="Type a message..."
               style="flex: 1; padding: 8px; border: 1px solid #ddd; border-radius: 4px;">
```

```
        <button type="submit" style="padding: 8px 16px; background: #333; color: white; border: none; border-r
            Send
        </button>
    </form>

    <script src="/js/frond.js"></script>
    <script>
        const room = "{{ room }}";
// ...
```

Create the route to serve the page:

```
from tina4_python.core.router import get, template

@get("/chat/{room}")
async def chat_page(request, response):
    room = request.params["room"]
    return response(template("chat.html", room=room))
```

Visit `http://localhost:7145/chat/general` in two browser tabs. Type in one tab and watch the message appear in the other instantly.

12. Exercise: Build a Real-Time Chat Room

Build a WebSocket chat room with the following features:

Requirements

1. WebSocket endpoint at `/ws/room/{room_name}` that handles:

• `open`: Send welcome message with connection count

• `message` with type `set-name`: Set the user's display name

• `message` with type `chat`: Broadcast the message to all clients with the sender's name

• `close`: Broadcast that the user left

2. HTTP endpoint at `GET /room/{room_name}` that serves an HTML chat page

3. The chat page should:

• Prompt for a username on load

• Display messages in real time

• Show system messages (join/leave) in a different style

• Show the count of online users

Test by:

1. Open `http://localhost:7145/room/test` in two browser tabs

2. Set different usernames in each

3. Send messages from both tabs and verify they appear in both

4. Close one tab and verify the "user left" message appears in the other

13. Solution

Create `src/routes/chat_room.py`:

```
import json
from datetime import datetime, timezone
from tina4_python.core.router import websocket, get, template

room_users = {}

@websocket("/ws/room/{room_name}")
async def room_ws_handler(connection, event, data):
    room = connection.params["room_name"]
    key = f"{room}:{connection.id}"

    if event == "open":
        room_users[key] = "Anonymous"

        await connection.send(json.dumps({
            "type": "system",
            "message": f"Welcome to room: {room}. Set your name with: "
                       '{"type": "set-name", "name": "YourName"}',
            "online": connection.connection_count()
        }))

        await connection.broadcast(json.dumps({
            "type": "system",
            "message": "A new user joined",
            "online": connection.connection_count()
// ...
```

Create `src/templates/room.html`:

```
{% extends "base.html" %}

{% block title %}Room: {{ room }}{% endblock %}

{% block content %}
    <h1>Room: {{ room }}</h1>
    <p id="online" style="color: #666;">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: auto; padding: 12px; margin-b
    </div>

    <form id="form" style="display: flex; gap: 8px;">
        <input type="text" id="input" placeholder="Type a message..."
               style="flex: 1; padding: 10px; border: 1px solid #ddd; border-radius: 4px; font-size: 14px;">
        <button type="submit"
                style="padding: 10px 20px; background: #1a1a2e; color: white; border: none; border-radius: 4px
            Send
        </button>
    </form>

    <script src="/js/frond.js"></script>
    <script>
        const room = "{{ room }}";
        const username = prompt("Choose a username:") || "Anonymous";
        const ws = frond.ws("/ws/room/" + room);
// ...
```

Open `http://localhost:7145/room/test` in two browser tabs. Set different usernames. Send messages from one tab and watch them appear in both. Close one tab and verify the "left the room" message appears.

14. Gotchas

1. WebSocket Needs a Persistent Server

Problem: WebSocket connections drop immediately or do not work at all.

Cause: You are running behind a traditional WSGI server like Gunicorn with sync workers. WSGI does not support persistent connections. Each request is handled and the connection is closed.

Fix: Use `uv run python app.py` which runs Tina4's async server that handles both HTTP and WebSocket. For production, use an ASGI server like uvicorn or hypercorn (Tina4 auto-detects them). You can still use Nginx as a reverse proxy, but it must be configured for WebSocket proxying with `proxy_set_header Upgrade $http_upgrade` and `proxy_set_header Connection "upgrade"`.

2. Messages Are Strings, Not Dicts

Problem: `data` in the message handler is a string, not a Python dict.

Cause: WebSocket transmits raw strings. If the client sends JSON, you receive the JSON string, not a decoded dict.

Fix: Always `json.loads(data)` when you expect JSON messages. Always `json.dumps()` when you send structured data. WebSocket does not know about content types -- it is just bytes.

3. Connection Count Is Per-Path

Problem: `connection.connection_count()` returns a lower number than expected.

Cause: Connection count is scoped to the WebSocket path. Clients on `/ws/chat/room-1` are counted separately from `/ws/chat/room-2`.

Fix: This is by design. Each path is an isolated group. If you need a global connection count across all paths, maintain a counter in a shared variable or use a module-level dict.

4. Broadcasting Does Not Scale Across Servers

Problem: Users connected to different server instances do not see each other's messages.

Cause: `connection.broadcast()` only sends to clients connected to the same server process. With multiple server instances behind a load balancer, each instance has its own set of connections.

Fix: Use a pub/sub backend like Redis to relay messages across server instances. Each server subscribes to a Redis channel, and broadcast messages are published to Redis so all servers receive them.

5. Large Messages Cause Disconnects

Problem: The connection drops when sending a large message.

Cause: WebSocket servers typically have a maximum message size. The default is usually around 1MB. If your message exceeds this, the server closes the connection.

Fix: Keep messages small (under 64KB is a good target). For large data transfers, use HTTP endpoints instead of WebSocket. WebSocket is designed for small, frequent messages -- not bulk data transfer.

6. Memory Leak from Tracking Connected Users

Problem: The server's memory usage grows over time and eventually crashes.

Cause: You store user data in a dict (like `chat_users`) but do not clean it up when users disconnect. If the `close` event handler does not remove the user from the dict, the dict grows indefinitely.

Fix: Always clean up in the `close` handler. Remove disconnected users from tracking dicts: `chat_users.pop(connection.id, None)`. Test by connecting and disconnecting repeatedly to verify memory stays stable.

7. No Authentication on WebSocket Connect

Problem: Anyone can connect to your WebSocket endpoint and see all messages.

Cause: The WebSocket upgrade request does not carry your JWT token in the `Authorization` header (browsers do not support custom headers on WebSocket connections).

Fix: Pass the token as a query parameter: `ws://localhost:7145/ws/chat?token=eyJ...`. In your `open` handler, validate the token and disconnect if invalid. Use a short-lived token specifically for WebSocket connections. Alternatively, authenticate via an HTTP endpoint first, store the session, and check the session cookie during the WebSocket upgrade.

Chapter 13: Email with Messenger

1. Every App Sends Email

Signup confirmations. Password resets. Weekly digests. Attachments, HTML templates, reliable delivery. Every application needs email. Nobody enjoys implementing it. SMTP configuration. Plain text fallbacks. Attachment encoding. Connection timeouts. The details pile up fast.

Tina4's `Messenger` class absorbs that complexity. Configure via `.env`. Create an instance. Send. In development mode, emails are intercepted and shown in the dev dashboard -- no real SMTP server needed.

2. Messenger Configuration via .env

All email configuration lives in `.env`:

```
TINA4_MAIL_SMTP_HOST=smtp.example.com
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=your-email@example.com
TINA4_MAIL_SMTP_PASSWORD=your-app-password
TINA4_MAIL_SMTP_ENCRYPTION=tls
TINA4_MAIL_FROM_ADDRESS=noreply@example.com
TINA4_MAIL_FROM_NAME=My Store
```

| Variable | Description | Common Values |

|---------|----------|----------|

| `TINA4_MAIL_SMTP_HOST` | SMTP server hostname | `smtp.gmail.com`, `smtp.mailgun.org`, `smtp.sendgrid.net` |

| `TINA4_MAIL_SMTP_PORT` | SMTP port | `587` (TLS), `465` (SSL), `25` (unencrypted) |

| `TINA4_MAIL_SMTP_USERNAME` | Login username | Usually your email address |

| `TINA4_MAIL_SMTP_PASSWORD` | Login password or app-specific password | App passwords for Gmail |

| `TINA4_MAIL_SMTP_ENCRYPTION` | Encryption method | `tls` (recommended), `ssl`, `none` |

| `TINA4_MAIL_FROM_ADDRESS` | Default "From" address | `noreply@yourdomain.com` |

| `TINA4_MAIL_FROM_NAME` | Default "From" display name | `My Store`, `Acme Corp` |

Common Provider Configurations

Gmail:

```
TINA4_MAIL_SMTP_HOST=smtp.gmail.com
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=your-email@gmail.com
TINA4_MAIL_SMTP_PASSWORD=your-app-password
TINA4_MAIL_SMTP_ENCRYPTION=tls
```

Note: Gmail requires an "App Password" (not your regular password) when two-factor authentication is enabled.

Mailgun:

```
TINA4_MAIL_SMTP_HOST=smtp.mailgun.org
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=postmaster@mg.yourdomain.com
TINA4_MAIL_SMTP_PASSWORD=your-mailgun-smtp-password
TINA4_MAIL_SMTP_ENCRYPTION=tls
```

SendGrid:

```
TINA4_MAIL_SMTP_HOST=smtp.sendgrid.net
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=apikey
TINA4_MAIL_SMTP_PASSWORD=your-sendgrid-api-key
TINA4_MAIL_SMTP_ENCRYPTION=tls
```

3. Constructor Override Pattern

If you need to use different SMTP settings for different purposes (transactional emails from one account, marketing from another), override the configuration in the constructor:

```
from tina4_python.messenger import Messenger

# Uses .env defaults
mailer = Messenger()

# Override specific settings
marketing_mailer = Messenger(
    host="smtp.mailgun.org",
    port=587,
    username="marketing@mg.yourdomain.com",
    password="marketing-smtp-password",
    encryption="tls",
    from_address="newsletter@yourdomain.com",
    from_name="My Store Newsletter"
)
```

The constructor accepts keyword arguments that override `.env` values. Unspecified keys fall back to `.env`.

4. Sending Plain Text Email

The simplest email you can send:

```
from tina4_python.core.router import post
from tina4_python.messenger import Messenger

@post("/api/contact")
async def contact_form(request, response):
    body = request.body

    mailer = Messenger()

    result = mailer.send(
        to=body["email"],
        subject="Contact Form Submission",
        body=f"Name: {body['name']}\n"
             f"Email: {body['email']}\n"
             f"Message:\n{body['message']}"
    )
```

```
    if result["success"]:
        return response({"message": "Email sent successfully"})

    return response({"error": "Failed to send email", "details": result["error"]}, 500)
```

```
curl -X POST http://localhost:7145/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "message": "I love your products!"}'
```

```
{"message":"Email sent successfully"}
```

The `send()` method returns a dict with `"success"` (boolean) and `"error"` (string, only present on failure).

The send() Method

```
mailer.send(to, subject, body, **options)
```

• **to**: Recipient email address (string)

• **subject**: Email subject line (string)

• **body**: Email body content (string -- plain text or HTML)

• **options**: Optional keyword arguments (see below)

5. Sending HTML Email with Text Fallback

Most emails should be HTML with a plain text fallback for email clients that do not render HTML:

```
from tina4_python.messenger import Messenger

mailer = Messenger()

html_body = """
<html>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 20px; text-align: center;">
        <h1 style="margin: 0;">Welcome to My Store!</h1>
    </div>
    <div style="padding: 20px;">
        <p>Hi Alice,</p>
        <p>Thank you for creating your account. We are excited to have you!</p>
        <p>Here is what you can do next:</p>
        <ul>
            <li>Browse our <a href="https://mystore.com/products">product catalog</a></li>
            <li>Set up your <a href="https://mystore.com/profile">profile</a></li>
            <li>Check out our <a href="https://mystore.com/deals">current deals</a></li>
        </ul>
        <p>If you have any questions, reply to this email and we will get back to you within 24 hours.</p>
        <p>Cheers,<br>The My Store Team</p>
    </div>
    <div style="background: #f5f5f5; padding: 12px; text-align: center; font-size: 12px; color: #888;">
        <p>You received this because you signed up at mystore.com</p>
    </div>
// ...
```

The `text_body` option provides the plain text fallback. Email clients that cannot render HTML will show the text version instead.

6. Adding Attachments

Attach files by providing their paths:

```
from tina4_python.messenger import Messenger

mailer = Messenger()

result = mailer.send(
    to="accounting@example.com",
    subject="Monthly Invoice #1042",
    body="<h2>Invoice #1042</h2><p>Please find the invoice attached.</p>",
    attachments=[
        "/path/to/invoices/invoice-1042.pdf",
        "/path/to/reports/monthly-summary.csv"
    ]
)
```

Each attachment is the absolute file path. Tina4 reads the file, determines the MIME type, and encodes it for email transmission.

Inline Attachments with Custom Names

```
result = mailer.send(
    to="alice@example.com",
    subject="Your Export",
    body="<p>Here is your data export.</p>",
    attachments=[
        {
            "path": "/tmp/export-20260322.csv",
            "name": "my-store-export.csv"
        }
    ]
)
```

The recipient sees the file named `my-store-export.csv` regardless of its actual filename on disk.

7. CC and BCC

```
from tina4_python.messenger import Messenger

mailer = Messenger()

result = mailer.send(
    to="alice@example.com",
    subject="Team Meeting Notes",
    body="<p>Here are the notes from today's meeting.</p>",
    cc=["bob@example.com", "charlie@example.com"],
    bcc=["manager@example.com"],
    reply_to="alice@example.com"
)
```

• **cc**: List of email addresses to carbon copy. All recipients can see CC addresses.

• **bcc**: List of email addresses to blind carbon copy. Recipients cannot see BCC addresses.

• **reply_to**: When the recipient clicks "Reply", this address is used instead of the "From" address.

8. Reading Inbox via IMAP

Tina4's Messenger can also read emails via IMAP:

```
TINA4_MAIL_IMAP_HOST=imap.example.com
TINA4_MAIL_IMAP_PORT=993
TINA4_MAIL_IMAP_USERNAME=support@example.com
TINA4_MAIL_IMAP_PASSWORD=your-imap-password
TINA4_MAIL_IMAP_ENCRYPTION=ssl
```

```
from tina4_python.core.router import get
from tina4_python.messenger import Messenger

@get("/api/inbox")
async def get_inbox(request, response):
    mailer = Messenger()

    emails = mailer.get_inbox(limit=20, unread_only=True)

    messages = []
    for email in emails:
        messages.append({
            "id": email["id"],
            "from": email["from"],
            "subject": email["subject"],
            "date": email["date"],
            "preview": email["text_body"][:200],
            "has_attachments": bool(email.get("attachments"))
        })

    return response({"messages": messages, "count": len(messages)})
```

```
curl http://localhost:7145/api/inbox
```

```
{
  "messages": [
    {
      "id": "12345",
      "from": "customer@example.com",
      "subject": "Order question",
      "date": "2026-03-22T10:30:00+00:00",
      "preview": "Hi, I have a question about my recent order...",
      "has_attachments": false
    }
  ],
  "count": 1
}
```

Reading a Specific Email

```
@get("/api/inbox/{email_id}")
async def get_email(request, response):
    mailer = Messenger()
    email_id = request.params["email_id"]

    email = mailer.get_message(email_id)

    if email is None:
        return response({"error": "Email not found"}, 404)

    return response({
        "id": email["id"],
        "from": email["from"],
        "to": email["to"],
```

```
        "subject": email["subject"],
        "date": email["date"],
        "html_body": email["html_body"],
        "text_body": email["text_body"],
        "attachments": [
            {"name": a["name"], "size": a["size"], "type": a["type"]}
            for a in email.get("attachments", [])
        ]
    })
```

9. Dev Mode: Email Interception

When `TINA4_DEBUG=true`, Tina4 intercepts all outgoing emails and shows them in the dev dashboard. No real recipients receive anything. No accidental emails during development.

Navigate to `/tina4/console` and look for the "Mail" section. You will see:

• Every email that was "sent" during the current session

• The To, CC, and BCC addresses

• The subject and body (both HTML and plain text)

• Attachments (viewable inline)

• The timestamp

This is invaluable for testing email functionality without configuring a real SMTP server or polluting someone's inbox.

Disabling Interception

If you need to test real email delivery during development, override the interception:

```
TINA4_MAIL_INTERCEPT=false
```

With this set, emails are sent to real recipients even when `TINA4_DEBUG=true`. Use with caution -- you do not want to accidentally email your entire user base from a dev machine.

10. Using Templates for Email Content

Hardcoded HTML in Python strings is ugly and hard to maintain. Templates fix this:

Create `src/templates/emails/welcome.html`:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
</head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; background: #f5f5f5; padding: 2
    <div style="background: #1a1a2e; color: white; padding: 24px; text-align: center; border-radius: 8px 8px (
        <h1 style="margin: 0; font-size: 24px;">Welcome, {{ name }}!</h1>
    </div>
    <div style="background: white; padding: 24px; border-radius: 0 0 8px 8px;">
        <p>Hi {{ name }},</p>
        <p>Your account has been created successfully. Here are your details:</p>

        <table style="width: 100%; border-collapse: collapse; margin: 16px 0;">
```

```
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Email</td>
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ email }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Account ID</td>
                <td style="padding: 8px; border-bottom: 1px solid #eee;">#{{ user_id }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Signed Up</td>
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ signed_up_at }}</td>
// ...
```

Rendering and Sending

```
import os
import secrets
from datetime import datetime
from tina4_python.core.router import post, template
from tina4_python.messenger import Messenger

@post("/api/register")
async def register_user(request, response):
    body = request.body

    # Create user (database logic)
    user_id = 42

    # Render the email template
    email_data = {
        "name": body["name"],
        "email": body["email"],
        "user_id": user_id,
        "signed_up_at": datetime.now().strftime("%B %d, %Y"),
        "base_url": os.getenv("APP_URL", "http://localhost:7145"),
        "app_name": "My Store",
        "promo_code": "WELCOME10",
        "unsubscribe_token": secrets.token_hex(16)
    }

// ...
```

```
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "email_sent": true,
  "user_id": 42
}
```

With `TINA4_DEBUG=true`, the email appears in the dev dashboard instead of being sent. You can inspect the rendered HTML, check that template variables were substituted correctly, and verify the layout.

11. Sending Email via Queues

In production, never send email inside a route handler. The SMTP call blocks the response. Use the queue system from Chapter 11:

```
from tina4_python.core.router import post, template
```

```
from tina4_python.queue import Queue
from tina4_python.messenger import Messenger

# In the route handler, just queue the email
@post("/api/register")
async def register_user(request, response):
    body = request.body
    user_id = 42  # Simulated

    queue = Queue(topic="emails")
    queue.push({
        "template": "emails/welcome.html",
        "to": body["email"],
        "subject": f"Welcome to My Store, {body['name']}!",
        "data": {
            "name": body["name"],
            "email": body["email"],
            "user_id": user_id,
            "signed_up_at": "March 22, 2026",
            "base_url": "http://localhost:7145",
            "app_name": "My Store",
            "promo_code": "WELCOME10"
        }
    })
// ...
```

The route handler returns in under 50 milliseconds. The queue worker sends the email on its own timeline. If the SMTP server is down, retries happen automatically.

12. Exercise: Build a Contact Form with Email Notification

Build a contact form that sends an email notification when submitted.

Requirements

1. Create a `GET /contact` page that renders a contact form with fields: name, email, subject, and message

2. Create a `POST /contact` endpoint that:

• Validates all fields are present

• Sends an email notification to the site admin (`admin@example.com`)

• The email should include all form fields, nicely formatted

• Shows a flash message on success

• Redirects back to the contact page

3. Create an email template at `src/templates/emails/contact-notification.html` that formats the contact submission

Test with:

```
# View the form
curl http://localhost:7145/contact

# Submit the form
curl -X POST http://localhost:7145/contact \
  -H "Content-Type: application/json" \
```

```
  -d '{"name": "Bob", "email": "bob@example.com", "subject": "Product inquiry", "message": "Do you ship intern

# Check the dev dashboard for the intercepted email
# Navigate to http://localhost:7145/tina4/console
```

13. Solution

Create `src/templates/emails/contact-notification.html`:

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 16px; text-align: center;">
        <h2 style="margin: 0;">New Contact Form Submission</h2>
    </div>
    <div style="padding: 20px; background: white;">
        <table style="width: 100%; border-collapse: collapse;">
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold; width: 100px;">Fro
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ name }} ({{ email }})</td>
            </tr>
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold;">Subject</td>
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ subject }}</td>
            </tr>
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold;">Date</td>
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ submitted_at }}</td>
            </tr>
        </table>

        <div style="margin-top: 16px; padding: 16px; background: #f8f9fa; border-radius: 4px;">
            <h3 style="margin-top: 0;">Message</h3>
// ...
```

Create `src/templates/contact.html`:

```
{% extends "base.html" %}

{% block title %}Contact Us{% endblock %}

{% block content %}
    <h1>Contact Us</h1>

    {% if flash %}
        <div style="padding: 12px; border-radius: 4px; margin-bottom: 16px;
            {% if flash.type == 'success' %}background: #d4edda; color: #155724;{% endif %}
            {% if flash.type == 'error' %}background: #f8d7da; color: #721c24;{% endif %}">
            {{ flash.message }}
        </div>
    {% endif %}

    <form method="POST" action="/contact" style="max-width: 500px;">
        <div style="margin-bottom: 12px;">
            <label for="name" style="display: block; margin-bottom: 4px; font-weight: bold;">Name</label>
            <input type="text" name="name" id="name" required
                   style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;">
        </div>

        <div style="margin-bottom: 12px;">
            <label for="email" style="display: block; margin-bottom: 4px; font-weight: bold;">Email</label>
```

```
            <input type="email" name="email" id="email" required
// ...
```

Create `src/routes/contact.py`:

```
import os
from datetime import datetime
from tina4_python.core.router import get, post, template
from tina4_python.messenger import Messenger

@get("/contact")
async def contact_page(request, response):
    flash = request.session.get("_flash")
    if flash:
        del request.session["_flash"]

    return response(template("contact.html", flash=flash))


@post("/contact")
async def contact_submit(request, response):
    body = request.body

    # Validate
    errors = []
    if not body.get("name"):
        errors.append("Name is required")
    if not body.get("email"):
        errors.append("Email is required")
    if not body.get("subject"):
// ...
```

Testing:

1. Open `http://localhost:7145/contact` in your browser

2. Fill in the form and submit

3. You should see a green "Thank you" flash message

4. Open `http://localhost:7145/tina4/console` to see the intercepted email

5. The email should show the sender details, subject, message, and formatted HTML

API test:

```
curl -X POST http://localhost:7145/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Bob", "email": "bob@example.com", "subject": "Product inquiry", "message": "Do you ship intern
  -c cookies.txt -b cookies.txt
```

The response is a `302` redirect to `/contact`. Follow the redirect to see the flash message:

```
curl http://localhost:7145/contact -b cookies.txt
```

The HTML response includes the success flash message.

14. Gotchas

1. Gmail Blocks "Less Secure" Apps

Problem: Sending via Gmail fails with "Authentication failed" or "Username and Password not accepted".

Cause: Gmail blocks SMTP access from apps that do not use OAuth2 by default. Using your regular password will not work if two-factor authentication is enabled.

Fix: Generate an "App Password" in your Google Account settings (Security > 2-Step Verification > App Passwords). Use this 16-character password as `TINA4_MAIL_SMTP_PASSWORD`. This is separate from your regular Google password.

2. Emails Go to Spam

Problem: Emails are delivered but land in the recipient's spam folder.

Cause: Your sending domain lacks proper DNS records (SPF, DKIM, DMARC) or you are sending from a free email provider (Gmail, Yahoo).

Fix: Use a dedicated sending domain with proper DNS records. Set up SPF, DKIM, and DMARC records. Use a transactional email service like Mailgun, SendGrid, or Amazon SES that handles email reputation for you.

3. HTML Email Looks Broken

Problem: The email looks fine in Gmail but broken in Outlook or Apple Mail.

Cause: Email clients have wildly different HTML/CSS support. CSS flexbox, grid, and many modern properties do not work in email.

Fix: Use inline styles (not external stylesheets or `<style>` blocks). Use table-based layouts for complex designs. Test with an email preview tool. Keep it simple -- most transactional emails do not need elaborate designs.

4. Attachment File Not Found

Problem: `Messenger.send()` returns an error about a missing file.

Cause: The attachment path is relative or incorrect. The file does not exist at the specified location.

Fix: Use absolute paths for attachments. Verify the file exists before calling `send()`: `if not os.path.exists(path): ...`. If the file is generated dynamically (like a PDF), make sure the generation completes before sending.

5. Dev Mode Silently Intercepts Emails

Problem: You set up SMTP correctly but no emails arrive. No errors either.

Cause: `TINA4_DEBUG=true` intercepts all emails and shows them in the dev dashboard. The email is never sent to the SMTP server.

Fix: Check the dev dashboard at `/tina4/console` for intercepted emails. If you want to send real emails during development, set `TINA4_MAIL_INTERCEPT=false`. Remember to remove this setting before committing.

6. Email Template Variables Not Substituted

Problem: The email body shows `{{ name }}` literally instead of the user's name.

Cause: You passed the raw template file content instead of rendering it through the template engine. The template engine was not invoked.

Fix: Use `template("emails/template.html", **data)` to render the template with variables substituted. Do not read the file with `open()` -- that gives you the raw template source.

7. Connection Timeout on Send

Problem: `Messenger.send()` hangs for 30 seconds and then fails with a timeout error.

Cause: The SMTP server is unreachable from your network, the port is blocked by a firewall, or the hostname is wrong.

Fix: Test SMTP connectivity: `telnet smtp.example.com 587`. Verify the hostname, port, and encryption settings. Check that your firewall allows outbound connections on the SMTP port. If you are behind a corporate firewall, port 587 or 465 might be blocked -- ask your network administrator.

Chapter 14: Caching

1. From 800ms to 3ms

Your product catalog page runs 12 database queries. 800 milliseconds to render. Every visitor triggers the same queries, the same template rendering, the same JSON serialization -- for data that changes once a day.

Add caching. The first request takes 800ms. The next 10,000 take 3ms each. A 266x improvement from one line of configuration.

Caching stores the result of expensive operations for reuse. Tina4 provides three levels: response caching (entire HTTP responses), database query caching, and a direct cache API for custom use cases.

2. Response Caching with ResponseCache Middleware

The fastest way to cache is at the HTTP response level. The `ResponseCache` middleware stores the complete response (headers and body) and serves it directly on subsequent requests without calling your route handler at all.

```
from tina4_python.core.router import get

@get("/api/products", middleware=["ResponseCache:300"])
async def list_products(request, response):
    # This handler runs 12 database queries and takes 800ms
    # With ResponseCache, it only runs once every 5 minutes

    print("Handler called -- this should only appear once every 5 minutes")

    products = [
        {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
        {"id": 2, "name": "USB-C Hub", "price": 49.99},
        {"id": 3, "name": "Monitor Stand", "price": 129.99}
    ]

    return response({"products": products, "generated_at": datetime.now(timezone.utc).isoformat()})
```

The `"ResponseCache:300"` middleware caches the response for 300 seconds (5 minutes). During those 5 minutes:

• The first request runs the handler (800ms)

• The next 10,000 requests serve the cached response (3ms each)

• After 300 seconds, the cache expires and the next request runs the handler again

```
curl http://localhost:7145/api/products

{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
    {"id": 2, "name": "USB-C Hub", "price": 49.99},
    {"id": 3, "name": "Monitor Stand", "price": 129.99}
  ],
  "generated_at": "2026-03-22T14:30:00+00:00"
}
```

Call it again within 5 minutes:

```
curl http://localhost:7145/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
    {"id": 2, "name": "USB-C Hub", "price": 49.99},
    {"id": 3, "name": "Monitor Stand", "price": 129.99}
  ],
  "generated_at": "2026-03-22T14:30:00+00:00"
}
```

Notice the `generated_at` timestamp is the same. The handler did not run -- the response came from cache.

Cache Headers

The `ResponseCache` middleware automatically sets cache-related headers:

```
X-Cache: HIT
X-Cache-TTL: 247
Cache-Control: public, max-age=300
```

- `X-Cache: HIT` or `X-Cache: MISS` tells you whether the response came from cache
- `X-Cache-TTL` shows the remaining time-to-live in seconds
- `Cache-Control` enables browser and CDN caching

Caching with Query Parameters

By default, the cache key includes the full URL with query parameters. `/api/products?page=1` and `/api/products?page=2` are cached separately:

```
@get("/api/products", middleware=["ResponseCache:300"])
async def list_products(request, response):
    page = int(request.query.get("page", 1))
    limit = 20
    offset = (page - 1) * limit

    return response({
        "page": page,
        "products": [],
        "generated_at": datetime.now(timezone.utc).isoformat()
    })
```

```
curl "http://localhost:7145/api/products?page=1"  # Cache MISS, stores for page=1
curl "http://localhost:7145/api/products?page=2"  # Cache MISS, stores for page=2
curl "http://localhost:7145/api/products?page=1"  # Cache HIT
```

What Not to Cache

Do not use `ResponseCache` on:

- **POST, PUT, PATCH, DELETE routes**: Only GET responses should be cached
- **User-specific endpoints**: `/api/profile` returns different data for each user
- **Real-time data**: Stock prices, live scores, chat messages
- **Authenticated endpoints**: Unless the cache is scoped per user

```
# GOOD: Public, rarely changing data
@get("/api/categories", middleware=["ResponseCache:3600"])
async def list_categories(request, response):
    return response({"categories": []})
```

```
# BAD: User-specific data -- do NOT cache
@get("/api/profile", middleware=["auth_middleware"])
async def get_profile(request, response):
    return response(request.user)
```

3. Memory Cache (Default)

Tina4's cache system stores data in memory by default. No configuration needed.

```
# This is the default -- you do not need to set it explicitly
TINA4_CACHE_BACKEND=memory
```

Memory cache is the fastest option (no disk I/O, no network calls) but it resets when the server restarts. It is ideal for development and single-server deployments where losing the cache on restart is acceptable.

4. Redis Cache

For production deployments where you want cache persistence across server restarts and shared cache across multiple server instances, use Redis:

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_HOST=localhost
TINA4_CACHE_PORT=6379
TINA4_CACHE_PASSWORD=your-redis-password
TINA4_CACHE_PREFIX=myapp:cache:
```

Your code does not change. The `cache_get`, `cache_set`, and `ResponseCache` middleware all work identically. Only the storage backend changes.

Why Redis?

• Cache survives server restarts

• Shared across multiple server instances (behind a load balancer)

• Sub-millisecond reads and writes

• Built-in key expiry (TTL cleanup is automatic)

• Same Redis instance can serve sessions, cache, and queues

5. File Cache

If you want cache persistence but do not have Redis, use file-based caching:

```
TINA4_CACHE_BACKEND=file
TINA4_CACHE_PATH=/path/to/cache/directory
```

File cache stores each cache entry as a file on disk. It is slower than memory or Redis but survives server restarts without extra infrastructure.

When to Use File Cache

• You need cache persistence but cannot run Redis

• Your hosting environment is limited (shared hosting, no external services)

• Cache entries are large and you do not want them in memory

6. Direct Cache API

For custom caching logic, use the `cache_get`, `cache_set`, and `cache_delete` functions directly:

cache_set

```
from tina4_python.cache import cache_set

# Cache a value for 300 seconds
cache_set("product:42", {
    "id": 42,
    "name": "Wireless Keyboard",
    "price": 79.99,
    "in_stock": True
}, ttl=300)

# Cache a string
cache_set("exchange_rate:USD_EUR", "0.92", ttl=3600)

# Cache indefinitely (no TTL)
cache_set("app:config", {"theme": "dark", "lang": "en"})
```

cache_get

```
from tina4_python.cache import cache_get, cache_set

product = cache_get("product:42")
# Returns the cached value, or None if not found or expired

if product is None:
    # Cache miss -- fetch from database
    product = fetch_product_from_database(42)
    cache_set("product:42", product, ttl=300)

return response(product)
```

cache_delete

```
from tina4_python.cache import cache_delete

# Delete a specific key
cache_delete("product:42")

# Delete multiple keys
cache_delete("product:42")
cache_delete("product:43")
cache_delete("product:44")
```

Real-World Pattern: Cache-Aside

The most common caching pattern is cache-aside (also called lazy loading):

```
from tina4_python.core.router import get
from tina4_python.cache import cache_get, cache_set

@get("/api/products/{product_id}")
async def get_product(request, response):
    product_id = request.params["product_id"]
    cache_key = f"product:{product_id}"
```

```
    # 1. Try the cache first
    product = cache_get(cache_key)

    if product is not None:
        # Cache hit -- return immediately
        return response({**product, "source": "cache"})

    # 2. Cache miss -- fetch from database
    db = Database.get_connection()
    product = db.fetch_one(
        "SELECT id, name, category, price, in_stock FROM products WHERE id = :id",
        {"id": product_id}
    )

    if product is None:
        return response({"error": "Product not found"}, 404)

// ...
```

```
curl http://localhost:7145/api/products/42
```

First call (cache miss):

```
{
  "id": 42,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "source": "database"
}
```

Second call (cache hit):

```
{
  "id": 42,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "source": "cache"
}
```

7. Database Query Caching

Tina4 can cache database query results automatically. Enable it in `.env`:

```
TINA4_DB_CACHE=true
TINA4_DB_CACHE_TTL=300
```

With database caching enabled, identical queries return cached results instead of hitting the database:

```
from tina4_python.core.router import get

@get("/api/categories")
async def list_categories(request, response):
    db = Database.get_connection()

    # First call: executes the query (20ms)
    # Subsequent calls within 300 seconds: returns cached result (0.1ms)
    categories = db.fetch_all("SELECT * FROM categories ORDER BY name")

    return response({"categories": categories})
```

The cache key is derived from the SQL query and its parameters. Different queries or different parameters produce different cache keys:

```
# These are cached separately:
db.fetch_all("SELECT * FROM products WHERE category = :cat", {"cat": "Electronics"})
db.fetch_all("SELECT * FROM products WHERE category = :cat", {"cat": "Fitness"})
```

When to Use DB Cache

• Read-heavy applications where the same queries run repeatedly

• Reference data that changes infrequently (categories, countries, settings)

• Dashboard queries that aggregate large datasets

When Not to Use DB Cache

• Write-heavy applications where data changes constantly

• Queries with real-time requirements (inventory counts, live prices)

• Queries that must always return the latest data

Skipping Cache for Specific Queries

```
# Force a fresh query, bypassing the cache
fresh_data = db.fetch_all("SELECT * FROM products", no_cache=True)
```

8. Cache Invalidation Strategies

Cache invalidation is the hard problem. Stale cache serves outdated data. Premature invalidation throws away performance gains. Three strategies handle this.

Strategy 1: Time-Based Expiry (TTL)

The simplest strategy. Set a TTL and let the cache expire naturally:

```
cache_set("products:featured", featured_products, ttl=600)  # Expires in 10 minutes
```

Good for data where near-real-time accuracy is acceptable. A 10-minute delay in updating the featured products list is usually fine.

Strategy 2: Event-Based Invalidation

Clear the cache when the underlying data changes:

```
from tina4_python.core.router import put
from tina4_python.cache import cache_set, cache_delete

@put("/api/products/{product_id}")
async def update_product(request, response):
    product_id = request.params["product_id"]
    body = request.body
    db = Database.get_connection()

    db.execute(
        "UPDATE products SET name = :name, price = :price WHERE id = :id",
        {"name": body["name"], "price": body["price"], "id": product_id}
    )

    # Invalidate the cache for this product
```

```
    cache_delete(f"product:{product_id}")

    # Also invalidate any list caches that might include this product
    cache_delete("products:all")
    cache_delete("products:featured")

    updated = db.fetch_one("SELECT * FROM products WHERE id = :id", {"id": product_id})

    return response(updated)
```

This is the most accurate strategy -- the cache is always fresh after a write. The downside is you must remember to invalidate everywhere the data could be cached.

Strategy 3: Write-Through Cache

Update the cache at the same time as the database:

```
@put("/api/products/{product_id}")
async def update_product(request, response):
    product_id = request.params["product_id"]
    body = request.body
    db = Database.get_connection()

    db.execute(
        "UPDATE products SET name = :name, price = :price WHERE id = :id",
        {"name": body["name"], "price": body["price"], "id": product_id}
    )

    updated = db.fetch_one("SELECT * FROM products WHERE id = :id", {"id": product_id})

    # Write the new data directly to cache (instead of deleting)
    cache_set(f"product:{product_id}", updated, ttl=600)

    return response(updated)
```

This ensures the cache always has the latest data. No cache miss after an update -- the next read comes from the already-warm cache.

9. TTL Management

Choosing the right TTL depends on how often the data changes and how acceptable stale data is:

| Data Type | Suggested TTL | Reasoning |

|----------|----------|----------|

| Static config (categories, countries) | 3600 (1 hour) | Changes rarely, stale data is harmless |

| Product catalog | 300 (5 min) | Updates several times per day |

| User profile | 60 (1 min) | Users expect changes to appear quickly |

| Search results | 120 (2 min) | Balance between freshness and performance |

| Dashboard stats | 30 (30 sec) | Near-real-time but expensive to compute |

| Exchange rates | 60 (1 min) | Updates frequently, slight delay is acceptable |

| Shopping cart | 0 (no cache) | Must always reflect current state |

Dynamic TTL

Adjust TTL based on data characteristics:

```
from tina4_python.cache import cache_get, cache_set

def get_cached_product(product_id):
    cache_key = f"product:{product_id}"
    product = cache_get(cache_key)

    if product is not None:
        return product

    product = fetch_product_from_database(product_id)

    # Popular products: shorter TTL (more likely to change)
    # Inactive products: longer TTL (rarely change)
    ttl = 60 if product["view_count"] > 1000 else 3600

    cache_set(cache_key, product, ttl=ttl)

    return product
```

10. Cache Statistics

Monitor cache performance to verify that caching is actually helping:

```
from tina4_python.core.router import get
from tina4_python.cache import cache_stats

@get("/api/cache/stats")
async def get_cache_stats(request, response):
    stats = cache_stats()
    return response(stats)
```

```
curl http://localhost:7145/api/cache/stats
```

```
{
  "backend": "memory",
  "entries": 42,
  "hits": 15234,
  "misses": 891,
  "hit_rate": "94.5%",
  "memory_bytes": 524288,
  "oldest_entry_age": 3542
}
```

Hit rate above 90%: your caching strategy works. Below 80%: TTLs are too short, cache is too small, or you are caching data that is not accessed often enough to benefit.

The dev dashboard at `/tina4/console` shows cache statistics too -- per-key hit counts and miss counts. You see which keys earn their keep.

11. Combining Cache Layers

For maximum performance, layer multiple cache strategies:

```
from datetime import datetime, timezone
from tina4_python.core.router import get
from tina4_python.cache import cache_get, cache_set
import math
```

```
@get("/api/catalog", middleware=["ResponseCache:60"])
async def get_catalog(request, response):
    page = int(request.query.get("page", 1))
    cache_key = f"catalog:page:{page}"

    # Layer 1: Check application cache
    catalog = cache_get(cache_key)

    if catalog is not None:
        return response({**catalog, "cache": "application"})

    # Layer 2: Database query (with DB-level caching if TINA4_DB_CACHE=true)
    db = Database.get_connection()
    limit = 20
    offset = (page - 1) * limit

    products = db.fetch_all(
        """SELECT p.*, c.name as category_name
           FROM products p
           JOIN categories c ON p.category_id = c.id
// ...
```

This creates three cache layers:

1. **ResponseCache** (60 seconds): The entire HTTP response is cached. No Python code runs at all.

2. **Application cache** (300 seconds): If the response cache expired but the app cache is still fresh, skip the database queries.

3. **DB query cache** (if enabled): Individual query results are cached even if the application cache missed.

The first visitor after a full cache expiry waits 800ms. Everyone else gets the response in under 5ms.

12. Exercise: Cache an Expensive Product Listing Endpoint

Build a product listing endpoint that uses caching at multiple levels.

Requirements

1. Create a `GET /api/store/products` endpoint that:

• Accepts query parameters: `category`, `page`, `limit`

• Returns a list of products with pagination metadata

• Uses the direct cache API (`cache_get`/`cache_set`) with a 5-minute TTL

• Includes a `source` field in the response (`"cache"` or `"database"`)

2. Create a `POST /api/store/products` endpoint that:

• Creates a new product

• Invalidates the relevant cache entries

3. Create a `GET /api/store/cache-stats` endpoint that shows cache statistics

Test with:

```
# First call -- cache miss, slow
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"
```

```
# Second call -- cache hit, fast
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"

# Different category -- cache miss
curl "http://localhost:7145/api/store/products?category=Fitness&page=1"

# Create a product -- should invalidate cache
curl -X POST http://localhost:7145/api/store/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Smart Watch", "category": "Electronics", "price": 299.99}'

# Same query again -- cache miss (invalidated by the POST)
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"

# Check cache stats
curl http://localhost:7145/api/store/cache-stats
```

13. Solution

Create `src/routes/store_cached.py`:

```
import json
import time
import hashlib
from datetime import datetime, timezone
from tina4_python.core.router import get, post
from tina4_python.cache import cache_get, cache_set, cache_delete, cache_stats


def get_product_store():
    return [
        {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": True},
        {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": True},
        {"id": 3, "name": "Coffee Grinder", "category": "Kitchen", "price": 49.99, "in_stock": False},
        {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": True},
        {"id": 5, "name": "Running Shoes", "category": "Fitness", "price": 119.99, "in_stock": True},
        {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "in_stock": True},
        {"id": 7, "name": "Resistance Bands", "category": "Fitness", "price": 14.99, "in_stock": True},
        {"id": 8, "name": "French Press", "category": "Kitchen", "price": 34.99, "in_stock": True},
    ]


@get("/api/store/products")
async def list_store_products(request, response):
    category = request.query.get("category")
    page = int(request.query.get("page", 1))
// ...
```

Expected output -- first call (cache miss):

```
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"

{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": true},
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": true},
    {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "in_stock": true}
  ],
  "page": 1,
  "limit": 20,
  "total": 3,
  "pages": 1,
```

```
  "generated_at": "2026-03-22T14:30:00+00:00",
  "source": "database"
}
```

Expected output -- second call (cache hit):

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": true},
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": true},
    {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "in_stock": true}
  ],
  "page": 1,
  "limit": 20,
  "total": 3,
  "pages": 1,
  "generated_at": "2026-03-22T14:30:00+00:00",
  "source": "cache"
}
```

Notice: same `generated_at`, but `source` changed from `"database"` to `"cache"`. The handler did not run.

14. Gotchas

1. Caching Authenticated Responses

Problem: User A's profile is served to User B because the response was cached.

Cause: `ResponseCache` caches by URL only. If `/api/profile` returns different data per user but all requests hit the same URL, the first user's response is served to everyone.

Fix: Do not use `ResponseCache` on user-specific endpoints. Use the direct cache API with user-specific keys instead: `cache_set(f"profile:{user_id}", data, ttl=300)`.

2. Cache Stampede

Problem: When a popular cache key expires, hundreds of requests hit the database simultaneously (all experiencing a cache miss at the same moment).

Cause: All requests see the cache miss and all try to rebuild the cache independently.

Fix: Use cache locking or "stale-while-revalidate". One request rebuilds the cache while others serve the stale value. Tina4's `ResponseCache` handles this automatically by serving the expired response to concurrent requests while one request refreshes it.

3. Memory Cache Lost on Restart

Problem: After restarting the server, performance drops until the cache warms up.

Cause: Memory cache is lost when the process restarts. Every request is a cache miss until data is cached again.

Fix: For production, use Redis cache (`TINA4_CACHE_BACKEND=redis`). It persists across server restarts. Alternatively, implement a cache warmup script that pre-populates the cache with frequently accessed data.

4. Stale Data After Database Update

Problem: You updated a product's price in the database, but the API still returns the old price.

Cause: The cache still has the old data and has not expired yet.

Fix: Always invalidate (or update) the cache when you modify the underlying data. Use `cache_delete(f"product:{product_id}")` after an update, or use write-through caching with `cache_set()` to update the cache with the new value.

5. Cache Key Collisions

Problem: Two different queries return the same cached data.

Cause: Your cache keys are not specific enough. Using `"products"` as a key for both the full list and a filtered list causes collisions.

Fix: Include all relevant parameters in the cache key: `"products:category:Electronics:page:1:limit:20"`. Or use an MD5 hash of the parameters: `f"products:{hashlib.md5(json.dumps(params).encode()).hexdigest()}"`.

6. Serialization Overhead

Problem: Caching makes certain requests slower, not faster.

Cause: The cached object is very large. Serializing and deserializing it takes more time than re-computing it.

Fix: Only cache data that is expensive to compute. If the original operation takes 5ms and cache serialization takes 10ms, caching is counterproductive. Profile before and after caching to verify the improvement.

7. Forgetting to Set TTL

Problem: Cache entries never expire and the server's memory grows until it crashes.

Cause: You called `cache_set("key", value)` without a TTL. The entry lives forever (or until the server restarts).

Fix: Always set a TTL: `cache_set("key", value, ttl=300)`. Even for data that "never changes," set a long TTL like 86400 (24 hours). This provides a safety net against stale data and prevents unbounded memory growth.

Chapter 15: Frontend with tina4css

1. The Problem with Frontend Toolchains

Your client wants a dashboard. You know the drill. Install Node.js. Run `npm install`. Wait for 200MB of `node_modules`. Configure webpack or Vite. Set up PostCSS. Add a CSS framework. Pray nothing breaks when you upgrade a dependency six months from now.

Tina4 skips all of that. The framework ships with **tina4css** -- a Bootstrap-compatible CSS framework -- and **frond.js** -- a lightweight JavaScript helper. Both are included when you scaffold a project. No npm. No webpack. No build step. Link the files. Start building.

By the end of this chapter, you will have a complete admin dashboard with sidebar, navigation, cards, tables, modals, and dark mode -- without touching npm.

2. What Ships with Tina4

When you run `tina4 init python`, two files appear in your project:

```
src/public/
■■■ css/
■   ■■■ tina4.css        # The CSS framework
■■■ js/
■   ■■■ frond.js         # AJAX helpers, form submission, token management
■■■ scss/
    ■■■ tina4.scss       # SCSS source (optional, for customization)
```

Include them in any template:

```
<link rel="stylesheet" href="/css/tina4.css">
<script src="/js/frond.js"></script>
```

That is all you need. No CDN, no package manager, no version conflicts.

3. The Grid System

tina4css uses a 12-column responsive grid, compatible with Bootstrap's class names. If you already know Bootstrap, you already know tina4css.

```
<div class="container">
    <div class="row">
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
    </div>
</div>
```

Breakpoints:

| Class Prefix | Screen Width | Typical Device |

|---|---|---|

| `col-` | All sizes | Phones and up |

| `col-sm-` | >= 576px | Large phones |

| `col-md-` | >= 768px | Tablets |

| `col-lg-` | >= 992px | Desktops |

| `col-xl-` | >= 1200px | Large desktops |

Columns stack vertically on screens smaller than their breakpoint. A `col-md-6` element takes half the row on tablets and up, but full width on phones.

4. Components

Navbar

```
<nav class="navbar navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="/">My Dashboard</a>
        <ul class="navbar-nav">
            <li class="nav-item"><a class="nav-link" href="/dashboard">Dashboard</a></li>
            <li class="nav-item"><a class="nav-link" href="/products">Products</a></li>
            <li class="nav-item"><a class="nav-link" href="/settings">Settings</a></li>
        </ul>
    </div>
</nav>
```

Cards

```
<div class="card">
    <div class="card-header">Monthly Revenue</div>
    <div class="card-body">
        <h2 class="card-title">$12,450</h2>
        <p class="card-text">Up 12% from last month</p>
    </div>
    <div class="card-footer text-muted">Updated 5 minutes ago</div>
</div>
```

Buttons

```
<button class="btn btn-primary">Save</button>
<button class="btn btn-secondary">Cancel</button>
<button class="btn btn-danger">Delete</button>
<button class="btn btn-success">Publish</button>
<button class="btn btn-outline-primary">Outlined</button>
<button class="btn btn-sm btn-primary">Small</button>
<button class="btn btn-lg btn-primary">Large</button>
```

Tables

```
<table class="table table-striped table-hover">
    <thead>
        <tr>
            <th>Name</th>
            <th>Category</th>
            <th>Price</th>
            <th>Status</th>
```

```
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Wireless Keyboard</td>
            <td>Electronics</td>
            <td>$79.99</td>
            <td><span class="badge bg-success">In Stock</span></td>
        </tr>
        <tr>
            <td>Standing Desk</td>
            <td>Furniture</td>
            <td>$549.99</td>
            <td><span class="badge bg-danger">Out of Stock</span></td>
        </tr>
    </tbody>
</table>
```

Alerts

```
<div class="alert alert-success">Product created successfully!</div>
<div class="alert alert-danger">Failed to save changes.</div>
<div class="alert alert-warning">Your trial expires in 3 days.</div>
<div class="alert alert-info">A new version is available.</div>
```

Modals

```
<button class="btn btn-primary" data-toggle="modal" data-target="#confirmModal">Delete Product</button>

<div class="modal" id="confirmModal">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Confirm Delete</h5>
                <button class="btn-close" data-dismiss="modal"></button>
            </div>
            <div class="modal-body">
                <p>Are you sure you want to delete this product? This action cannot be undone.</p>
            </div>
            <div class="modal-footer">
                <button class="btn btn-secondary" data-dismiss="modal">Cancel</button>
                <button class="btn btn-danger">Delete</button>
            </div>
        </div>
    </div>
</div>
```

Forms

```
<form>
    <div class="form-group">
        <label for="name" class="form-label">Product Name</label>
        <input type="text" class="form-control" id="name" placeholder="Enter product name">
    </div>

    <div class="form-group">
        <label for="category" class="form-label">Category</label>
        <select class="form-control" id="category">
            <option value="">Select a category</option>
            <option value="electronics">Electronics</option>
            <option value="furniture">Furniture</option>
        </select>
    </div>
```

```
    <div class="form-group">
        <label for="description" class="form-label">Description</label>
        <textarea class="form-control" id="description" rows="4"></textarea>
    </div>

    <div class="form-check">
        <input type="checkbox" class="form-check-input" id="featured">
        <label class="form-check-label" for="featured">Featured product</label>
    </div>

// ...
```

5. SCSS Customization

The default tina4css is ready to use, but if you want to customize colors, fonts, or spacing, edit the SCSS source:

Edit `src/public/scss/tina4.scss`:

```
// Override variables before importing the framework
$primary: #2d6a4f;
$secondary: #52b788;
$dark: #1b4332;
$font-family-base: 'Inter', sans-serif;
$border-radius: 8px;

// Import the framework
@import 'tina4-base';
```

Compile SCSS to CSS:

```
tina4 scss
```

```
Compiling SCSS...
  src/public/scss/tina4.scss -> src/public/css/tina4.css
Done (0.12s)
```

The compiled CSS replaces the default `tina4.css`. Your custom colors and fonts are now active across the entire application.

Live SCSS Compilation

During development, run SCSS compilation in watch mode:

```
tina4 scss --watch
```

Every time you save a `.scss` file, it recompiles automatically. Combined with Tina4's live reload, you see changes in the browser within a second.

6. frond.js -- The JavaScript Helper

`frond.js` is a lightweight JavaScript library that ships with Tina4. It handles common frontend tasks without jQuery or any other dependency.

AJAX Requests

```
// GET request
frond.get("/api/products", function (data) {
```

```
    console.log("Products:", data);
});

// POST request
frond.post("/api/products", {
    name: "New Product",
    price: 29.99
}, function (data) {
    console.log("Created:", data);
});

// PUT request
frond.put("/api/products/1", {
    name: "Updated Product"
}, function (data) {
    console.log("Updated:", data);
});

// DELETE request
frond.delete("/api/products/1", function (data) {
    console.log("Deleted:", data);
});
```

Form Submission via AJAX

```
<form id="product-form" data-frond-submit="/api/products" data-frond-method="POST">
    <input type="text" name="name" placeholder="Product name">
    <input type="number" name="price" placeholder="Price">
    <button type="submit">Create</button>
</form>

<script src="/js/frond.js"></script>
<script>
    frond.onFormSuccess("product-form", function (data) {
        alert("Product created: " + data.name);
    });

    frond.onFormError("product-form", function (error) {
        alert("Error: " + error.message);
    });
</script>
```

The `data-frond-submit` attribute tells frond.js to intercept the form submission and send it as an AJAX request. No page reload.

Token Management

frond.js automatically includes the JWT token in all requests:

```
// Store the token (usually after login)
frond.setToken("eyJhbGciOiJIUzI1NiIs...");

// All subsequent requests automatically include:
// Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
frond.get("/api/profile", function (data) {
    console.log("Profile:", data);
});

// Clear the token (logout)
frond.clearToken();
```

WebSocket (Covered in Chapter 12)

```
const ws = frond.ws("/ws/chat/general");
```

```
ws.on("message", function (data) {
    console.log("Message:", JSON.parse(data));
});
```

7. Building an Admin Dashboard

Let us build a complete admin dashboard. This is the kind of page that powers the backend of every web application.

Base Template

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Dashboard{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
</head>
<body>
    <nav class="navbar navbar-dark bg-dark">
        <div class="container-fluid">
            <a class="navbar-brand" href="/admin">TaskFlow Admin</a>
            <ul class="navbar-nav">
                <li class="nav-item"><a class="nav-link" href="/admin">Dashboard</a></li>
                <li class="nav-item"><a class="nav-link" href="/admin/products">Products</a></li>
                <li class="nav-item"><a class="nav-link" href="/admin/users">Users</a></li>
                <li class="nav-item">
                    <button class="btn btn-sm btn-outline-light" onclick="toggleDarkMode()">Dark Mode</button>
                </li>
            </ul>
        </div>
    </nav>

    <div class="container-fluid mt-4">
        <div class="row">
// ...
```

Dashboard Page

Create `src/templates/dashboard.html`:

```
{% extends "base.html" %}

{% block title %}Dashboard - Admin{% endblock %}

{% block content %}
    <h1>Dashboard</h1>

    <div class="row mb-4">
        <div class="col-md-3">
            <div class="card">
                <div class="card-body">
                    <h6 class="card-subtitle text-muted">Total Products</h6>
                    <h2 class="card-title">{{ stats.total_products }}</h2>
                </div>
            </div>
        </div>
        <div class="col-md-3">
```

```
            <div class="card">
                <div class="card-body">
                    <h6 class="card-subtitle text-muted">Total Orders</h6>
                    <h2 class="card-title">{{ stats.total_orders }}</h2>
                </div>
            </div>
        </div>
        <div class="col-md-3">
// ...
```

Dashboard Route

Create `src/routes/admin.py`:

```
from tina4_python.core.router import get, template

@get("/admin")
async def admin_dashboard(request, response):
    stats = {
        "total_products": 156,
        "total_orders": 1243,
        "revenue": "24,580",
        "active_users": 89
    }

    recent_orders = [
        {"id": 1042, "customer": "Alice Johnson", "amount": "129.99", "status": "Shipped", "badge": "success"
        {"id": 1041, "customer": "Bob Smith", "amount": "549.99", "status": "Processing", "badge": "warning"},
        {"id": 1040, "customer": "Carol White", "amount": "79.99", "status": "Delivered", "badge": "info"},
        {"id": 1039, "customer": "Dave Brown", "amount": "34.99", "status": "Cancelled", "badge": "danger"},
    ]

    return response(template("dashboard.html", stats=stats, recent_orders=recent_orders))
```

Start the server and visit `http://localhost:7145/admin`. You have a complete admin dashboard with a sidebar, stats cards, a data table, and quick action buttons.

8. Dark Mode

tina4css supports dark mode via the `data-theme` attribute on the `<html>` element:

```
<!-- Light mode (default) -->
<html data-theme="light">

<!-- Dark mode -->
<html data-theme="dark">
```

The toggle function from the base template handles switching:

```
function toggleDarkMode() {
    const html = document.documentElement;
    const current = html.getAttribute("data-theme");
    html.setAttribute("data-theme", current === "dark" ? "light" : "dark");
    localStorage.setItem("theme", current === "dark" ? "light" : "dark");
}
```

Dark mode transforms every surface. Backgrounds darken. Text colors invert. Borders shift. Cards, tables, buttons -- all adjust contrast. Zero CSS rules required from you.

Respecting System Preference

To match the user's operating system dark mode setting:

```
// Check if user prefers dark mode
if (window.matchMedia && window.matchMedia('(prefers-color-scheme: dark)').matches) {
    document.documentElement.setAttribute("data-theme", "dark");
}

// Listen for changes
window.matchMedia('(prefers-color-scheme: dark)').addEventListener('change', function (e) {
    document.documentElement.setAttribute("data-theme", e.matches ? "dark" : "light");
});
```

9. Responsive Design

tina4css is mobile-first. Components adapt to screen size automatically. Here are the key patterns:

Responsive Sidebar

On mobile, the sidebar collapses into a hamburger menu:

```
<button class="btn btn-dark d-md-none" onclick="toggleSidebar()">Menu</button>

<div id="sidebar" class="d-none d-md-block col-md-2">
    <!-- sidebar content -->
</div>

<script>
function toggleSidebar() {
    const sidebar = document.getElementById("sidebar");
    sidebar.classList.toggle("d-none");
}
</script>
```

Responsive Tables

Tables scroll horizontally on small screens:

```
<div class="table-responsive">
    <table class="table">
        <!-- table content -->
    </table>
</div>
```

Hiding Elements by Screen Size

```
<div class="d-none d-md-block">Only visible on tablet and up</div>
<div class="d-md-none">Only visible on mobile</div>
```

10. Exercise: Build an Admin Dashboard

Build a complete admin dashboard for a product management system.

Requirements

1. Create a base template with:

• A dark navbar with the app name and navigation links

• A sidebar with menu items (Dashboard, Products, Orders, Settings)

• A main content area

• Dark mode toggle that persists across page loads

2. Create a dashboard page at `GET /admin` with:

• Four stat cards (Products, Orders, Revenue, Users)

• A table showing recent orders with status badges

• Quick action buttons

3. Create a product list page at `GET /admin/products` with:

• A table of products with name, category, price, and stock status

• An "Add Product" button

• Search/filter by category

4. Use tina4css classes throughout (no custom CSS needed)

Test by:

1. Visit `http://localhost:7145/admin` -- you should see the dashboard with stats and orders

2. Click "Dark Mode" -- the entire page should switch to dark theme

3. Refresh the page -- dark mode should persist

4. Resize the browser to mobile width -- the sidebar should collapse

5. Visit `http://localhost:7145/admin/products` -- you should see the product table

11. Solution

The base template and dashboard route are shown in sections 7 and above. Here is the product list page:

Create `src/templates/products.html`:

```
{% extends "base.html" %}

{% block title %}Products - Admin{% endblock %}

{% block content %}
    <div class="d-flex justify-content-between align-items-center mb-4">
        <h1>Products</h1>
        <a href="/admin/products/new" class="btn btn-primary">Add Product</a>
    </div>

    <div class="card mb-4">
        <div class="card-body">
            <form class="row g-3" method="GET" action="/admin/products">
                <div class="col-md-4">
                    <input type="text" class="form-control" name="search"
                           placeholder="Search products..." value="{{ search }}">
                </div>
                <div class="col-md-3">
                    <select class="form-control" name="category">
                        <option value="">All Categories</option>
                        {% for cat in categories %}
                        <option value="{{ cat }}" {% if cat == selected_category %}selected{% endif %}>{{ cat
```

```
                        {% endfor %}
                    </select>
                </div>
// ...
```

Add the route:

```
@get("/admin/products")
async def admin_products(request, response):
    search = request.query.get("search", "")
    selected_category = request.query.get("category", "")

    products = [
        {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": "79.99", "in_stock": True},
        {"id": 2, "name": "Standing Desk", "category": "Furniture", "price": "549.99", "in_stock": True},
        {"id": 3, "name": "Coffee Grinder", "category": "Kitchen", "price": "49.99", "in_stock": False},
        {"id": 4, "name": "Yoga Mat", "category": "Fitness", "price": "29.99", "in_stock": True},
        {"id": 5, "name": "USB-C Hub", "category": "Electronics", "price": "49.99", "in_stock": True},
    ]

    if selected_category:
        products = [p for p in products if p["category"] == selected_category]

    if search:
        products = [p for p in products if search.lower() in p["name"].lower()]

    categories = ["Electronics", "Furniture", "Kitchen", "Fitness"]

    return response(template("products.html",
        products=products,
        categories=categories,
        search=search,
// ...
```

12. Gotchas

1. CSS Not Loading

Problem: The page looks unstyled -- no colors, no layout, just plain text.

Cause: The path to `tina4.css` is wrong. The file is in `src/public/css/` but you are linking to `/css/tina4.css`.

Fix: Tina4 serves everything in `src/public/` as static files from the root path. Link to `/css/tina4.css` (not `/src/public/css/tina4.css`). Verify the file exists at `src/public/css/tina4.css`.

2. frond.js Functions Not Found

Problem: `frond.get is not a function` or `frond is not defined` in the browser console.

Cause: The `frond.js` script tag is missing or placed after the code that uses it.

Fix: Include `<script src="/js/frond.js"></script>` before any script that calls `frond.*`. Put it at the bottom of the body, just before your custom scripts.

3. Dark Mode Flickers on Page Load

Problem: The page loads in light mode and then flashes to dark mode.

Cause: The dark mode JavaScript runs after the page is rendered. The browser paints the light theme first, then switches.

Fix: Add the theme detection script in the `<head>` (before the body renders):

```
<head>
    <script>
        const t = localStorage.getItem("theme");
        if (t) document.documentElement.setAttribute("data-theme", t);
    </script>
</head>
```

4. SCSS Not Compiling

Problem: You edited `tina4.scss` but the CSS did not change.

Cause: SCSS does not compile automatically. You need to run `tina4 scss` or use `--watch` mode.

Fix: Run `tina4 scss --watch` during development. For production builds, run `tina4 scss` as part of your build process.

5. Modal Does Not Open

Problem: Clicking the button does nothing -- the modal stays hidden.

Cause: The `data-toggle` and `data-target` attributes require frond.js to be loaded. Without it, there is no JavaScript to handle the modal toggle.

Fix: Ensure `frond.js` is loaded. Verify the `data-target` matches the modal's `id` exactly (including the `#` prefix).

6. Grid Columns Do Not Stack on Mobile

Problem: Columns stay side-by-side on phone screens instead of stacking vertically.

Cause: You used `col-4` instead of `col-md-4`. The `col-4` class applies at all screen sizes, so the columns always take one-third width.

Fix: Use responsive prefixes: `col-md-4` means "one-third on medium screens and up, full width on small screens." For mobile-first design, always use `col-md-*` or `col-lg-*`.

7. Static Files Return 404

Problem: CSS, JS, or image files return 404 Not Found.

Cause: The files are not in the `src/public/` directory, or the directory structure is wrong.

Fix: Static files must be in `src/public/`. The URL path maps directly to the file path within that directory. `/css/tina4.css` maps to `src/public/css/tina4.css`. Check file paths and ensure the `src/public/` directory exists.

Chapter 16: GraphQL

1. The Problem GraphQL Solves

Your mobile app needs a list of products. Each product carries a name, price, 20 image URLs, a full description, 15 review objects, and 8 fields you do not need. REST sends all of it -- 50KB of JSON when you needed 2KB. On a mobile connection, that waste stings.

Now your web dashboard needs the same products plus category, stock status, and supplier info. REST forces you to make three requests and stitch them together, or build a custom endpoint with query parameter parsing.

GraphQL ends both problems. The client asks for the fields it needs. The server returns those fields. One endpoint. One request. One response shaped to fit.

Tina4 includes a built-in GraphQL engine. No external packages. No Strawberry. No Ariadne. Part of the framework.

2. GraphQL vs REST -- A Quick Comparison

| Aspect | REST | GraphQL |

|--------|------|---------|

| Endpoints | One per resource (`/products`, `/users`) | One endpoint (`/graphql`) |

| Data shape | Server decides | Client decides |

| Over-fetching | Common (get all fields) | Never (get only requested fields) |

| Under-fetching | Common (multiple requests needed) | Never (get related data in one query) |

| Versioning | `/api/v1/`, `/api/v2/` | Schema evolves, no versioning needed |

| Caching | HTTP caching works naturally | Requires client-side cache management |

| Learning curve | Low | Moderate |

REST is still great for simple APIs. GraphQL shines when clients have diverse data needs -- mobile apps, dashboards, third-party integrations.

3. Enabling GraphQL

GraphQL is available by default in Tina4. The engine serves requests at `/graphql`. If you want to change the endpoint, set it in `.env`:

```
TINA4_GRAPHQL_ENDPOINT=/graphql
```

To verify it is working, start the server and send a test query:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __schema { queryType { name } } }"}'
```

```
{
  "data": {
```

```
    "__schema": {
      "queryType": {
        "name": "Query"
      }
    }
  }
}
```

If you see that response, GraphQL is running.

4. Defining a Schema

A GraphQL schema defines the types of data your API can return and the queries and mutations clients can execute.

Create `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

type Query {
    products: [Product!]!
    product(id: Int!): Product
}
```

This schema says:

• A `Product` has six fields. The `!` means the field is non-nullable.

• The `Query` type has two operations: `products` returns a list of products, and `product(id)` returns a single product (or null if not found).

Tina4 auto-discovers `.graphql` files in `src/graphql/`. You do not need to register them.

5. Writing Resolvers

A resolver is the function that runs when a query or mutation is executed. Resolvers live in `src/graphql/` as Python files.

Create `src/graphql/resolvers.py`:

```
from tina4_python.graphql import GraphQL

# Resolve the "products" query
@GraphQL.resolve("Query", "products")
async def resolve_products(root, args):
    product = Product()
    products = product.select("*", order_by="name ASC")
    return [p.to_dict() for p in products]


# Resolve the "product" query
@GraphQL.resolve("Query", "product")
```

```
async def resolve_product(root, args):
    product = Product()
    product.load(args["id"])

    if not product.id:
        return None

    return product.to_dict()
```

The `@GraphQL.resolve()` decorator takes two arguments:

1. **Type name** -- The GraphQL type this resolver belongs to (`Query`, `Mutation`, or a custom type).

2. **Field name** -- The field within the type this resolver handles.

The resolver function receives `root` (the parent value, if any) and `args` (the query arguments).

Testing the Queries

List all products:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ products { id name price inStock } }"}'
```

```
{
  "data": {
    "products": [
      {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "inStock": true},
      {"id": 2, "name": "USB-C Hub", "price": 49.99, "inStock": true},
      {"id": 3, "name": "Standing Desk", "price": 549.99, "inStock": false}
    ]
  }
}
```

Notice: the response only contains the four fields we asked for (`id`, `name`, `price`, `inStock`), not `category` or `createdAt`. That is GraphQL in action.

Get a single product with all fields:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 1) { id name category price inStock createdAt } }"}'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "category": "Electronics",
      "price": 79.99,
      "inStock": true,
      "createdAt": "2026-03-22 14:30:00"
    }
  }
}
```

Request a product that does not exist:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 999) { id name } }"}'
```

```
{
  "data": {
```

```
    "product": null
  }
}
```

6. Mutations

Mutations are how clients create, update, or delete data. They are defined in the schema alongside queries.

Update `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

input ProductInput {
    name: String!
    category: String
    price: Float!
    inStock: Boolean
}

input ProductUpdateInput {
    name: String
    category: String
    price: Float
    inStock: Boolean
}

type DeleteResult {
    success: Boolean!
// ...
```

Key concepts:

• **`input` types** define the shape of data the client sends. They are like regular types but used for arguments.

• **`Mutation` type** lists all write operations. By convention, mutations are named with verbs: `createProduct`, `updateProduct`, `deleteProduct`.

• Fields in `ProductUpdateInput` are all optional (no `!`) because the client may only want to update one field.

Mutation Resolvers

Add to `src/graphql/resolvers.py`:

```
@GraphQL.resolve("Mutation", "createProduct")
async def resolve_create_product(root, args):
    input_data = args["input"]

    product = Product()
    product.name = input_data["name"]
    product.category = input_data.get("category", "Uncategorized")
```

```
    product.price = float(input_data["price"])
    product.in_stock = bool(input_data.get("inStock", True))
    product.save()

    return product.to_dict()


@GraphQL.resolve("Mutation", "updateProduct")
async def resolve_update_product(root, args):
    product = Product()
    product.load(args["id"])

    if not product.id:
        return None

    input_data = args["input"]
    if "name" in input_data:
        product.name = input_data["name"]
// ...
```

Testing Mutations

Create a product:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { createProduct(input: { name: \"Desk Lamp\", category: \"Office\", price: 39.99 }) { i
  }'
```

```
{
  "data": {
    "createProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 39.99
    }
  }
}
```

Update a product:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { updateProduct(id: 4, input: { price: 44.99, inStock: false }) { id name price inStock
  }'
```

```
{
  "data": {
    "updateProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 44.99,
      "inStock": false
    }
  }
}
```

Delete a product:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { deleteProduct(id: 4) { success message } }"
```

```
  }'
```

```
{
  "data": {
    "deleteProduct": {
      "success": true,
      "message": "Product deleted"
    }
  }
}
```

7. Nested Types and Relationships

The real power of GraphQL: traversing relationships in a single query. Authors, posts, comments -- all in one request.

Update `src/graphql/schema.graphql`:

```
type User {
    id: Int!
    name: String!
    email: String!
    posts: [Post!]!
}

type Post {
    id: Int!
    title: String!
    body: String!
    published: Boolean!
    author: User!
    comments: [Comment!]!
    commentCount: Int!
}

type Comment {
    id: Int!
    authorName: String!
    body: String!
    post: Post!
}

type Query {
// ...
```

Add resolvers for the nested types:

```
@GraphQL.resolve("Query", "posts")
async def resolve_posts(root, args):
    post = Post()
    posts = post.select("*", "published = :pub", {"pub": 1}, order_by="created_at DESC")
    return [p.to_dict() for p in posts]


@GraphQL.resolve("Query", "post")
async def resolve_post(root, args):
    post = Post()
    post.load(args["id"])
    return post.to_dict() if post.id else None


# Resolve Post.author (nested field)
```

```
@GraphQL.resolve("Post", "author")
async def resolve_post_author(post, args):
    user = User()
    user.load(post["user_id"])
    return user.to_dict()


# Resolve Post.comments (nested field)
@GraphQL.resolve("Post", "comments")
async def resolve_post_comments(post, args):
// ...
```

Now clients can query across relationships in a single request:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "{ posts { id title author { name email } comments { authorName body } commentCount } }"
  }'
```

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {
          "name": "Alice",
          "email": "alice@example.com"
        },
        "comments": [
          {"authorName": "Bob", "body": "Great post!"}
        ],
        "commentCount": 1
      }
    ]
  }
}
```

One request. Exactly the fields the client needs. No over-fetching. No under-fetching.

8. Auto-Generating Schema from ORM Models

If your ORM models have `auto_crud = True`, Tina4 can auto-generate GraphQL types and basic CRUD resolvers for them. Enable this in `.env`:

```
TINA4_GRAPHQL_AUTO_SCHEMA=true
```

With this setting, every ORM model with `auto_crud = True` automatically gets:

• A GraphQL type with fields matching the model properties

• A query to list all records (e.g., `products`)

• A query to get one by ID (e.g., `product(id: Int!)`)

• A mutation to create (e.g., `createProduct(input: ProductInput!)`)

• A mutation to update (e.g., `updateProduct(id: Int!, input: ProductUpdateInput!)`)

• A mutation to delete (e.g., `deleteProduct(id: Int!)`)

You can still define custom resolvers that override the auto-generated ones. Custom resolvers always take precedence.

9. The GraphiQL Playground

When `TINA4_DEBUG=true`, Tina4 serves a GraphiQL interactive playground at:

```
http://localhost:7145/graphql/playground
```

GraphiQL gives you:

• A query editor with syntax highlighting and auto-completion

• A documentation explorer showing all types, queries, and mutations

• A results panel showing the response

• Query history

This is the fastest way to explore and test your GraphQL API during development. Type a query on the left, click the play button, and see the results on the right. The documentation explorer on the right sidebar shows every type, field, and argument available in your schema.

GraphiQL is only available when `TINA4_DEBUG=true`. In production, it is disabled automatically.

10. Query Variables

For production use, clients should send query variables separately from the query string. This prevents injection and allows query caching.

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "query GetProduct($id: Int!) { product(id: $id) { id name price } }",
    "variables": {"id": 1}
  }'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "price": 79.99
    }
  }
}
```

The query uses `$id` as a placeholder, and the actual value comes from the `variables` object. This is safer and more efficient than string interpolation.

11. Exercise: Build a GraphQL API for a Blog

Build a complete GraphQL API for a blog with posts, authors, and comments.

Requirements

1. **Schema** -- Define types for `User`, `Post`, and `Comment` with the following fields:

• `User`: id, name, email, posts (nested)

• `Post`: id, title, body, published, author (nested), comments (nested), commentCount (computed)

• `Comment`: id, authorName, body, createdAt

2. **Queries**:

• `posts` -- List all published posts

• `post(id: Int!)` -- Get a single post by ID

• `user(id: Int!)` -- Get a user with their posts

3. **Mutations**:

• `createPost(userId: Int!, title: String!, body: String!, published: Boolean)` -- Create a new post

• `addComment(postId: Int!, authorName: String!, body: String!)` -- Add a comment to a post

4. Use the ORM models from Chapter 6 (User, Post, Comment).

Test with these queries:

```
# List published posts with author names
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ posts { id title author { name } commentCount } }"}'

# Get a specific post with all comments
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ post(id: 1) { title body author { name email } comments { authorName body } } }"}'

# Create a post
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { createPost(userId: 1, title: \"GraphQL is great\", body: \"Here is why...\", publi

# Add a comment
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { addComment(postId: 1, authorName: \"Carol\", body: \"Nice article!\") { id authorN
```

12. Solution

Schema

Create `src/graphql/blog_schema.graphql`:

```
type User {
    id: Int!
    name: String!
    email: String!
    posts: [Post!]!
}

type Post {
    id: Int!
    title: String!
```

```
    body: String!
    published: Boolean!
    author: User!
    comments: [Comment!]!
    commentCount: Int!
    createdAt: String
}

type Comment {
    id: Int!
    authorName: String!
    body: String!
    createdAt: String
}

// ...
```

Resolvers

Create `src/graphql/blog_resolvers.py`:

```
from tina4_python.graphql import GraphQL


@GraphQL.resolve("Query", "posts")
async def resolve_posts(root, args):
    post = Post()
    posts = post.select("*", "published = :pub", {"pub": 1}, order_by="created_at DESC")
    return [p.to_dict() for p in posts]


@GraphQL.resolve("Query", "post")
async def resolve_post(root, args):
    post = Post()
    post.load(args["id"])
    return post.to_dict() if post.id else None


@GraphQL.resolve("Query", "user")
async def resolve_user(root, args):
    user = User()
    user.load(args["id"])
    return user.to_dict() if user.id else None


@GraphQL.resolve("Post", "author")
// ...
```

Expected output for `{ posts { id title author { name } commentCount } }`:

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {"name": "Alice"},
        "commentCount": 1
      },
      {
        "id": 2,
        "title": "GraphQL is great",
        "author": {"name": "Alice"},
        "commentCount": 0
```

```
      }
    ]
  }
}
```

13. Gotchas

1. Schema File Not Found

Problem: Queries return errors about unknown types or fields.

Cause: The `.graphql` file is not in `src/graphql/`, or has a syntax error that prevents parsing.

Fix: Make sure schema files are in `src/graphql/` and end with `.graphql`. Check for syntax errors -- a missing `!` or unmatched brace will cause the entire schema to fail silently.

2. Resolver Not Called

Problem: A query returns `null` even though data exists.

Cause: The resolver is not registered, or the type/field names do not match the schema exactly. GraphQL is case-sensitive.

Fix: Verify that `@GraphQL.resolve("Query", "products")` matches `type Query { products: ... }` exactly. Check capitalization.

3. Nested Resolver Returns Wrong Data

Problem: `post.author` returns the entire post instead of the user.

Cause: The nested resolver receives the parent object as its first argument (`post`), not a fresh model. If you return the parent instead of loading the related record, you get the wrong data.

Fix: In a nested resolver, always load the related record from the database using the foreign key from the parent:

```
@GraphQL.resolve("Post", "author")
async def resolve_post_author(post, args):
    user = User()
    user.load(post["user_id"])  # Use the foreign key from the parent
    return user.to_dict()
```

4. Mutation Input Not Parsed

Problem: `args["input"]` is `None` inside a mutation resolver.

Cause: The mutation signature in the schema uses an `input` type, but the client query passes arguments directly instead of wrapping them in an `input` object.

Fix: Match the query to the schema. If the schema says `createProduct(input: ProductInput!)`, the client must send `createProduct(input: { name: "Widget", price: 9.99 })`, not `createProduct(name: "Widget", price: 9.99)`.

5. N+1 Query Problem

Problem: Loading 50 posts with their authors generates 51 database queries (1 for posts + 50 for authors).

Cause: Each nested resolver runs independently. When you resolve `author` for each post, that is one query per post.

Fix: Use data loader patterns or batch loading. For simple cases, you can pre-load all related records in the parent resolver and pass them down. For complex cases, consider using the REST API with eager loading instead of GraphQL for that particular endpoint.

6. GraphQL Playground Returns 404

Problem: `/graphql/playground` returns a 404 error.

Cause: `TINA4_DEBUG` is set to `false`. The playground is only available in debug mode.

Fix: Set `TINA4_DEBUG=true` in your `.env` file. The playground is intentionally disabled in production.

7. Type Mismatch Between Schema and Resolver

Problem: A field declared as `Int!` in the schema receives a string from the resolver, causing a type error.

Cause: Python's dynamic typing means your resolver might return `"42"` (a string) for an `Int!` field. GraphQL rejects it.

Fix: Cast values to the correct type in your resolvers. Use `int()`, `float()`, `bool()` explicitly:

```
return {
    "id": int(product.id),
    "price": float(product.price),
    "inStock": bool(product.in_stock)
}
```

The `to_dict()` method handles this automatically for model properties with type declarations, but computed or derived fields need manual casting.

Chapter 17: Testing

1. Why Tests Matter More Than You Think

Friday afternoon. Your client reports a critical bug in production. You fix it -- one line of code. But did that fix break something else? 47 routes. 12 ORM models. 3 middleware functions. Clicking through every page: an hour. Running the test suite: 2 seconds.

```
tina4 test

Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product
    [PASS] test_update_product
    [PASS] test_delete_product
    [PASS] test_list_products_with_filter

  AuthTest
    [PASS] test_login_with_valid_credentials
    [PASS] test_login_with_invalid_password
    [PASS] test_protected_route_without_token
    [PASS] test_protected_route_with_valid_token

  9 tests, 9 passed, 0 failed (0.34s)
```

Everything still works. You deploy with confidence. Weekend intact.

Tina4 includes an inline testing framework. No external packages. No configuration. Write a test. Run it. Done.

2. Your First Test

Tests live in the `tests/` directory. Every `.py` file in that directory is auto-discovered when you run `tina4 test`.

Create `tests/test_basic.py`:

```
from tina4_python.test import Test, assert_equal, assert_true

class BasicTest(Test):

    def test_addition(self):
        assert_equal(2 + 2, 4, "Basic addition should work")

    def test_string_contains(self):
        greeting = "Hello, World!"
        assert_true("World" in greeting, "Greeting should contain 'World'")

    def test_array_length(self):
        items = [1, 2, 3, 4, 5]
        assert_equal(len(items), 5, "List should have 5 items")
```

Run it:

```
tina4 test

Running tests...
```

```
BasicTest
  [PASS] test_addition
  [PASS] test_string_contains
  [PASS] test_array_length

3 tests, 3 passed, 0 failed (0.02s)
```

How It Works

1. Your test class extends `Test`.

2. Every method that starts with `test` is a test case. The method name is converted to a readable label: `test_addition` stays as `test_addition`.

3. Inside each test, you call assertion functions to verify behavior.

4. If all assertions pass, the test passes. If any assertion fails, the test fails and you see the failure message.

3. Assertion Methods

Tina4's testing framework provides these assertion functions:

assert_equal(actual, expected, message)

Checks that two values are equal.

```
assert_equal(4, 4, "Should be equal")              # PASS
assert_equal("hello", "hello", "Strings match")    # PASS
assert_equal(4, 5, "Not equal")                     # FAIL
```

assert_true(value, message)

Checks that a value is truthy.

```
assert_true(True, "Should be true")                # PASS
assert_true(1, "1 is truthy")                      # PASS
assert_true("yes", "Non-empty string is truthy")   # PASS
assert_true(False, "This fails")                   # FAIL
assert_true(0, "Zero is falsy")                    # FAIL
```

assert_false(value, message)

Checks that a value is falsy.

```
assert_false(False, "Should be false")             # PASS
assert_false(0, "Zero is falsy")                   # PASS
assert_false("", "Empty string is falsy")          # PASS
assert_false(True, "This fails")                   # FAIL
```

assert_raises(callable, exception_class, message)

Checks that a function raises a specific exception.

```
assert_raises(
    lambda: int("not-a-number"),
    ValueError,
    "Should raise ValueError"
)

assert_raises(
    lambda: 10 / 0,
```

```
    ZeroDivisionError,
    "Should raise on division by zero"
)
```

assert_not_equal(actual, expected, message)

Checks that two values are not equal.

```
assert_not_equal("hello", "world", "Strings differ")  # PASS
assert_not_equal(4, 4, "Same values")                  # FAIL
```

assert_none(value, message)

Checks that a value is None.

```
assert_none(None, "Should be None")     # PASS
assert_none("hello", "Not None")        # FAIL
```

assert_not_none(value, message)

Checks that a value is not None.

```
assert_not_none("hello", "Has value")   # PASS
assert_not_none(None, "Is None")        # FAIL
```

4. Testing ORM Models

Let us test a Product model. The test creates records, loads them, updates them, and deletes them.

Create `tests/test_product.py`:

```
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none

class ProductTest(Test):

    def test_create_product(self):
        product = Product()
        product.name = "Test Widget"
        product.category = "Testing"
        product.price = 19.99
        product.in_stock = True
        product.save()

        assert_not_none(product.id, "Product should have an ID after save")
        assert_true(product.id > 0, "Product ID should be positive")

    def test_load_product(self):
        product = Product()
        product.name = "Load Test Widget"
        product.category = "Testing"
        product.price = 29.99
        product.save()

        loaded = Product()
        loaded.load(product.id)

// ...
```

Run it:

```
tina4 test
```

```
Running tests...
```

```
  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product
    [PASS] test_update_product
    [PASS] test_delete_product
    [PASS] test_select_with_filter

  5 tests, 5 passed, 0 failed (0.18s)
```

Test Database

By default, `tina4 test` uses a separate test database so your development data is not affected. The test database is created at `data/test.db` (SQLite) and is reset before each test run. If you want to use a different database for tests, set it in `.env`:

```
TINA4_TEST_DATABASE_URL=sqlite:///data/test.db
```

5. Testing Routes

Tina4 provides a test client for making HTTP requests to your routes without starting a server.

Create `tests/test_routes.py`:

```
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none
import json

class RouteTest(Test):

    def test_health_endpoint(self):
        resp = self.get("/health")

        assert_equal(resp.status_code, 200, "Health check should return 200")

        body = json.loads(resp.body)
        assert_equal(body["status"], "ok", "Status should be 'ok'")
        assert_not_none(body.get("version"), "Should include version")

    def test_get_products(self):
        resp = self.get("/api/products")

        assert_equal(resp.status_code, 200, "Should return 200")

        body = json.loads(resp.body)
        assert_true("data" in body or "products" in body, "Should contain product data")

    def test_create_product(self):
        resp = self.post("/api/products", {
            "name": "Route Test Product",
// ...
```

Test Client Methods

The test client provides methods for all HTTP verbs:

```
# GET request
resp = self.get("/api/products")

# GET with query parameters
resp = self.get("/api/products?category=Electronics&page=2")

# POST with JSON body
```

```
resp = self.post("/api/products", {"name": "Widget", "price": 9.99})

# PUT with JSON body
resp = self.put("/api/products/1", {"name": "Updated Widget"})

# PATCH with JSON body
resp = self.patch("/api/products/1", {"price": 12.99})

# DELETE
resp = self.delete("/api/products/1")

# Request with custom headers
resp = self.get("/api/profile", headers={
    "Authorization": "Bearer eyJhbGciOiJIUzI1NiIs..."
})
```

Response Object

The response object has these properties:

```
resp.status_code   # HTTP status code (200, 201, 404, etc.)
resp.body          # Response body as a string
resp.headers       # Response headers as a dict
resp.content_type  # Content-Type header value
```

6. Testing Authentication

Create `tests/test_auth.py`:

```
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none
import json

class AuthTest(Test):

    def test_login_with_valid_credentials(self):
        resp = self.post("/api/auth/login", {
            "email": "admin@example.com",
            "password": "correct-password"
        })

        assert_equal(resp.status_code, 200, "Login should succeed")

        body = json.loads(resp.body)
        assert_not_none(body.get("token"), "Should return a JWT token")
        assert_true(len(body["token"]) > 50, "Token should be a substantial string")

    def test_login_with_invalid_password(self):
        resp = self.post("/api/auth/login", {
            "email": "admin@example.com",
            "password": "wrong-password"
        })

        assert_equal(resp.status_code, 401, "Should reject invalid password")

// ...
```

7. Setup and Teardown

Use `set_up()` and `tear_down()` methods to run code before and after each test:

```
from tina4_python.test import Test, assert_equal, assert_not_none
import time

class UserTest(Test):

    def set_up(self):
        # Runs before each test
        user = User()
        user.name = "Test User"
        user.email = f"test-{int(time.time())}@example.com"
        user.save()
        self.user_id = user.id

    def tear_down(self):
        # Runs after each test
        if self.user_id:
            user = User()
            user.load(self.user_id)
            if user.id:
                user.delete()

    def test_user_exists(self):
        user = User()
        user.load(self.user_id)
        assert_not_none(user.id, "User should exist")
// ...
```

`set_up()` runs before every test method, and `tear_down()` runs after every test method, regardless of whether the test passed or failed. This keeps tests isolated -- each test starts with a clean state.

8. Running Tests

Run All Tests

```
tina4 test
```

Run a Specific Test File

```
tina4 test --file tests/test_product.py
```

Run a Specific Test Method

```
tina4 test --file tests/test_product.py --method test_create_product
```

Verbose Output

```
tina4 test --verbose
```

```
Running tests...

  ProductTest
    [PASS] test_create_product (0.03s)
      assert_not_none: Product should have an ID after save
      assert_true: Product ID should be positive
    [PASS] test_load_product (0.02s)
      assert_equal: Name should match
      assert_equal: Category should match
      assert_equal: Price should match
    [PASS] test_update_product (0.04s)
      assert_equal: Name should be updated
      assert_equal: Price should be updated
    [PASS] test_delete_product (0.02s)
```

```
      assert_true: Deleted product should not be loadable
    [PASS] test_select_with_filter (0.05s)
      assert_true: Should find at least 2 FilterCat products
      assert_true: Should include Filter Test A
      assert_true: Should include Filter Test B

  5 tests, 5 passed, 0 failed (0.16s)
```

Verbose mode shows each assertion within each test, along with timing information.

Failed Test Output

When a test fails, you see exactly what went wrong:

```
  ProductTest
    [PASS] test_create_product
    [FAIL] test_load_product
      assert_equal FAILED: Name should match
        Expected: "Load Test Widget"
        Actual:   "Wrong Name"
        File: tests/test_product.py:34
    [PASS] test_update_product

  3 tests, 2 passed, 1 failed (0.12s)
```

The failure message shows the assertion that failed, what was expected, what was actually received, and the file and line number.

9. pytest Integration

If your team already uses pytest, Tina4 tests can run alongside it. Install pytest:

```
uv add --dev pytest
```

Create `pyproject.toml` test configuration:

```
[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = ["test_*.py"]
python_classes = ["*Test"]
python_functions = ["test_*"]
```

Tina4's `Test` class is compatible with pytest. Your test files work with both `tina4 test` and `pytest` without modification.

```
uv run pytest

========================= test session starts ==========================
collected 5 items

tests/test_product.py .....                                      [100%]

========================= 5 passed in 0.34s ============================
```

Code Coverage

With pytest, you can generate code coverage reports:

```
uv add --dev pytest-cov
uv run pytest --cov=src --cov-report=term

---------- coverage: ... ----------
```

```
Name                      Stmts   Miss  Cover
---------------------------------------------
src/routes/products.py       45      5    89%
src/orm/product.py           28      0   100%
src/middleware/auth.py        32      8    75%
---------------------------------------------
TOTAL                       105     13    88%
```

For HTML coverage reports:

```
uv run pytest --cov=src --cov-report=html
```

Open `htmlcov/index.html` in your browser to see a visual breakdown of which lines are covered by tests.

10. Testing Best Practices

Test One Thing Per Test

Each test method should verify one behavior. If it fails, you know exactly what broke.

```
# Good: each test verifies one thing
def test_create_product_returns_201(self):
    resp = self.post("/api/products", {"name": "Widget", "price": 9.99})
    assert_equal(resp.status_code, 201, "Should return 201")

def test_create_product_returns_created_product(self):
    resp = self.post("/api/products", {"name": "Widget", "price": 9.99})
    body = json.loads(resp.body)
    assert_equal(body["name"], "Widget", "Should return the product name")

# Avoid: testing multiple unrelated things
def test_everything(self):
    # Creates, reads, updates, deletes, checks auth, validates input...
    # When this fails, you don't know which part broke
    pass
```

Use Descriptive Assertion Messages

```
# Good: tells you what went wrong
assert_equal(user.name, "Alice", "User name should be 'Alice' after creation")

# Bad: unhelpful when it fails
assert_equal(user.name, "Alice", "Failed")
```

Isolate Tests

Each test should create its own data and clean up after itself. Never depend on data from another test or from the development database.

```
# Good: creates its own data
def test_delete_product(self):
    product = Product()
    product.name = "Temporary"
    product.price = 1.00
    product.save()

    product.delete()

    check = Product()
    check.load(product.id)
    assert_true(not check.id, "Product should be deleted")
```

```
# Bad: depends on data from another test or the dev database
def test_delete_product(self):
    product = Product()
    product.load(1)  # Assumes product with ID 1 exists
    product.delete()
```

11. Exercise: Test a User Model and Authentication Flow

Write a complete test suite for a User model and authentication system.

Requirements

Create `tests/test_user_model.py` with these tests:

1. **test_create_user** -- Create a user with name, email. Verify the user gets an ID and all fields are correct.

2. **test_duplicate_email** -- Try to create two users with the same email. Verify the second one raises an exception or returns an error.

3. **test_update_user** -- Create a user, update their name, reload and verify.

4. **test_delete_user** -- Create a user, delete them, verify they cannot be loaded.

5. **test_select_users** -- Create 3 users, query all, verify count is at least 3.

Create `tests/test_auth_flow.py` with these tests:

1. **test_register_new_user** -- POST to `/api/auth/register` with name, email, password. Verify 201 status.

2. **test_register_duplicate_email** -- Register the same email twice. Verify the second returns 409 Conflict.

3. **test_login_success** -- Login with correct credentials. Verify you get a JWT token.

4. **test_login_failure** -- Login with wrong password. Verify 401 status.

5. **test_access_protected_route** -- Use the token from login to access `/api/profile`. Verify 200 status and correct user data.

6. **test_access_with_expired_token** -- Use a manually crafted expired token. Verify 401 status.

Expected output:

```
tina4 test

Running tests...

  UserModelTest
    [PASS] test_create_user
    [PASS] test_duplicate_email
    [PASS] test_update_user
    [PASS] test_delete_user
    [PASS] test_select_users

  AuthFlowTest
    [PASS] test_register_new_user
    [PASS] test_register_duplicate_email
    [PASS] test_login_success
    [PASS] test_login_failure
    [PASS] test_access_protected_route
    [PASS] test_access_with_expired_token
```

```
  11 tests, 11 passed, 0 failed (0.52s)
```

12. Solution

tests/test_user_model.py

```
import uuid
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none, assert_raises

class UserModelTest(Test):

    def test_create_user(self):
        user = User()
        user.name = "Test User"
        user.email = f"testuser-{uuid.uuid4().hex[:8]}@example.com"
        user.save()

        assert_not_none(user.id, "User should have an ID after save")
        assert_equal(user.name, "Test User", "Name should match")
        assert_true("@example.com" in user.email, "Email should be set")

    def test_duplicate_email(self):
        email = f"duplicate-{uuid.uuid4().hex[:8]}@example.com"

        user1 = User()
        user1.name = "First User"
        user1.email = email
        user1.save()

        def create_duplicate():
            user2 = User()
// ...
```

tests/test_auth_flow.py

```
import uuid
import json
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none

class AuthFlowTest(Test):

    def set_up(self):
        self.test_email = f"auth-test-{uuid.uuid4().hex[:8]}@example.com"
        self.test_password = "SecurePassword123!"

    def test_register_new_user(self):
        resp = self.post("/api/auth/register", {
            "name": "Auth Test User",
            "email": self.test_email,
            "password": self.test_password
        })

        assert_equal(resp.status_code, 201, "Registration should return 201")

        body = json.loads(resp.body)
        assert_equal(body["name"], "Auth Test User", "Should return user name")
        assert_not_none(body.get("id"), "Should return user ID")

    def test_register_duplicate_email(self):
        email = f"dup-{uuid.uuid4().hex[:8]}@example.com"
// ...
```

13. Gotchas

1. Tests Run in Order

Problem: Test B depends on data created in Test A, but sometimes Test B fails.

Cause: While tests within a class run in the order they are defined, test classes may run in any order. If Test B depends on Test A being in a different class, this is fragile.

Fix: Each test should create its own data. Never depend on another test's side effects. Use `set_up()` to create the data each test needs.

2. Test Database vs Development Database

Problem: Running tests deletes your development data.

Cause: Tests are running against the same database as your development server.

Fix: Tina4 uses a separate test database by default (`data/test.db`). If your tests are modifying development data, check that `TINA4_TEST_DATABASE_URL` is set correctly in `.env`, or that you are running `tina4 test` (not executing test files manually with `python`).

3. Unique Constraint Failures

Problem: Tests fail with "UNIQUE constraint failed" on the email field.

Cause: Previous test runs left data in the test database, or two tests create records with the same email.

Fix: Use `uuid.uuid4().hex[:8]` or `int(time.time())` to generate unique values in tests:

```
user.email = f"test-{uuid.uuid4().hex[:8]}@example.com"
```

4. Test Method Not Discovered

Problem: You wrote a test method but it does not appear in the output.

Cause: The method name does not start with `test`. Tina4 only runs methods that begin with `test` (case-sensitive).

Fix: Rename the method to start with `test`: `test_create_product`, `test_login_failure`, etc.

5. Assertion Arguments Reversed

Problem: The failure message shows "Expected: 404, Actual: 200" but that seems backward.

Cause: You passed the arguments in the wrong order. `assert_equal(actual, expected)` -- actual comes first, expected comes second.

Fix: Follow the convention: `assert_equal(resp.status_code, 200, "message")`. The first argument is what you got, the second is what you expected.

6. Cannot Test Routes That Require Authentication Setup

Problem: Tests for protected routes fail because the authentication middleware expects a database table or JWT secret that does not exist in the test environment.

Cause: The test database is empty -- it does not have the users table or the JWT secret is not configured.

Fix: Use `set_up()` to create the necessary data:

```
def set_up(self):
    user = User()
    user.create_table()

    user.name = "Test Admin"
    user.email = "admin@test.com"
    user.password_hash = hash_password("password")
    user.save()
```

7. Tests Pass Locally but Fail in CI

Problem: All tests pass on your machine but fail in continuous integration.

Cause: The CI environment may have a different Python version, missing packages, or different filesystem permissions. Also, tests that depend on time or random values can be flaky.

Fix: Make tests deterministic. Avoid relying on the current time, filesystem state, or external services. If you must test time-dependent behavior, mock the clock or use fixed timestamps. Check that your CI environment matches your local Python version and has all required packages.

Chapter 18: Dev Tools

1. Debugging at 2am

2am. Production monitoring pings you -- a 500 error on checkout. You pull up the dev dashboard. Find the failing request in the request inspector. See the full stack trace with source code context. Line 47 of `src/routes/checkout.py` -- an `AttributeError` on the shipping address because the user skipped the form field. You add a None check. Push the fix. Go back to sleep. Total time: 30 seconds.

Tina4's dev tools are not an afterthought. They are built into the framework from day one. When `TINA4_DEBUG=true`, you get a development dashboard, an error overlay with source code, live reload, a request inspector, a SQL query runner, and more -- all without installing extra packages.

2. Enabling the Dev Dashboard

The dev dashboard is available when you enable the console in your `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=my-dev-token
```

Restart your server and navigate to:

```
http://localhost:7145/tina4/console
```

Enter your console token (`my-dev-token`) when prompted. You are now in the dev dashboard.

The console token guards the dashboard from unauthorized access. In development, any string works. In shared environments, use a strong random token. In production, set `TINA4_DEBUG=false` and the entire dashboard disappears.

3. Dashboard Overview

The dev dashboard has several sections, accessible from the navigation tabs at the top:

System Overview

The landing page shows at a glance:

• **Framework version** -- The installed Tina4 Python version

• **Python version** -- The running Python version and loaded packages

• **Uptime** -- How long the server has been running

• **Memory usage** -- Current and peak memory consumption

• **Database status** -- Connection status, database engine, file size (for SQLite)

• **Environment** -- Current `.env` variables (sensitive values are masked)

• **Project structure** -- Directory listing of your project with file counts

This is the first thing to check when something feels off. Is the database connected? Is the right Python version running? Are the environment variables loaded?

4. The Dev Toolbar

When you visit any HTML page in your application (like `/products` or `/admin`), a debug toolbar appears at the bottom of the page. It is a thin bar that expands when you click on it.

The toolbar shows:

| Field | What It Means |

|-------|--------------|

| **Request** | HTTP method and URL of the current request |

| **Status** | HTTP response status code |

| **Time** | Total request processing time in milliseconds |

| **DB** | Number of database queries executed and total query time |

| **Memory** | Python memory used for this request |

| **Template** | The template rendered and how long rendering took |

| **Session** | Session ID and session data summary |

| **Route** | Which route handler matched this request |

Click any section to expand it and see details. For example, clicking "DB" shows every SQL query that ran during the request, with the query text, parameters, execution time, and number of rows returned.

Disabling the Toolbar

The toolbar is automatically hidden when `TINA4_DEBUG=false`. You can also hide it for specific routes by returning a response with the `X-Debug-Toolbar: off` header:

```
from tina4_python.core.router import get

@get("/api/data")
async def api_data(request, response):
    return response({"data": "no toolbar"}, headers={
        "X-Debug-Toolbar": "off"
    })
```

This is useful for API endpoints that return JSON -- the toolbar is only meaningful for HTML pages.

5. Error Overlay

When an unhandled exception occurs, Tina4 does not show a generic "500 Internal Server Error" page. Instead, it shows a detailed error overlay with:

• **Exception type and message** -- What went wrong, in plain language

• **Stack trace** -- Every function call that led to the error, from the entry point to the crash

• **Source code** -- The actual Python code around the line that threw the exception, with the failing line highlighted

• **Request data** -- The HTTP method, URL, headers, body, and query parameters of the request that triggered the error

• **Environment** -- Relevant `.env` variables at the time of the error

Example Error

If you accidentally write:

```
@get("/api/users/{user_id}")
async def get_user(request, response):
    user_id = request.params["user_id"]
    user = get_user_from_db(user_id)
    return response({"name": user.name, "email": user.email})
```

And `get_user_from_db()` returns `None` for a missing user, the error overlay shows:

```
AttributeError: 'NoneType' object has no attribute 'name'

  File: src/routes/users.py, line 5

  3 ■ async def get_user(request, response):
  4 ■     user_id = request.params["user_id"]
  5 ■     user = get_user_from_db(user_id)
→ 6 ■     return response({"name": user.name, "email": user.email})
  7 ■

  Request: GET /api/users/999
  Headers: {"Accept": "application/json"}
```

You can see exactly what happened, where it happened, and what request triggered it.

Error Overlay in Production

When `TINA4_DEBUG=false`, the error overlay is disabled. Instead, users see a clean error page. The full error details are written to the log file (`logs/error.log`) so you can investigate later.

6. The Gallery

The gallery is a collection of ready-to-use code examples built into the dev dashboard. Each gallery item includes:

• A description of what it does

• The complete source code (routes, templates, models)

• A "Try It" button that installs the example into your project

Available gallery items include:

• **JWT Authentication** -- Complete login/register flow with token management

• **CRUD API** -- Full REST API with ORM model

• **File Upload** -- Multipart form handling with file storage

• **WebSocket Chat** -- Real-time chat with rooms

• **Email Contact Form** -- Form with Messenger integration

• **Dashboard Template** -- Admin dashboard with tina4css

Click "Try It" on any gallery item, and Tina4 creates the necessary files in your project. You can then modify them to fit your needs.

7. Live Reload

When `TINA4_DEBUG=true`, Tina4 watches your project files for changes and automatically reloads the server. Edit a route file, save it, and the browser refreshes with the new code -- no manual restart required.

```
uv run python app.py
```

```
  Tina4 Python v3.0.0
  HTTP server running at http://0.0.0.0:7145
  Live reload enabled -- watching for changes
```

Live reload watches:

• `src/routes/*.py` -- Route definitions

• `src/orm/*.py` -- ORM models

• `src/middleware/*.py` -- Middleware

• `src/templates/*.html` -- Templates (browser refresh only, no server restart)

• `.env` -- Environment variables

How It Works

Tina4 uses file system monitoring to detect changes. When a Python file changes, the server restarts automatically. When a template changes, only the browser refreshes (no server restart needed).

The reload happens in under a second. You edit code, switch to the browser, and the changes are already there.

8. Hot-Patching with jurigged

For even faster iteration, Tina4 supports hot-patching via jurigged. Hot-patching updates function definitions in the running server without restarting it. This means:

• No connection drops (WebSocket clients stay connected)

• No cache loss (in-memory caches stay warm)

• No session interruption

• Sub-second updates

Install jurigged:

```
uv add --dev jurigged
```

Start the server with hot-patching:

```
uv run python app.py --hot
```

```
  Tina4 Python v3.0.0
  HTTP server running at http://0.0.0.0:7145
  Hot-patching enabled (jurigged)
```

Now edit any route handler and save. The function is updated in place -- without restarting the server. The next request uses the new code immediately.

When Hot-Patching Cannot Help

Hot-patching works for function body changes. It does not work for:

• Adding or removing route decorators (requires restart)

• Changing module-level variables (requires restart)

• Modifying class definitions (may require restart)

• Changing `.env` (requires restart)

For these changes, the server does a full reload automatically.

9. Request Inspector

The request inspector in the dev dashboard shows every HTTP request that has hit the server. For each request, you see:

• **Timestamp** -- When the request arrived

• **Method** -- GET, POST, PUT, DELETE, etc.

• **URL** -- The full URL including query parameters

• **Status** -- The HTTP status code returned

• **Time** -- How long the request took to process

• **Body** -- The request body (for POST/PUT)

• **Response** -- The response body

• **Headers** -- Request and response headers

• **Queries** -- All database queries executed during the request

No more `print()` statements scattered through your code. The inspector shows what happened. Every request. Every detail.

Filtering Requests

The inspector supports filtering by:

• URL pattern (e.g., `/api/` to see only API requests)

• HTTP method (e.g., only POST requests)

• Status code (e.g., only 500 errors)

• Time range (e.g., requests in the last 5 minutes)

10. SQL Query Runner

The dev dashboard includes a SQL query runner that lets you execute queries directly against your database. This is useful for:

• Inspecting data during development

• Running ad-hoc queries to debug issues

• Testing SQL before writing it in your route handlers

• Examining table schemas

```
SELECT * FROM products WHERE category = 'Electronics' ORDER BY price DESC;
```

The results are displayed in a table with column headers, row numbers, and data type indicators. You can copy results, export as CSV, or run another query.

The query runner only works when `TINA4_DEBUG=true` and requires the console token. It is completely disabled in production.

11. Exercise: Debug a Failing Route

Your colleague wrote a route that is failing in mysterious ways. Use the dev tools to find and fix the bug.

Setup

Create `src/routes/buggy.py`:

```
from tina4_python.core.router import get, post
from tina4_python.cache import cache_get, cache_set

@get("/api/buggy/users")
async def buggy_users(request, response):
    users = cache_get("all_users")
    if users:
        return response({"users": users, "source": "cache"})

    # Bug 1: This query has an error
    db = Database.get_connection()
    users = db.fetch_all("SELCT * FROM users ORDER BY name")

    cache_set("all_users", users, ttl=300)
    return response({"users": users, "source": "database"})


@post("/api/buggy/users")
async def buggy_create_user(request, response):
    body = request.body

    # Bug 2: Missing validation
    user = User()
    user.name = body["name"]
    user.email = body["email"]
// ...
```

Requirements

1. Open the dev dashboard at `http://localhost:7145/tina4/console`

2. Hit the `GET /api/buggy/users` endpoint and find Bug 1 using the error overlay

3. Hit the `POST /api/buggy/users` endpoint without a body and find Bug 2 using the request inspector

4. Fix all three bugs:

• Bug 1: Fix the SQL syntax error

• Bug 2: Add validation for required fields

• Bug 3: Return 201 instead of 200

Solution

```
from tina4_python.core.router import get, post
from tina4_python.cache import cache_get, cache_set

@get("/api/buggy/users")
async def buggy_users(request, response):
    users = cache_get("all_users")
    if users:
        return response({"users": users, "source": "cache"})

    db = Database.get_connection()
    users = db.fetch_all("SELECT * FROM users ORDER BY name")  # Fixed: SELCT -> SELECT

    cache_set("all_users", users, ttl=300)
    return response({"users": users, "source": "database"})


@post("/api/buggy/users")
async def buggy_create_user(request, response):
    body = request.body

    # Fixed: Added validation
    if not body.get("name") or not body.get("email"):
        return response({"error": "Name and email are required"}, 400)

    user = User()
// ...
```

The dev tools made it easy: the error overlay showed the SQL syntax error with the exact line, the request inspector showed the missing body causing a `KeyError`, and you knew to fix the status code from the API conventions covered in Chapter 3.

12. Gotchas

1. Dev Dashboard Accessible Without Token

Problem: Anyone on your network can access the dev dashboard.

Cause: `TINA4_CONSOLE_TOKEN` is not set, or it is set to an empty string.

Fix: Always set a console token: `TINA4_CONSOLE_TOKEN=a-strong-random-token`. In production, set `TINA4_DEBUG=false` to disable the dashboard entirely.

2. Live Reload Causes Connection Drops

Problem: WebSocket connections drop every time you save a file.

Cause: The server restarts on file change, which closes all active connections. This is expected behavior for live reload.

Fix: Use hot-patching (`--hot` flag) instead of live reload for WebSocket development. Hot-patching updates function bodies without restarting the server, so connections stay open.

3. Error Overlay Shows in Production

Problem: Users see Python stack traces when errors occur.

Cause: `TINA4_DEBUG=true` is set in the production environment.

Fix: Set `TINA4_DEBUG=false` in production. The error overlay is replaced by a clean error page, and the full details are logged to `logs/error.log`.

4. Request Inspector Slows Down the Server

Problem: The server becomes slower as it runs longer.

Cause: The request inspector stores every request in memory. After thousands of requests, memory usage grows.

Fix: The inspector automatically limits storage to the last 1000 requests. If you notice slowness, clear the inspector from the dashboard. In production, the inspector is disabled.

5. SQL Runner Executes Destructive Queries

Problem: Someone ran `DROP TABLE users` in the SQL query runner.

Cause: The SQL runner executes any valid SQL query, including destructive ones. There is no confirmation step.

Fix: The SQL runner is only available when `TINA4_DEBUG=true` and requires the console token. Never leave the dashboard accessible in production. For sensitive development databases, use a read-only database connection for the SQL runner.

6. Hot-Patching Does Not Pick Up New Routes

Problem: You added a new route decorator but it does not respond to requests.

Cause: Hot-patching (jurigged) can update function bodies but cannot register new decorators. Adding `@get("/new-route")` requires the decorator to execute, which only happens at import time.

Fix: Restart the server when adding or removing routes. Hot-patching is for modifying the behavior of existing routes, not for adding new ones.

7. Template Changes Not Reflected

Problem: You edited a template but the page still shows the old version.

Cause: Template caching is enabled (`TINA4_CACHE_TEMPLATES=true`). The compiled template is served from cache.

Fix: In development, set `TINA4_CACHE_TEMPLATES=false` (this is the default when `TINA4_DEBUG=true`). If you manually enabled template caching, disable it for development.

Chapter 19: CLI & Scaffolding

1. Getting a New Developer Up to Speed

Monday morning. A new developer joins your team. You hand them the repo URL. By 10am they have a running project, a new database model, CRUD routes, a migration, and a deployment to staging. All from the command line. No documentation scavenger hunt. No boilerplate copy-paste.

The Tina4 CLI is a single Rust binary that manages all four Tina4 frameworks (PHP, Python, Ruby, Node.js). The commands are identical across languages. Learn the CLI for Python. You know it for PHP.

2. tina4 init -- Project Scaffolding

You saw this in Chapter 1, but let us look at it in detail.

```
tina4 init my-project

Creating Tina4 project in ./my-project ...
  Detected language: Python (pyproject.toml)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created src/migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-project
// ...
```

Language Detection

The CLI detects the language from existing files in the directory:

| File Present | Language |

|----------|----------|

| `composer.json` | PHP |

| `pyproject.toml` or `requirements.txt` | Python |

| `Gemfile` | Ruby |

| `package.json` | Node.js |

If no language-specific file exists, the CLI asks you:

```
tina4 init my-project

No language detected. Which language?
  1. PHP
  2. Python
  3. Ruby
  4. Node.js
> 2

Creating Tina4 Python project in ./my-project ...
```

Explicit Language Selection

Skip the prompt by specifying the language:

```
tina4 init python my-project
```

This creates a Python project directly, including `pyproject.toml`, `app.py`, and the full directory structure.

Init into an Existing Directory

If you already have a project and want to add Tina4 structure:

```
cd existing-project
tina4 init .
```

The CLI only creates files and directories that do not already exist. It never overwrites existing files.

3. tina4 serve -- Dev Server

```
tina4 serve

  Tina4 Python v3.0.0
  HTTP server running at http://0.0.0.0:7145
  WebSocket server running at ws://0.0.0.0:7145
  Live reload enabled
  Press Ctrl+C to stop
```

`tina4 serve` detects the language and starts the appropriate server. For Python, it runs `uv run python app.py` with live reload enabled.

Options

```
tina4 serve --port 8080        # Custom port
tina4 serve --host 127.0.0.1   # Bind to localhost only
tina4 serve --production       # Production mode (no live reload, debug off)
```

Direct Python Execution

You can also start the server directly with Python:

```
uv run python app.py
```

This is identical to `tina4 serve` but gives you more control over the Python runtime.

4. tina4 generate -- Code Generation

The `generate` command creates boilerplate code for common patterns. Every generated file follows Tina4 conventions and is immediately functional.

Generate a Model

```
tina4 generate model Product

Created src/orm/product.py

  class Product(ORM):
      table_name = "products"
      id: int
      name: str
      created_at: str
```

The model is ready to use. Add your fields and run migrations.

Generate a Route

```
tina4 generate route products

Created src/routes/products.py

  @get("/api/products")
  @get("/api/products/{product_id}")
  @post("/api/products")
  @put("/api/products/{product_id}")
  @delete("/api/products/{product_id}")
```

A complete CRUD route file with all five REST endpoints, proper imports, and response handling.

Generate a Migration

```
tina4 generate migration create_products_table

Created src/migrations/20260322120000_create_products_table.sql

  -- UP
  CREATE TABLE products (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT NOT NULL,
      created_at TEXT DEFAULT CURRENT_TIMESTAMP
  );

  -- DOWN
  DROP TABLE IF EXISTS products;
```

The migration file is timestamped for ordering. Add your column definitions to the `UP` section.

Generate Middleware

```
tina4 generate middleware rate_limit

Created src/middleware/rate_limit.py

  async def rate_limit(request, response, next_handler):
      # Add your middleware logic here
      return await next_handler(request, response)
```

Generate All at Once

```
tina4 generate model Product --with-route --with-migration

Created src/orm/product.py
Created src/routes/products.py
```

```
Created src/migrations/20260322120000_create_products_table.sql
```

This creates the model, a CRUD route file, and a migration -- all wired together and ready to use.

5. tina4 doctor -- Health Check

The `doctor` command checks your project for common issues:

```
tina4 doctor

Tina4 Doctor -- Checking your project...

  [OK] Python 3.12.0 detected
  [OK] uv package manager found
  [OK] tina4_python package installed (v3.0.0)
  [OK] .env file exists
  [OK] Database connection: sqlite:///data/app.db
  [OK] Database is accessible
  [OK] src/routes/ directory exists (3 route files)
  [OK] src/orm/ directory exists (2 model files)
  [OK] src/templates/ directory exists (5 templates)
  [OK] src/public/ directory exists (static files served)
  [OK] tests/ directory exists (4 test files)
  [WARN] No migrations found in src/migrations/
  [OK] AI context: Claude Code detected, CLAUDE.md present
  [OK] .gitignore includes .env, data/, logs/

  12 checks passed, 1 warning, 0 errors
```

Doctor checks:

• Python version and package manager

• Tina4 package installation and version

• `.env` file existence and critical variables

• Database connectivity

• Directory structure

• Missing files or configurations

• AI tool context files

• Git configuration

The warnings give you actionable advice. If your database is not configured, it tells you exactly what to add to `.env`.

6. tina4 test -- Running Tests

```
tina4 test

Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product
```

```
  2 tests, 2 passed, 0 failed (0.12s)
```

This runs all tests in the `tests/` directory. See Chapter 17 for full testing documentation.

Test Options

```
tina4 test --file tests/test_product.py           # Specific file
tina4 test --file tests/test_product.py --method test_create  # Specific method
tina4 test --verbose                                # Show assertion details
```

7. tina4 routes -- Route Listing

See all registered routes in your project:

```
tina4 routes

Registered Routes:

  Method  Path                        Handler                     Middleware
  ------  --------------------------  --------------------------  ----------
  GET     /health                     health_check                -
  GET     /api/products               list_products               ResponseCache:300
  GET     /api/products/{product_id}  get_product                 -
  POST    /api/products               create_product              auth_middleware
  PUT     /api/products/{product_id}  update_product              auth_middleware
  DELETE  /api/products/{product_id}  delete_product              auth_middleware
  GET     /api/auth/login             -                           -
  POST    /api/auth/login             login                       -
  GET     /admin                      admin_dashboard             -

  9 routes registered
```

This is useful for verifying that your routes are registered correctly and for finding the handler function for a specific URL.

8. tina4 migrate -- Database Migrations

Run pending migrations:

```
tina4 migrate

Running migrations...
  [UP] 20260322000100_create_users_table.sql
  [UP] 20260322000200_create_products_table.sql
  [UP] 20260322000300_add_category_to_products.sql

  3 migrations applied
```

Rollback

```
tina4 migrate --down

Rolling back last migration...
  [DOWN] 20260322000300_add_category_to_products.sql

  1 migration rolled back
```

Status

```
tina4 migrate --status
```

```
Migration Status:

  Status    Migration
  --------  ----------------------------------------
  Applied   20260322000100_create_users_table.sql
  Applied   20260322000200_create_products_table.sql
  Applied   20260322000300_add_category_to_products.sql
  Pending   20260322000400_create_orders_table.sql

  3 applied, 1 pending
```

9. Exercise: Scaffold a Feature in 5 Commands

Scaffold a complete "Customer" feature from scratch using only CLI commands.

Requirements

Starting from an existing Tina4 Python project, run exactly 5 commands to create:

1. A Customer ORM model with name, email, phone, and company fields

2. A CRUD route file with all five REST endpoints

3. A migration that creates the customers table

4. Run the migration to create the table

5. Run the tests to verify everything works

Expected Commands

```
# 1. Generate the model with route and migration
tina4 generate model Customer --with-route --with-migration

# 2. Edit the model to add fields (this is manual, not a CLI command)
# Add fields to src/orm/customer.py

# 3. Edit the migration to add columns (this is manual)
# Add columns to the migration file

# 4. Run the migration
tina4 migrate

# 5. Run the doctor to verify everything is set up
tina4 doctor
```

Solution

Command 1: Generate everything:

```
tina4 generate model Customer --with-route --with-migration
```

Command 2: Edit `src/orm/customer.py`:

```
from tina4_python.orm import ORM

class Customer(ORM):
    table_name = "customers"

    id: int
    name: str
    email: str
```

```
    phone: str
    company: str
    created_at: str
```

Command 3: Edit the migration file to add columns:

```
-- UP
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT,
    company TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_customers_email ON customers(email);

-- DOWN
DROP TABLE IF EXISTS customers;
```

Command 4: Run the migration:

```
tina4 migrate
```

```
Running migrations...
  [UP] 20260322120000_create_customers_table.sql

  1 migration applied
```

Command 5: Verify with doctor:

```
tina4 doctor
```

```
  [OK] src/orm/ directory exists (3 model files)
  [OK] src/routes/ directory exists (4 route files)
  [OK] Database connection: sqlite:///data/app.db
  ...
```

Now test the API:

```
curl -X POST http://localhost:7145/api/customers \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Corp", "email": "alice@corp.com", "phone": "+1-555-0100", "company": "Alice Corp"}'
```

```
{
  "id": 1,
  "name": "Alice Corp",
  "email": "alice@corp.com",
  "phone": "+1-555-0100",
  "company": "Alice Corp",
  "created_at": "2026-03-22 12:00:00"
}
```

10. Gotchas

1. tina4 Command Not Found

Problem: Running `tina4` gives "command not found".

Cause: The Tina4 CLI is not installed or not in your PATH.

Fix: Install the CLI: `curl -fsSL https://tina4.com/install.sh | sh`. Verify with `tina4 --version`. If installed but not found, add the installation directory to your PATH.

2. Wrong Language Detected

Problem: `tina4 init` creates a PHP project instead of Python.

Cause: A `composer.json` file exists in the directory (perhaps from a previous project).

Fix: Use explicit language selection: `tina4 init python my-project`. Or delete the conflicting language file before running `init`.

3. Generated Files Overwrite Existing Code

Problem: Running `tina4 generate route products` overwrites your custom route file.

Cause: The generate command creates files at fixed paths. If the file already exists, it is overwritten.

Fix: The CLI warns you before overwriting. Always check if the file exists first. If you need to regenerate, rename the existing file first: `mv src/routes/products.py src/routes/products_backup.py`.

4. Migration Order Issues

Problem: A migration fails because it references a table that does not exist yet.

Cause: Migration files are run in alphabetical (timestamp) order. If migration B depends on a table created by migration A, but A has a later timestamp, B runs first and fails.

Fix: Use consistent timestamps. The `tina4 generate migration` command uses the current timestamp, so generating migrations in order ensures correct execution order. If you need to fix ordering, rename the migration files to adjust their timestamps.

5. tina4 serve Uses Wrong Port

Problem: `tina4 serve` starts on port 7145 but you need port 8080.

Cause: The default port is 7145 unless overridden.

Fix: Set it in `.env`: `TINA4_PORT=8080`. Or pass it as a flag: `tina4 serve --port 8080`. The `.env` value takes precedence over the default, and the command-line flag overrides everything.

6. Doctor Shows False Warnings

Problem: `tina4 doctor` warns about missing migrations, but your project does not use migrations.

Cause: Doctor checks for common conventions. If your project uses a different approach (like manual schema management), it still warns about missing migrations.

Fix: These are warnings, not errors. You can ignore warnings that do not apply to your project. Doctor is a guide, not a gatekeeper.

7. Generate Creates Python 3.12+ Syntax

Problem: Generated code uses `type: int` syntax that does not work in Python 3.10.

Cause: The generator targets the latest Python version supported by Tina4.

Fix: Use Python 3.12 or later with Tina4 Python. Check your Python version with `python --version`. If you must use an older version, modify the generated code to use compatible syntax.

Chapter 20: Deployment

1. From Development to Production

The app works on `localhost:7145`. Now it needs to run 24/7 on a real server. Handle 10,000 concurrent users. Survive server restarts. Not leak memory. The gap between "works on my machine" and "works in production" is where projects stumble.

This chapter covers everything for a production deployment: environment configuration, ASGI server setup, Docker packaging, health checks, graceful shutdown, log rotation, and scaling.

2. Production .env Configuration

The first step is configuring your `.env` for production. Development defaults are optimized for debugging. Production defaults are optimized for performance and security.

Create a production `.env`:

```
# Core
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
TINA4_PORT=7145

# Database
DATABASE_URL=sqlite:///data/app.db

# Security
CORS_ORIGINS=https://yourdomain.com
JWT_SECRET=your-long-random-secret-at-least-32-characters
TINA4_RATE_LIMIT=120

# Performance
TINA4_CACHE_TEMPLATES=true
TINA4_MINIFY_HTML=true

# Console (disabled in production)
TINA4_CONSOLE=false
```

Key Differences from Development

| Setting | Development | Production | Why |

|---------|------------|------------|-----|

| `TINA4_DEBUG` | `true` | `false` | Hides stack traces, disables toolbar |

| `TINA4_LOG_LEVEL` | `ALL` | `WARNING` | Reduces log noise |

| `CORS_ORIGINS` | `*` | Your domain | Prevents cross-origin abuse |

| `TINA4_CONSOLE` | `true` | `false` | Hides dev dashboard |

| `TINA4_CACHE_TEMPLATES` | `false` | `true` | Caches compiled templates |

| `TINA4_MINIFY_HTML` | `false` | `true` | Reduces response size |

Sensitive Values

Production secrets never go into version control. The `.env` file is gitignored by default. For deployment, use environment variables from your hosting platform, CI/CD secrets, or a secrets manager.

```
# Docker: pass env vars at runtime
docker run -e JWT_SECRET=your-secret -e DATABASE_URL=sqlite:///data/app.db my-app

# Fly.io: set secrets
fly secrets set JWT_SECRET=your-secret

# Railway: use the dashboard or CLI
railway variables set JWT_SECRET=your-secret
```

3. ASGI Server Auto-Detection

Tina4 Python auto-detects ASGI servers at startup. ASGI servers provide production-grade performance with:

• Multiple worker processes

• Async request handling

• Graceful shutdown

• Better error recovery

How Auto-Detection Works

When you run `uv run python app.py` or `tina4 serve --production`, Tina4 checks for installed ASGI servers:

1. If `uvicorn` is installed, it uses uvicorn with optimal settings

2. If `hypercorn` is installed, it uses hypercorn

3. If neither is installed, it falls back to the built-in development server

```
# Install uvicorn for production
uv add uvicorn

# With uvicorn installed
uv run python app.py

  Tina4 Python v3.0.0
  Server: uvicorn (4 workers)
  Running at http://0.0.0.0:7145

# Without uvicorn
uv run python app.py

  Tina4 Python v3.0.0
  Server: built-in (development)
  Running at http://0.0.0.0:7145
  WARNING: Do not use the built-in server in production
```

Uvicorn Configuration

You can fine-tune uvicorn through environment variables:

```
TINA4_WORKERS=4
TINA4_WORKER_TIMEOUT=30
TINA4_KEEP_ALIVE=5
```

Or pass options directly:

```
uvicorn app:app --workers 4 --host 0.0.0.0 --port 7145
```

How Many Workers?

A good starting point is `(2 * CPU cores) + 1`:

```
import multiprocessing
workers = (2 * multiprocessing.cpu_count()) + 1
```

| CPU Cores | Workers | Use Case |

|----------|---------|----------|

| 1 | 3 | Small VPS, hobbyist |

| 2 | 5 | Small production app |

| 4 | 9 | Medium production app |

| 8 | 17 | High-traffic app |

4. Docker Deployment

Docker is the most portable deployment path. Your app runs the same way on your laptop, in CI, and on the production server.

Dockerfile

Create `Dockerfile`:

```
FROM python:3.12-slim

# Install uv for fast package management
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

WORKDIR /app

# Copy dependency files first (for better layer caching)
COPY pyproject.toml uv.lock ./

# Install dependencies
RUN uv sync --frozen --no-dev

# Copy application code
COPY . .

# Create directories for data and logs
RUN mkdir -p data logs

# Expose port
EXPOSE 7145

# Health check
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:7145/health || exit 1
// ...
```

.dockerignore

Create `.dockerignore`:

```
.git
.env
__pycache__
*.pyc
.pytest_cache
htmlcov
.claude
node_modules
data/*.db
logs/*.log
.venv
```

Building and Running

```
# Build the image
docker build -t my-tina4-app .

# Run the container
docker run -d \
  --name my-app \
  -p 7145:7145 \
  -e JWT_SECRET=your-production-secret \
  -e DATABASE_URL=sqlite:///data/app.db \
  -v $(pwd)/data:/app/data \
  -v $(pwd)/logs:/app/logs \
  my-tina4-app
```

Docker Compose

For a complete setup with the app and supporting services, use Docker Compose.

Create `docker-compose.yml`:

```
services:
  app:
    build: .
    ports:
      - "7145:7145"
    environment:
      - TINA4_DEBUG=false
      - TINA4_LOG_LEVEL=WARNING
      - JWT_SECRET=${JWT_SECRET}
      - DATABASE_URL=sqlite:///data/app.db
      - TINA4_CACHE_BACKEND=redis
      - TINA4_CACHE_HOST=redis
    volumes:
      - app-data:/app/data
      - app-logs:/app/logs
    depends_on:
      - redis
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:7145/health"]
      interval: 30s
      timeout: 5s
      retries: 3

  redis:
// ...
```

```
# Start everything
docker compose up -d

# View logs
```

```
docker compose logs -f app

# Stop everything
docker compose down
```

5. Health Checks

Production deployments need a health check endpoint so load balancers, container orchestrators, and monitoring tools can verify the app is running.

Create a health check route:

```
from datetime import datetime, timezone
from tina4_python.core.router import get

@get("/health")
async def health_check(request, response):
    db = Database.get_connection()
    db_ok = False

    try:
        db.fetch_one("SELECT 1")
        db_ok = True
    except Exception:
        pass

    status = "ok" if db_ok else "degraded"
    status_code = 200 if db_ok else 503

    return response({
        "status": status,
        "version": "1.0.0",
        "database": "connected" if db_ok else "disconnected",
        "timestamp": datetime.now(timezone.utc).isoformat()
    }, status_code)
```

This endpoint:

• Returns `200` when everything is healthy

• Returns `503` when the database is down (so the load balancer stops routing traffic)

• Includes version information for deployment tracking

• Runs fast (no heavy queries, no authentication)

6. Graceful Shutdown

Deploy a new version. The old process must stop. Graceful shutdown finishes active requests before terminating.

Tina4 handles this when it receives a `SIGTERM` signal (the standard shutdown signal from Docker, Kubernetes, and systemd):

1. Stop accepting new connections

2. Wait for active requests to complete (up to 30 seconds)

3. Close database connections

4. Flush logs

5. Exit

Configuring Shutdown Timeout

```
TINA4_SHUTDOWN_TIMEOUT=30
```

If active requests do not finish within the timeout, they are terminated forcefully. Set this based on your longest expected request time. For most applications, 30 seconds is generous.

Docker Stop Grace Period

Docker sends `SIGTERM`, waits for the grace period, then sends `SIGKILL`. Match the Docker grace period to your shutdown timeout:

```
services:
  app:
    stop_grace_period: 30s
```

7. Log Rotation

In production, logs grow indefinitely unless rotated. Tina4 writes logs to `logs/app.log` and `logs/error.log`.

Using logrotate (Linux)

Create `/etc/logrotate.d/tina4`:

```
/app/logs/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 www-data www-data
    postrotate
        kill -USR1 $(cat /app/tina4.pid) 2>/dev/null || true
    endscript
}
```

This rotates logs daily, keeps 14 days of history, and compresses old logs. The `USR1` signal tells Tina4 to reopen its log files after rotation.

Docker Logging

Docker captures stdout/stderr automatically. Configure log rotation in the Docker daemon or compose file:

```
services:
  app:
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "5"
```

This keeps at most 50MB of logs (5 files of 10MB each).

8. Reverse Proxy with Nginx

In production, Tina4 runs behind Nginx. Nginx handles:

• SSL/TLS termination (HTTPS)

• Static file serving (faster than Python)

• Request buffering

• Rate limiting

• WebSocket proxying

Create `/etc/nginx/sites-available/my-app`:

```
server {
    listen 80;
    server_name yourdomain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    # Static files (served by Nginx, faster than Python)
    location /css/ {
        alias /app/src/public/css/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    location /js/ {
        alias /app/src/public/js/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
// ...
```

9. Exercise: Docker Deploy

Deploy a Tina4 Python application using Docker.

Requirements

1. Create a `Dockerfile` that:

• Uses `python:3.12-slim` as the base image

• Installs dependencies with `uv`

• Copies the application code

• Exposes port 7145

• Includes a health check

• Runs the app with `uv run python app.py`

2. Create a `docker-compose.yml` that:

• Builds and runs the app

• Starts a Redis container for caching

• Mounts volumes for data persistence

• Sets environment variables for production

3. Create a `/health` endpoint that checks database connectivity

4. Build, run, and verify:

```
# Build
docker compose build

# Start
docker compose up -d

# Test health
curl http://localhost:7145/health

# Test the app
curl http://localhost:7145/api/products

# View logs
docker compose logs -f app

# Stop
docker compose down
```

Solution

The Dockerfile and docker-compose.yml are shown in sections 4 above. The health check route is shown in section 5. Combine them all in your project, then:

```
docker compose up -d --build
```

```
[+] Building 12.3s
[+] Running 2/2
  ✔ Container redis    Started
  ✔ Container my-app   Started
```

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "version": "1.0.0",
  "database": "connected",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

The app is running in production mode, with Redis caching, persistent data volumes, automatic restarts, and health monitoring.

10. Gotchas

1. .env Not Loaded in Docker

Problem: Environment variables from `.env` are not available in the container.

Cause: Docker does not automatically read `.env` files. The `.env` file is in `.dockerignore` (as it should be -- never ship secrets in the image).

Fix: Pass environment variables via `docker run -e`, `docker-compose.yml` environment section, or an `env_file` directive. For secrets, use Docker secrets or your platform's secret management.

2. SQLite Database Lost on Container Restart

Problem: All data disappears when the container restarts.

Cause: The SQLite database file is inside the container. When the container is recreated, the file is lost.

Fix: Mount a volume for the data directory: `-v $(pwd)/data:/app/data`. In Docker Compose, use a named volume: `volumes: [app-data:/app/data]`.

3. WebSocket Connections Drop Behind Nginx

Problem: WebSocket connections fail or drop immediately when behind Nginx.

Cause: Nginx does not proxy WebSocket by default. It treats the upgrade request as a regular HTTP request.

Fix: Add WebSocket proxy headers in your Nginx config:

```
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
proxy_read_timeout 86400;
```

4. Built-in Server Used in Production

Problem: The server logs show "WARNING: Do not use the built-in server in production".

Cause: No ASGI server (uvicorn or hypercorn) is installed. The built-in server is single-threaded and not designed for production load.

Fix: Install uvicorn: `uv add uvicorn`. Tina4 auto-detects it and uses it with multiple workers.

5. Static Files Slow

Problem: CSS, JS, and images load slowly in production.

Cause: Python is serving static files. Every static file request goes through the Python process, which is much slower than a dedicated web server.

Fix: Serve static files from Nginx (see the Nginx config in section 8). Nginx serves static files from memory-mapped files, which is orders of magnitude faster than Python.

6. Memory Usage Grows Over Time

Problem: The container's memory usage increases steadily until it crashes with OOM (out of memory).

Cause: Memory leaks in your application -- unclosed database connections, growing caches without TTL, accumulating request data in global variables.

Fix: Set TTLs on all cache entries. Close database connections properly. Avoid storing request data in module-level variables. Use `docker stats` to monitor memory usage. Set memory limits in Docker Compose: `deploy: {resources: {limits: {memory: 512M}}}`.

7. Container Starts Before Database Is Ready

Problem: The app crashes on startup because the database is not ready yet.

Cause: Docker Compose starts services in parallel. The app container starts before the database container is fully initialized.

Fix: For SQLite, this is not an issue (the file is created automatically). For PostgreSQL or MySQL, use a startup script that waits for the database, or use Docker Compose `healthcheck` on the database service with `depends_on: {db: {condition: service_healthy}}`.

Chapter 21: Building a Complete App

1. Putting It All Together

Twenty chapters of building blocks. Routing. Templates. Databases. ORM. Authentication. Middleware. Queues. WebSocket. Caching. Frontend. GraphQL. Testing. Dev tools. CLI scaffolding. Deployment. Now all of it comes together in one application.

TaskFlow -- a task management system with:

• User registration and JWT authentication

• Task creation, assignment, and tracking

• A dashboard with real-time updates

• Email notifications when tasks are assigned

• Caching for dashboard performance

• A full test suite

• Docker deployment

Not a toy project. A complete, production-ready application. Every major Tina4 feature working together.

2. Planning the App

Models

| Model | Table | Fields |

|-------|-------|--------|

| User | users | id, name, email, password_hash, role, created_at |

| Task | tasks | id, title, description, status, priority, created_by, assigned_to, due_date, completed_at, created_at, updated_at |

Relationships

• A User has many Tasks (created by them)

• A User has many Tasks (assigned to them)

• A Task belongs to a User (creator)

• A Task belongs to a User (assignee)

Routes

| Method | Path | Description | Auth |

|--------|------|-------------|------|

| POST | /api/auth/register | Register a new user | public |

| POST | /api/auth/login | Login, get JWT | public |

| GET | /api/profile | Get current user profile | secured |

| GET | /api/tasks | List tasks (with filters) | secured |

| GET | /api/tasks/{id} | Get a single task | secured |

| POST | /api/tasks | Create a task | secured |

| PUT | /api/tasks/{id} | Update a task | secured |

| DELETE | /api/tasks/{id} | Delete a task | secured |

| GET | /api/dashboard/stats | Dashboard statistics | secured |

| GET | /admin | Dashboard HTML page | public |

Templates

• `base.html` -- Base layout with sidebar and topbar

• `dashboard.html` -- Dashboard with stats, task list, quick actions

• `login.html` -- Login page

• `register.html` -- Registration page

3. Step 1: Init Project and Set Up Database

```
tina4 init python taskflow
cd taskflow
uv sync
```

Update `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=dev-token
JWT_SECRET=taskflow-dev-secret-change-in-production
JWT_EXPIRY=86400
```

Create Migrations

Create `src/migrations/20260322000100_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);

-- DOWN
DROP TABLE IF EXISTS users;
```

Create `src/migrations/20260322000200_create_tasks_table.sql`:

```
-- UP
CREATE TABLE tasks (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    status TEXT NOT NULL DEFAULT 'pending',
    priority TEXT NOT NULL DEFAULT 'medium',
    created_by INTEGER NOT NULL,
    assigned_to INTEGER,
    due_date TEXT,
    completed_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id)
);

CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_assigned_to ON tasks(assigned_to);
CREATE INDEX idx_tasks_created_by ON tasks(created_by);

-- DOWN
DROP TABLE IF EXISTS tasks;
```

Run migrations:

```
tina4 migrate
```

```
Running migrations...
  [UP] 20260322000100_create_users_table.sql
  [UP] 20260322000200_create_tasks_table.sql

  2 migrations applied
```

4. Step 2: ORM Models

Create `src/orm/user.py`:

```
from tina4_python.orm import ORM
import hashlib
import hmac
import os

class User(ORM):
    table_name = "users"
    auto_crud = True

    id: int
    name: str
    email: str
    password_hash: str
    role: str
    created_at: str

    def set_password(self, password):
        salt = os.urandom(32)
        key = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100000)
        self.password_hash = salt.hex() + ":" + key.hex()

    def verify_password(self, password):
        if not self.password_hash:
            return False
        salt_hex, key_hex = self.password_hash.split(":")
// ...
```

Create `src/orm/task.py`:

```
from tina4_python.orm import ORM

class Task(ORM):
    table_name = "tasks"
    auto_crud = True

    id: int
    title: str
    description: str
    status: str
    priority: str
    created_by: int
    assigned_to: int
    due_date: str
    completed_at: str
    created_at: str
    updated_at: str

    STATUSES = ["pending", "in_progress", "completed", "cancelled"]
    PRIORITIES = ["low", "medium", "high", "urgent"]
```

5. Step 3: Authentication Routes

Create `src/routes/auth.py`:

```
import os
import jwt
import time
from tina4_python.core.router import post, get

@post("/api/auth/register")
async def register(request, response):
    body = request.body

    if not body.get("name") or not body.get("email") or not body.get("password"):
        return response({"error": "Name, email, and password are required"}, 400)

    # Check for existing user
    existing = User()
    results = existing.select("*", "email = :email", {"email": body["email"]})
    if results:
        return response({"error": "Email already registered"}, 409)

    user = User()
    user.name = body["name"]
    user.email = body["email"]
    user.set_password(body["password"])
    user.role = "user"
    user.save()

// ...
```

6. Step 4: Auth Middleware

Create `src/middleware/auth.py`:

```
import os
```

```
import jwt

async def auth_middleware(request, response, next_handler):
    auth_header = request.headers.get("Authorization", "")

    if not auth_header.startswith("Bearer "):
        return response({"error": "Authentication required"}, 401)

    token = auth_header.replace("Bearer ", "")
    secret = os.getenv("JWT_SECRET", "dev-secret")

    try:
        payload = jwt.decode(token, secret, algorithms=["HS256"])
        request.user = payload
        request.user_id = payload["user_id"]
    except jwt.ExpiredSignatureError:
        return response({"error": "Token expired"}, 401)
    except jwt.InvalidTokenError:
        return response({"error": "Invalid token"}, 401)

    return await next_handler(request, response)
```

7. Step 5: Task CRUD Routes

Create `src/routes/tasks.py`:

```
from datetime import datetime, timezone
from tina4_python.core.router import get, post, put, delete
from tina4_python.cache import cache_get, cache_set, cache_delete

@get("/api/tasks", middleware=["auth_middleware"])
async def list_tasks(request, response):
    status = request.query.get("status")
    assigned_to = request.query.get("assigned_to")
    page = int(request.query.get("page", 1))
    limit = int(request.query.get("limit", 20))
    offset = (page - 1) * limit

    task = Task()
    where = "1=1"
    params = {}

    if status:
        where += " AND status = :status"
        params["status"] = status
    if assigned_to:
        where += " AND assigned_to = :assigned_to"
        params["assigned_to"] = int(assigned_to)

    tasks = task.select("*", where, params, order_by="created_at DESC",
                        limit=limit, offset=offset)
// ...
```

8. Step 6: Dashboard Stats with Caching

Create `src/routes/dashboard.py`:

```
from tina4_python.core.router import get, template
from tina4_python.cache import cache_get, cache_set
```

```
@get("/api/dashboard/stats", middleware=["auth_middleware"])
async def dashboard_stats(request, response):
    cached = cache_get("dashboard:stats")
    if cached:
        return response({**cached, "source": "cache"})

    db = Database.get_connection()

    total = db.fetch_one("SELECT COUNT(*) as count FROM tasks")
    pending = db.fetch_one("SELECT COUNT(*) as count FROM tasks WHERE status = 'pending'")
    in_progress = db.fetch_one("SELECT COUNT(*) as count FROM tasks WHERE status = 'in_progress'")
    completed = db.fetch_one("SELECT COUNT(*) as count FROM tasks WHERE status = 'completed'")
    overdue = db.fetch_one(
        "SELECT COUNT(*) as count FROM tasks WHERE due_date < date('now') AND status != 'completed'"
    )
    users = db.fetch_one("SELECT COUNT(*) as count FROM users")

    recent = db.fetch_all(
        "SELECT t.*, u.name as assignee_name FROM tasks t "
        "LEFT JOIN users u ON t.assigned_to = u.id "
        "ORDER BY t.created_at DESC LIMIT 10"
    )
// ...
```

9. Step 7: WebSocket Real-Time Updates

Create `src/routes/task_ws.py`:

```
import json
from tina4_python.core.router import websocket, push_to_websocket

@websocket("/ws/tasks")
async def task_updates(connection, event, data):
    if event == "open":
        await connection.send(json.dumps({
            "type": "connected",
            "message": "Listening for task updates"
        }))

    if event == "message":
        message = json.loads(data)
        if message.get("type") == "ping":
            await connection.send(json.dumps({"type": "pong"}))


async def push_task_update(action, task):
    """Push a task update to all connected WebSocket clients."""
    await push_to_websocket("/ws/tasks", json.dumps({
        "type": "task_update",
        "action": action,
        "task": task.to_dict()
    }))
```

10. Step 8: Email Notifications

Create `src/routes/notifications.py`:

```
from tina4_python.messenger import Messenger
from tina4_python.core.router import template
```

```
async def notify_assignee(task):
    """Send email notification when a task is assigned."""
    assignee = User()
    assignee.load(task.assigned_to)

    if not assignee.id:
        return

    creator = User()
    creator.load(task.created_by)

    html_body = template("emails/task-assigned.html",
        assignee_name=assignee.name,
        task_title=task.title,
        task_description=task.description or "No description",
        task_priority=task.priority,
        task_due_date=task.due_date or "No due date",
        creator_name=creator.name if creator.id else "Unknown",
        task_url=f"http://localhost:7145/admin#task-{task.id}"
    )

    mailer = Messenger()
// ...
```

Create `src/templates/emails/task-assigned.html`:

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 20px; text-align: center;">
        <h2 style="margin: 0;">New Task Assigned</h2>
    </div>
    <div style="padding: 20px; background: white;">
        <p>Hi {{ assignee_name }},</p>
        <p><strong>{{ creator_name }}</strong> assigned you a new task:</p>

        <div style="background: #f8f9fa; padding: 16px; border-radius: 8px; border-left: 4px solid #1a1a2e;">
            <h3 style="margin-top: 0;">{{ task_title }}</h3>
            <p>{{ task_description }}</p>
            <table style="width: 100%;">
                <tr><td><strong>Priority:</strong></td><td>{{ task_priority }}</td></tr>
                <tr><td><strong>Due:</strong></td><td>{{ task_due_date }}</td></tr>
            </table>
        </div>

        <p style="margin-top: 16px;">
            <a href="{{ task_url }}" style="background: #1a1a2e; color: white; padding: 10px 20px; text-decora
                View Task
            </a>
        </p>
// ...
```

11. Step 9: Dashboard Template

Create `src/templates/dashboard.html`:

```
{% extends "base.html" %}

{% block title %}TaskFlow Dashboard{% endblock %}

{% block content %}
```

```
<h1>Dashboard</h1>

<div id="stats" class="row mb-4">
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Total</h6><h2 id="stat-total">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Pending</h6><h2 id="stat-pending">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">In Progress</h6><h2 id="stat-progress">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Completed</h6><h2 id="stat-completed">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Overdue</h6><h2 id="stat-overdue" class="text-danger">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Users</h6><h2 id="stat-users">--</h2>
// ...
```

12. Step 10: Tests

Create `tests/test_taskflow.py`:

```
import uuid
import json
from tina4_python.test import Test, assert_equal, assert_true, assert_not_none

class TaskFlowTest(Test):

    def _register_and_login(self):
        email = f"test-{uuid.uuid4().hex[:8]}@example.com"
        self.post("/api/auth/register", {
            "name": "Test User",
            "email": email,
            "password": "TestPassword123!"
        })
        login_resp = self.post("/api/auth/login", {
            "email": email,
            "password": "TestPassword123!"
        })
        return json.loads(login_resp.body)["token"]

    def test_register(self):
        resp = self.post("/api/auth/register", {
            "name": "Alice",
            "email": f"alice-{uuid.uuid4().hex[:8]}@example.com",
            "password": "SecurePass123!"
        })
// ...
```

Run the tests:

```
tina4 test
```

```
Running tests...

  TaskFlowTest
    [PASS] test_register
    [PASS] test_login
```

```
    [PASS] test_create_task
    [PASS] test_list_tasks
    [PASS] test_update_task_status
    [PASS] test_delete_task
    [PASS] test_dashboard_stats
    [PASS] test_unauthorized_access

  8 tests, 8 passed, 0 failed (0.84s)
```

13. Step 11: Docker Deployment

Create `Dockerfile`:

```
FROM python:3.12-slim

COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

WORKDIR /app

COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev

COPY . .
RUN mkdir -p data logs

EXPOSE 7145

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:7145/health || exit 1

CMD ["uv", "run", "python", "app.py"]
```

Create `docker-compose.yml`:

```
services:
  app:
    build: .
    ports:
      - "7145:7145"
    environment:
      - TINA4_DEBUG=false
      - JWT_SECRET=${JWT_SECRET:-change-me-in-production}
      - DATABASE_URL=sqlite:///data/app.db
      - TINA4_CACHE_BACKEND=redis
      - TINA4_CACHE_HOST=redis
    volumes:
      - taskflow-data:/app/data
      - taskflow-logs:/app/logs
    depends_on:
      - redis
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    restart: unless-stopped

volumes:
  taskflow-data:
  taskflow-logs:
```

Deploy:

```
docker compose up -d --build
```

Verify:

```
curl http://localhost:7145/health
```

```
{"status": "ok", "version": "1.0.0", "database": "connected"}
```

TaskFlow is running in production. Authenticated APIs. Real-time WebSocket updates. Email notifications. Cached dashboard stats. Tested. Dockerized. Every major Tina4 feature, working together in a single application.

14. What You Built

This chapter used every major concept from the book:

| Feature | Chapter |

|---------|---------|

| Route decorators (`@get`, `@post`, `@put`, `@delete`) | Chapter 2 |

| Request/response handling | Chapter 3 |

| Jinja templates | Chapter 4 |

| Database queries | Chapter 5 |

| ORM models (User, Task) | Chapter 6 |

| JWT authentication | Chapter 7 |

| Auth middleware | Chapter 8 |

| Email notifications (Messenger) | Chapter 13 |

| Cache (dashboard stats) | Chapter 14 |

| Frontend (tina4css dashboard) | Chapter 15 |

| WebSocket (live updates) | Chapter 12 |

| Testing (full test suite) | Chapter 17 |

| Docker deployment | Chapter 20 |

A production-ready task management application. From here, add features: file attachments, team management, calendar integration, reporting. The foundation holds.

Chapter 22: Vibe Coding with AI

Why Tina4 is Built for AI-Assisted Development

Tell your AI assistant: "Add a product catalog with search, pagination, and category filtering." In most frameworks, the AI needs to guess which packages to install, which config files to create, which naming conventions to follow. It hallucinates half of it.

With Tina4, the AI reads one file -- `CLAUDE.md` -- and knows everything. Every method signature. Every import path. Every convention. It generates correct, runnable code on the first try because there is only one way to do things in Tina4.

This is not accidental. Tina4 was designed from the ground up for AI-assisted development.

The Zero-Dependency Advantage

When an AI generates code for Django, it might suggest `from django.core.cache import cache` or `from django.views.decorators.cache import cache_page`. Both exist. Both work differently. The AI guesses.

Tina4 eliminates the guesswork. One cache. One queue. One ORM. One template engine. No ambiguity. No alternatives. No "it depends on which package you installed." The AI knows.

```
# There is only one way to cache in Tina4
from tina4_python.cache import cache_get
products = cache_get("products")

# There is only one way to queue in Tina4
from tina4_python.queue import Queue
queue = Queue(topic="emails")
queue.push({"to": "user@test.com"})

# There is only one way to send email in Tina4
from tina4_python.messenger import Messenger
mailer = Messenger()
mailer.send(to="user@test.com", subject="Welcome", body="Hello!")
```

Zero dependencies means zero confusion for AI.

CLAUDE.md -- The AI's Instruction Manual

Every Tina4 project includes a `CLAUDE.md` file that tells AI assistants exactly how the framework works. It contains:

- Every method signature with parameters and return types
- The project structure convention (src/routes/, src/orm/, src/templates/)
- The .env variable reference
- Code style rules (routes return `response()`, ORM uses `to_dict()`)
- Database connection string formats
- Common gotchas and how to avoid them

When you open a Tina4 project in Claude Code, Cursor, or GitHub Copilot -- the AI reads this file first and immediately understands how to write correct Tina4 code.

Skills -- Deep Framework Knowledge

Beyond CLAUDE.md, Tina4 ships with three AI skill files:

tina4-maintainer

For framework maintenance -- porting features between languages, reviewing PRs, running benchmarks, checking parity.

tina4-developer

For application development -- creating routes, defining models, writing templates, setting up auth, configuring queues.

tina4-js

For frontend development -- tina4-js signals, Tina4Element components, reactive templates, WebSocket client.

These skills are automatically installed when AI tools are detected:

```
# In your project root
AI.install_context(".")  # Detects Claude, Cursor, Copilot, etc.
```

The Convention Advantage

AI thrives on convention. Every Tina4 project follows the same structure. The AI never asks "where should I put this?"

```
src/
  routes/hello.py       -> AI knows: this is a route file
  orm/product.py        -> AI knows: this is an ORM model
  templates/home.html   -> AI knows: this is a template
  middleware/auth.py     -> AI knows: this is middleware
```

Tell the AI "add a User model" and it creates `src/orm/user.py` with the correct field definitions, the correct class structure, and the correct method stubs. Every time.

Real-World Vibe Coding Session

Here is an actual conversation with Claude Code in a Tina4 Python project:

You: "Add a blog with posts and comments. Posts belong to users. Comments belong to posts. I want a REST API and a template page that lists posts."

Claude Code generates:

1. `src/orm/post.py` -- Model with relationships to user and comments

2. `src/orm/comment.py` -- Model with relationship to post

3. `src/routes/api/posts.py` -- Full CRUD API (list, get, create, update, delete)

4. `src/routes/api/comments.py` -- Nested comments API

5. `src/routes/blog.py` -- Template route for the blog page

6. `src/templates/blog.html` -- Page with tina4css cards for each post

7. `src/migrations/20260322_create_posts_and_comments.sql` -- Database schema

All correct. All runnable. First try.

You: "Add pagination to the posts list"

Claude adds `request.query.get("page", 1)` handling and `to_dict()` with limit/offset -- because CLAUDE.md told it exactly how Tina4 pagination works.

You: "Cache the post list for 5 minutes"

Claude adds `cache_get(f"posts_page_{page}")` with `cache_set(f"posts_page_{page}", data, ttl=300)` -- because it knows the cache API.

You: "Send an email when a new comment is posted"

Claude adds `mailer = Messenger(); mailer.send(...)` -- because it knows Messenger reads from .env.

No research. No Stack Overflow. No package debates. Describe the intent. The AI builds it.

Python-Specific Patterns the AI Knows

The CLAUDE.md file teaches AI assistants the correct Python patterns for Tina4:

Route Decorators

```
from tina4_python.core.router import get, post, put, delete, template

@get("/api/products")
async def list_products(request, response):
    products = Product().select("*")
    return response({"products": [p.to_dict() for p in products]})

@post("/api/products")
async def create_product(request, response):
    body = request.body
    product = Product()
    product.name = body["name"]
    product.price = body["price"]
    product.save()
    return response(product.to_dict(), 201)
```

Template Rendering

```
@get("/products")
async def products_page(request, response):
    products = Product().select("*", order_by="name ASC")
    return response(template("products.html", products=[p.to_dict() for p in products]))
```

Middleware Attachment

```
@get("/api/profile", middleware=["auth_middleware"])
async def get_profile(request, response):
    user = User()
```

```
    user.load(request.user_id)
    return response(user.safe_dict())
```

WebSocket Handlers

```
from tina4_python.core.router import websocket

@websocket("/ws/updates")
async def updates_handler(connection, event, data):
    if event == "message":
        await connection.broadcast(data)
```

The AI generates all of these patterns correctly because CLAUDE.md specifies:

• Handlers are always `async def`

• Route handlers always receive `(request, response)`

• WebSocket handlers always receive `(connection, event, data)`

• Responses use `response()` with the data as the first argument

• Templates use `template("name.html", **kwargs)`

Supported AI Tools

Tina4 auto-detects and installs context for:

| Tool | Detection | Context installed |

|------|----------|----------|

| Claude Code | `.claude/` directory | CLAUDE.md + skills |

| Cursor | `.cursor/` directory | .cursorrules |

| GitHub Copilot | `.github/copilot/` | instructions.md |

| Windsurf | `.windsurfrules` file | .windsurfrules |

| Aider | `.aider.conf.yml` | .aider.conf.yml |

| Cline | `.cline/` directory | .clinerules |

| OpenAI Codex | `.codex/` directory | instructions.md |

Run `tina4 doctor` to see which tools are detected in your project.

The AI Chat in Dev Dashboard

The dev dashboard at `/tina4/console` includes an AI chat tab. Enter your Anthropic or OpenAI API key and chat with Claude/GPT about your code directly from the browser.

The AI has full context of your Tina4 project -- routes, models, templates, configuration. Ask it:

• "Why is my /api/users route returning 500?"

• "How do I add WebSocket support?"

• "Write a migration to add an 'avatar' column to users"

The "Ask about this error" button on the error overlay sends the full stack trace and source code to the AI for instant debugging.

Tips for Effective Vibe Coding with Tina4

1. **Be specific about what you want** -- "Add a product API with search by name and price range" works better than "add products"

2. **Let the AI use tina4 generate** -- "Run tina4 generate model Product then add price and category fields" gives the AI a starting point

3. **Reference the .env** -- "Configure the queue to use RabbitMQ" -- the AI knows to update .env with `TINA4_QUEUE_BACKEND=rabbitmq`

4. **Ask for tests** -- "Write tests for the Product API" -- the AI uses Tina4's inline testing framework

5. **Ask for the gallery** -- "Deploy the auth gallery example" -- the AI clicks Try It and you have a working JWT demo

6. **Review, don't rewrite** -- The AI's first attempt is usually 90% correct. Tweak the details instead of starting over.

Exercise

Open your Tina4 Python project in Claude Code (or your preferred AI tool) and try these prompts:

1. "Add a Contact model with name, email, phone, and message fields"

2. "Create a contact form page at /contact with an HTML template using tina4css"

3. "Add an API endpoint that accepts the form submission and saves it to the database"

4. "Send an email notification when a new contact is submitted"

5. "Add rate limiting to the contact form -- max 3 submissions per minute"

Each prompt should generate correct, runnable code on the first try. If it does not, check that CLAUDE.md is in your project root.

Gotchas

1. No CLAUDE.md?

Problem: The AI generates incorrect import paths or uses wrong API patterns.

Fix: Run `tina4 init` or `AI.install_context(".")` to generate the CLAUDE.md file. This gives the AI complete framework knowledge.

2. AI Suggests Wrong Import Paths

Problem: The AI writes `from tina4 import Router` instead of `from tina4_python.core.router import get`.

Fix: The CLAUDE.md might be outdated. Regenerate it with `AI.install_context(".", force=True)`. Check that the file includes the correct import paths for Python.

3. AI Hallucinates a Package

Problem: The AI suggests `pip install tina4-cache-redis` or similar packages that do not exist.

Fix: Remind it: "Tina4 has zero dependencies. Use only built-in features." Everything -- caching, queues, email, WebSocket, GraphQL -- is part of the framework.

4. AI Creates Files in Wrong Location

Problem: The AI creates routes in `routes/` instead of `src/routes/`.

Fix: Tell it: "Follow the src/routes, src/orm, src/templates convention." The CLAUDE.md file specifies this, but some AI tools need a reminder.

5. AI Code Does Not Run

Problem: The generated code has syntax errors or does not follow Tina4 patterns.

Fix: Check that route handlers use `async def` and return `response()`. Check that ORM models extend `ORM`. Check that templates use `template()` for rendering. These are the most common mistakes.

6. AI Uses Synchronous Code

Problem: The AI generates `def handler(request, response)` instead of `async def handler(request, response)`.

Fix: All Tina4 Python route handlers must be `async def`. Tell the AI: "All handlers are async in Tina4 Python."

7. AI Suggests Flask/Django Patterns

Problem: The AI writes `@app.route("/products")` or `return JsonResponse(data)`.

Fix: The AI is falling back to its general Python web framework knowledge. Point it to CLAUDE.md: "Read the CLAUDE.md file for Tina4 Python patterns. Do not use Flask or Django patterns."

The Philosophy

Tina4 was not made compatible with AI coding tools. It was designed for them.

Convention over configuration: the AI knows where things go. Zero dependencies: the AI never chooses between packages. A single CLAUDE.md: the AI has complete framework knowledge. Identical APIs across 4 languages: the AI's knowledge transfers instantly.

You describe the intent. The AI writes the code. You review and refine. That is the workflow Tina4 was built for.

This is not a framework. This is a partnership.